

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

Nghiên cứu kỹ thuật kết hợp nhiều dữ liệu thị giác

GIẢNG VIÊN HƯỚNG DẪN: **Ths. Lã Như Hải**

SINH VIÊN THỰC HIỆN:

1. Nguyễn Công Hào 22DH110918
2. Trần Phạm Hoàng Phong 22DH114682

TP. HỒ CHÍ MINH – 8/2025

LỜI CẢM ƠN

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến Ban Chủ nhiệm Khoa Công Nghệ Thông Tin và các thầy cô đã tận tình giảng dạy, cung cấp cho chúng em nền tảng kiến thức vững chắc trong suốt thời gian học tập. Chúng em xin bày tỏ lòng biết ơn sâu sắc đến **Thầy Tôn Quang Toại**, người đã trực tiếp hướng dẫn, kiên nhẫn chỉ bảo và luôn động viên chúng em vượt qua những khó khăn trong quá trình thực hiện báo cáo. Chúng em cũng xin trân trọng cảm ơn **Thầy Lã Như Hải** đã đồng ý hướng dẫn và tạo điều kiện thuận lợi để chúng em hoàn thành đề tài. Chúng em xin cảm ơn gia đình, bạn bè đã luôn ủng hộ, chia sẻ và khích lệ chúng em trên con đường học tập và nghiên cứu.

LỜI CAM ĐOAN

Chúng tôi xin cam đoan báo cáo này là kết quả nghiên cứu của nhóm chúng tôi dưới sự hướng dẫn của thầy **Lã Như Hải**. Các số liệu, kết quả trong báo cáo là trung thực và chưa từng được sử dụng để bảo vệ ở bất kỳ công trình nào khác. Chúng tôi xin chịu trách nhiệm hoàn toàn trước Khoa và Nhà trường về nội dung báo cáo này.

Tp. Hồ Chí Minh, ngày , tháng , năm

Sinh viên thực hiện

Họ và tên sinh viên 1 : Nguyễn Công Hào

Ký tên :

Họ và tên sinh viên 2 : Trần Phạm Hoàng Phong

Ký tên :

TÓM TẮT KHÓA LUẬN

Trong thực tế, nếu chỉ dùng ảnh RGB thường thì không đủ để phát hiện và định vị đối tượng chính xác trong không gian 3D – đặc biệt khi cần xác định độ sâu hoặc làm việc trong điều kiện môi trường có ánh sáng phức tạp, vật thể xa, bị che khuất một phần. Đề tài này tập trung nghiên cứu kết hợp dữ liệu RGB và LiDAR nhằm khắc phục những giới hạn của ảnh RGB đơn lẻ, với hai mục tiêu cụ thể:

1. Xác định thông tin độ sâu (khoảng cách) của bounding box.
2. Tăng cường độ chính xác phát hiện đối tượng so với YOLO truyền thống.

Các hướng tiếp cận phổ biến để kết hợp RGB và LiDAR phổ biến: Early Fusion (Ghép kênh đầu vào chung), Mid Fusion: (Trích đặc trưng rồi kết hợp lại), Late Fusion (Phát hiện độc lập trên từng nguồn dữ liệu và hợp nhất kết quả).

Khóa luận triển khai pipeline gồm 4 bước chính:

- Bước 1: Chuyển Point Cloud 3D lên ảnh 2D sử dụng ma trận hiệu chỉnh giữa camera và LiDAR. Giữ lại thông tin chiều sâu.
- Bước 2: Phát hiện đối tượng trên ảnh sử dụng YOLOv8n.
- Bước 3: Kết hợp thông tin bounding box 2D với Point Cloud để crop vùng dữ liệu 3D tương ứng.
- Bước 4: Tính toán độ sâu của tọa độ (X, Y, Z) trong vùng Point Cloud đã crop và đã hiệu chỉnh (calibration) về cùng tọa độ camera, cung cấp khoảng cách từ camera đến đối tượng.

Ngoài ra, nhóm thử Early Fusion theo hai cách:

1. Chỉnh Layer đầu YOLOV8n nhận 4 kênh (kết quả kém ổn định).
2. Thiết kế Adapter (vài block conv và skip connect nhằm mục đích nén RGBD về 3 kênh giữ thông tin depth) đạt kết quả khả quan trên moderate và hard.

Kết quả đạt được: Pipeline chiếu hình học kết hợp YOLOV8n và bước crop Point Cloud đã giúp xác định khoảng cách bounding box một cách ổn định và dễ triển khai. Ngoài ra việc thử nghiệm Adapter trong Early Fusion cho thấy khả năng cải thiện độ chính xác ở moderate và hard. Kết quả khẳng định hiệu quả của kết hợp Camera và Lidar trong phát hiện và xác định khoảng cách đối tượng.

MỤC LỤC

LỜI CẢM ƠN	I
LỜI CAM ĐOAN	II
TÓM TẮT KHÓA LUẬN	III
MỤC LỤC.....	IV
DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT	X
DANH MỤC CÁC BẢNG.....	XI
DANH MỤC CÁC HÌNH VẼ, BIỂU ĐỒ	XII
Chương 1. Giới thiệu bài toán.....	1
1.1 Câu hỏi nghiên cứu	1
1.2 Giới thiệu Dataset được sử dụng để giải quyết bài toán	3
1.3 Các phương pháp đánh giá bài toán	3
1.3.1 Phát hiện đối tượng	4
1.3.1.1 AP (Average Precision).....	4
1.3.1.2 mAP (Mean Average Precision)	6
1.3.2 Xác định khoảng cách	7
1.4 Tầm quan trọng	7
1.4.1 Tính mới	7
1.4.2 Tính hữu dụng	8
1.5 Giới hạn nghiên cứu	8
1.6 Bố cục khóa luận	9
Chương 2. Lý thuyết nền tảng.....	10
2.1 Quá trình hình thành ảnh.....	10
2.1.1 Camera Model PinHole.....	10
2.1.1.1 Mô hình Pinhole Camera	10

2.1.1.2 Các thành phần chính.....	10
2.1.1.3 Ma trận nội tại và ngoại tại	11
2.1.1.4 Ma trận chiếu.....	11
2.1.2 Các hệ trục tọa độ thế giới	12
2.1.2.1 Hệ tọa độ thế giới.....	12
2.1.2.2 Hệ tọa độ camera.....	13
2.1.2.3 Hệ tọa độ ảnh	14
2.1.3 Chuyển đổi hệ tọa độ.....	15
2.1.3.1 Hệ tọa độ thế giới sang hệ tọa độ camera	15
2.1.3.2 Hệ tọa độ ảnh sang hệ tọa độ mặt phẳng ảnh.....	15
2.2 Hiệu chỉnh camera.....	16
2.2.1 Hiệu chỉnh camera.....	16
2.2.2 Hiệu chỉnh một camera	16
2.2.3 Hiệu chỉnh hai camera.....	18
2.3 Khái niệm LiDAR.....	19
2.3.1 LiDAR.....	19
2.3.2 Các kiểu LiDAR.....	19
2.3.3 Cấu tạo LiDAR	21
2.3.3.1 Các thành phần chính.....	21
2.3.3.2 Hệ thống định vị và định hướng.....	22
2.3.3.3 Các thành phần hỗ trợ khác.....	22
2.3.4 Nguyên tắc hoạt động của LiDAR.....	22
2.4 Point Cloud.....	23
2.4.1 Khái niệm Point Cloud.....	23
2.4.2 Thư viện Open3D.....	23

2.4.3	Trực quan hóa dữ liệu PC	24
2.4.4	Cách biểu diễn Voxel	24
2.5	Các kỹ thuật xử lý PC cơ bản.....	25
2.5.1	Kỹ thuật lấy mẫu	25
2.5.1.1	Kỹ thuật Radius Outlier	28
2.5.1.2	Kỹ thuật Statistical Outlier.....	29
2.5.2	Kỹ thuật Surface normal	30
2.5.3	Tìm kiếm trên PC	31
2.5.3.1	Tìm kiếm lân cận K-Nearest Neighbors - KNN	31
2.5.3.2	Radius Nearest Neighbors.....	31
2.5.4	Cấu trúc dữ liệu trên PC	32
2.5.4.1	K Demensional Tree	32
2.5.4.2	OCtree.....	33
2.5.4.3	Cấu trúc dữ liệu giúp tìm kiếm hiệu quả.....	35
2.6	Các kỹ thuật xử lý PC nâng cao	35
2.6.1	Registration	35
2.6.2	Các phương pháp Registration.....	36
2.6.2.1	Thuật toán ICP	36
2.6.2.2	Registration nâng cao.....	37
2.7	Perceptron	39
2.7.1	Khái niệm	39
2.7.2	Cấu trúc và nguyên lý hoạt động	40
2.7.3	Ưu điểm.....	41
2.7.4	Nhược điểm.....	41
2.8	Mạng neuron đa tầng.....	41

2.8.1 Khái niệm	41
2.8.2 Cấu trúc và nguyên lý hoạt động	42
2.9 Mạng neuron tích chập.....	42
2.9.1 Khái niệm	42
2.9.2 Phép toán Convolution.....	42
2.9.3 Phép toán Padding.....	43
2.9.4 Phép toán Stride	44
2.10 Một số kiến trúc mạng neuron tích chập hiện đại	44
2.10.1 Mạng VGG.....	44
2.10.2 Mạng ResNet.....	45
2.10.3 Một số kiến trúc khác	46
2.11 Bài toán phát hiện đối tượng	46
2.11.1 Giới thiệu về bài toán.....	46
2.11.2 Mục tiêu	46
2.11.3 Các hướng tiếp cận chính.....	47
2.11.4 Ứng dụng thực tiễn.....	47
Chương 3. Các công trình liên quan	48
3.1 Sự xuất hiện của YOLO và các phiên bản nổi bật	48
3.2 Mô hình YOLO	50
3.2.1 Định nghĩa về YOLO	50
3.2.2 Đặc điểm	50
3.2.3 Input	50
3.2.4 OutPut	50
3.2.5 Kiến trúc YOLO.....	51
3.2.6 Cách hoạt động.....	53

3.2.7 Hàm lỗi.....	53
3.3 Anchor Boxes.....	55
3.4 IoU.....	56
3.5 Non-max suppression (NMS)	57
3.6 YOLOv8.....	58
3.6.1 Input	58
3.6.2 Output.....	59
3.6.3 Kiến trúc YOLOv8.....	59
3.6.4 Hàm lỗi.....	65
3.6.4.1 CIoU Loss	65
3.6.4.2 DFL (Distribution Focal Loss).....	67
3.6.4.3 Binary Cross Entropy cho nhiệm vụ Objectness	67
3.6.4.4 Binary Cross Entropy cho nhiệm vụ Classification.....	68
Chương 4. Giải pháp đề xuất	69
4.1 Phân tích dữ liệu.....	69
4.2 Mô hình	73
4.2.1 Pipeline chính của đề tài	73
4.2.1.1 Vẽ hình kiến trúc.....	74
4.2.1.2 Bảng mô tả chi tiết kiến trúc	74
4.2.1.3 Hàm activation của tầng cuối.....	76
4.2.1.4 Hàm loss.....	76
4.2.2 Pipeline hướng mở rộng của đề tài	77
4.2.2.1 Nhánh sử dụng YOLOv8n thay đổi đầu vào.....	81
4.3 Thực nghiệm chính	81
4.3.1 Thực nghiệm 1	81

4.3.1.1	Đánh giá Yolov8n (COCO-pretrained) trên bộ dữ liệu KITTI.....	81
4.3.1.2	Kết hợp dữ liệu và tính khoảng cách	82
4.3.2	Thực nghiệm 2	85
4.3.2.1	Fine-tune YOLOv8n trên KITTI.....	85
4.3.2.2	Kết hợp dữ liệu và tính khoảng cách	86
4.3.2.3	Thực nghiệm 3	87
4.3.2.4	Kết hợp dữ liệu và tính khoảng cách	87
4.3.3	Thực nghiệm 4	92
4.3.3.1	Kết hợp dữ liệu và tính khoảng cách	92
4.3.4	Thực nghiệm 5	94
4.3.4.1	Kết hợp dữ liệu và tính khoảng cách	94
4.4	Thực nghiệm phụ	97
4.4.1	Nhánh sử dụng tích chập để sinh ảnh có chứa thông tin depth.....	99
4.4.2	Nhánh thay đổi đầu vào của YOLOv8 từ 3 chanel thành 4 chanel.....	103
4.4.3	Tổng hợp kết quả.....	107
4.5	Web demo	107
KẾT LUẬN.....		107
TÀI LIỆU THAM KHẢO.....		113

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
1	PC	Point Cloud
2	Bbox	Bounding Box
3	n_boundingbox	1 hoặc nhiều bounding box
4	n_class	1 hoặc nhiều class
5	AP	Average Precision
6	mAP	Mean Average Precission
7	ICP	Iternative Closest Point

DANH MỤC CÁC BẢNG

Bảng 2.1. So sánh cấu trúc dữ liệu giúp tìm kiếm hiệu quả.....	35
Bảng 2.2. Bảng so sánh kết quả giữa ICP và Colored ICP	38
Bảng 3.1. Mô tả chi tiết CloU Loss.....	66
Bảng 4.1. Backbone: trích xuất đặc trưng từ hình ảnh đầu vào.	74
Bảng 4.2. Neck	75
Bảng 4.3. Head & Detect	76
Bảng 4.4. Adapter	78
Bảng 4.5. Bảng so sánh mAP của các lớp.....	82
Bảng 4.6. Bảng so sánh mAP của các lớp.....	86
Bảng 4. 7.Bảng so sánh mAP của các lớp.....	101
Bảng 4.8.Bảng so sánh mAP của các lớp.....	104
Bảng 4.9. Bảng tổng hợp kết quả mAP.....	107

DANH MỤC CÁC HÌNH VẼ, BIỂU ĐỒ

Hình 1.1. Ví dụ về tính khoảng cách của đối tượng dựa trên bounding box 2D và Point Cloud.....	2
Hình 1.2. Mô hình nhận diện được vật thể bị che khuất một phần.....	3
Hình 1.3. Hình ảnh về TP, FP, FN	4
Hình 1.4. Hình ảnh về bảng tổng hợp tính Precision và Recall	5
Hình 1.5. Hình ảnh vẽ các Precision và Recall.....	6
Hình 1.6. Sử dụng 11-point interpolation	6
Hình 1.7. Kết quả của bảng AP.....	6
Hình 2.1. Hình ảnh về lý thuyết Camera PinHole	10
Hình 2.2. Hình ảnh về các thành phần chính trong camera pinhole	11
Hình 2.3. Hình ảnh minh họa về cách ma trận chiếu giúp chuyển đổi từ 3D sang ảnh 2D	12
Hình 2.4. Hình ảnh minh họa về tọa độ thế giới	13
Hình 2.5. Hình ảnh minh họa về tọa độ của camera	14
Hình 2.6. Hình ảnh minh họa về tọa độ ảnh	15
Hình 2.7. Hình ảnh về tên của LiDAR và Camera được dùng để thu thập.....	21
Hình 2.8. Hình ảnh sau khi trực quan hóa dữ liệu Point Cloud	24
Hình 2.9. Hình ảnh sau khi một file Point Cloud được áp dụng Voxel downsampling	25
Hình 2.10. Hình ảnh sau Point Cloud áp dụng Voxel Grid	26
Hình 2.11. Hình ảnh sau Point Cloud áp dụng Centroid Point	27
Hình 2.12. Hình ảnh sau Point Cloud áp dụng Center Point	27
Hình 2.13. Hình ảnh sau Point Cloud áp dụng Furthest Point Sampling.....	28
Hình 2.14. Hình ảnh Point Cloud sau khi sử dụng tìm kiếm KNN và tô màu.....	31
Hình 2.15. Hình ảnh Point Cloud sau khi sử dụng tìm kiếm Radius Nearest Neighbors và tô màu	32
Hình 2.16. Hình ảnh minh họa về K Demensional Tree.....	32
Hình 2.17. Hình ảnh minh họa về Point Cloud trước khi OCTree	35
Hình 2.18. Hình ảnh minh họa về Point Cloud sau khi OCTree	35
Hình 2.19. Cấu trúc và nguyên lý hoạt động của Preceptron.....	40

Hình 2.20. Ảnh từ mô tả về Multilayer Perceptron	42
Hình 2.21. Hình ảnh minh họa về cơ chế của Convolution	43
Hình 2.22. Hình ảnh minh họa về Valid Padding	43
Hình 2.23. Ảnh minh họa về Same Padding	44
Hình 2.24. Hình ảnh về kiến trúc VGG-16	45
Hình 2.25. Sơ đồ của một khối Residual Block	45
Hình 2.26. Kiến trúc mạng ResNet-50	46
Hình 3.1. Mô tả quá trình phát triển của YOLO từ 2015 đến hiện tại	48
Hình 3.2. Ảnh OutPut mô phỏng của YOLO	51
Hình 3.3. Kiến trúc mô hình YOLO: 24 convolutions	51
Hình 3.4. Hình ảnh quá trình hoạt động của YOLO	53
Hình 3.5. Hình ảnh liên quan đến Anchor Boxes	55
Hình 3.6. Mô tả việc bị trùng lặp mid point	56
Hình 3.7. Hình ảnh về cách tính IoU	57
Hình 3.8. Hình ảnh mô tả về cách thức Non-Max Suppression hoạt động	58
Hình 3.9. Hiệu năng và tham số của YOLOv8 qua từng version	59
Hình 3.10. Bảng chi tiết tổng hợp về các phiên bản YOLOv8 trên tập COCO	60
Hình 3. 11. Kiến trúc của YOLOv8	61
Hình 3.12. Hình ảnh về cấu trúc của C2F	62
Hình 3.13. Hình ảnh về SPPF (Spatial Pyramid Pooling Fast)	63
Hình 3.14. Hình ảnh mô phỏng lại cách tính CIoU Loss	65
Hình 4.1. Hình ảnh về tập phân bổ tập train và test	69
Hình 4.2. Hình ảnh về dữ liệu hình ảnh có trong tập train và test	70
Hình 4.3. Hình ảnh phân bổ dữ liệu giữa tập train và test của Point Cloud	71
Hình 4.4. Ảnh về phân bổ dữ liệu trong tập KITTI	72
Hình 4.5. Hình ảnh về các lớp được sử dụng trong đề tài	73
Hình 4.6. Pipeline chính cho đề tài	74
Hình 4.7. Kiến trúc Yolo v8	74
Hình 4.8. Luồng xử lý của hướng kiểm nghiệm xem depth có thật sự mang lại thông tin hữu dụng.	77

Hình 4. 9. Mô hình RGBD $\rightarrow$ RGB hòa trộn thêm depth rồi sinh ra ảnh gốc sao cho gần giống với ảnh thật mà mang thêm thông tin depth đã được hòa trộn bằng mạng tích chập.	78
Hình 4. 10. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Min.....	84
Hình 4.11. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Max	84
Hình 4.12. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Median	85
Hình 4. 13. Hình ảnh về thực nghiệm 1 khoảng cách sau khi dùng Mean	85
Hình 4.14. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Min.....	86
Hình 4.15. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Max	87
Hình 4.16. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Median	87
Hình 4.17. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Mean	87
Hình 4.18. Hình ảnh về cấu trúc của xe được KITTI sử dụng để thu thập dữ liệu...	89
Hình 4.19. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Min.....	90
Hình 4.20. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Max	90
Hình 4.21. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Median	90
Hình 4.22. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Mean	91
Hình 4.23. Nhận xét	91
Hình 4. 24 Hình ảnh về thực nghiệm 4 khoảng cách sau khi dùng Min.....	92
Hình 4. 25 Hình ảnh về thực nghiệm 4 khoảng cách sau khi dùng Max	93
Hình 4. 26 Hình ảnh về thực nghiệm 4 khoảng cách sau khi dùng Median	93
Hình 4. 27 Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Mean	93
Hình 4. 28 Nhận xét	94
Hình 4. 29 Hình ảnh về video để thực hiện liên tục khoảng cách đối tượng.....	94
Hình 4. 30 Hình ảnh thực nghiệm 5	97
Hình 4. 31 Hình ảnh thực nghiệm 5	97
Hình 4.32. Ảnh Depth sau khi nội suy và tách từ kênh RGBD	98
Hình 4.33 Hình ảnh hàm loss của Apdapter	99
Hình 4.34 Hình ảnh sau khi qua mô hình Apdapter và sinh ảnh	100
Hình 4.35 Hình ảnh trước khi qua mô hình Apdapter và sinh ảnh	100
Hình 4.36 Chi tiết về hàm loss trong quá trình huấn luyện trên data RGB mới được sinh ra	100

Hình 4.37. Kết quả đánh giá trên Benchmark KITTI của lớp Car.....	101
Hình 4.38. Kết quả giá trên Benchmark KITTI của lớp Pedestrian.....	102
Hình 4.39. Kết quả đánh giá trên Benchmark KITTI của lớp Cyclist	102
Hình 4.40. Kết quả train và val loss	104
Hình 4.41. Kết quả đánh giá trên Benchmark KITTI của lớp Car.....	105
Hình 4.42. Kết quả đánh giá trên Benchmark KITTI của lớp Pedestrian.....	105
Hình 4.43. Kết quả đánh giá trên Benchmark KITTI của lớp Cyclist	106
Hình 4.44. Website demo trên streamlit	108

Chương 1. Giới thiệu bài toán

1.1 Câu hỏi nghiên cứu

Trong lĩnh vực thị giác máy tính ứng dụng vào xe tự hành, việc không chỉ phát hiện đối tượng mà còn xác định khoảng cách chính xác đến đối tượng là một yêu cầu then chốt, giúp hệ thống đưa ra các quyết định điều khiển kịp thời như phanh, tránh va chạm, hoặc giữ khoảng cách an toàn. Trong hệ thống perception của xe tự hành hiện đại, camera RGB và cảm biến LiDAR thường được kết hợp để bổ sung thông tin cho nhau: RGB cung cấp thông tin kết cấu, màu sắc, còn LiDAR cung cấp dữ liệu không gian 3D giàu thông tin độ sâu.

Tuy nhiên, việc xử lý ảnh RGB đơn lẻ vẫn còn nhiều hạn chế khi đối tượng bị che khuất, thiếu sáng, hoặc nằm quá xa khiến thông tin độ sâu bị mất. Ngược lại, Point Cloud từ LiDAR lại rất thưa, cần được căn chỉnh chính xác và xử lý nếu muốn kết hợp chúng với hình ảnh. Do đó, việc kết hợp dữ liệu RGB và LiDAR một cách hiệu quả để phát hiện và định vị đối tượng trong không gian thực là một bài toán quan trọng, mang tính thực tiễn cao trong các ứng dụng tự động hóa và xe thông minh.

Bài toán được đặt ra trong đề tài này là phát hiện và định vị các đối tượng (xe hơi, người đi bộ, người chạy xe đạp) trên ảnh RGB đồng bộ với dữ liệu Point Cloud, không chỉ dừng lại ở xác định bounding box 2D mà còn tính toán được khoảng cách từ camera đến đối tượng đó trong không gian 3D (tức là thông tin độ sâu). Nhóm đề xuất một pipeline xử lý gồm chiếu hình học Point Cloud lên ảnh, phát hiện đối tượng bằng mô hình YOLOV8, sau đó kết hợp bounding box với dữ liệu 3D để tính toán khoảng cách.

Bên cạnh mục tiêu chính là xác định độ sâu cho bounding box, đề tài cũng mở rộng đánh giá khả năng tăng cường độ chính xác của thuật toán phát hiện khi đưa thêm thông tin độ sâu vào mô hình, từ đó so sánh hiệu quả giữa YOLO_RGB và YOLO_RGBD. Cách tiếp cận này nhằm kiểm chứng vai trò của thông tin chiều sâu (depth) trong việc nâng cao chất lượng nhận dạng đối tượng trong điều kiện môi trường phức tạp.

Đầu vào của bài toán:

- Ảnh RGB chụp từ camera gắn trên xe

- Dữ liệu Point Cloud 3D thu từ cảm biến LiDAR
- Ma trận hiệu chỉnh (intrinsic và extrinsic) giữa camera và LiDAR
- (Hướng mở rộng) Label của ảnh RGB

Đầu ra của bài toán:

- Các bounding box 2D xác định đối tượng trong ảnh
- Giá trị khoảng cách từ camera đến đối tượng
- (Hướng mở rộng) So sánh độ chính xác giữa YOLO_RGB và YOLO_RGBD

Ví dụ minh họa:

Một chiếc xe tự hành thu được ảnh RGB và Point Cloud tại thời điểm t . Hệ thống phát hiện được hai chiếc xe khác nhau cách camera một khoảng lần lượt là 13.06 mét và 20.49 mét.



Hình 1.1. Ví dụ về tính khoảng cách của đối tượng dựa trên bounding box 2D và Point Cloud





Hình 1.2. Mô hình nhận diện được vật thể bị che khuất một phần

1.2 Giới thiệu Dataset được sử dụng để giải quyết bài toán

Trong đề tài này, nhóm sử dụng bộ dữ liệu KITTI 2D Object Detection – đây là một bộ dữ liệu thực tế thu thập trên xe tự hành ở thành phố Karlsruhe (Đức). KITTI cung cấp ảnh RGB độ phân giải cao được đồng bộ với dữ liệu LiDAR, cùng các nhãn bounding box 2D cho nhiều lớp đối tượng (Car, Pedestrian, Cyclist, v.v).

Dữ liệu đầu vào của đề tài bao gồm:

- Ảnh RGB (định dạng 2D)
- Point Cloud từ cảm biến LiDAR (dạng 3D)
- File calib: Chứa các tham số hiệu chỉnh giữa camera và LiDAR (intrinsic và extrinsic) để ánh xạ chính xác giữa không gian 3D và mặt phẳng 2D.
- File labels: Chứa thông tin về các bounding box của các ảnh RGB mà KITTI đã đánh nhãn sẵn. Phục vụ huấn luyện và đánh giá thuật toán phát hiện đối tượng.

Bộ dữ liệu này rất phổ biến trong nghiên cứu thị giác máy tính cho xe tự hành, cho phép phát triển các phương pháp phát hiện vật thể 2D trên ảnh RGB và đồng thời khai thác thông tin chiều sâu từ Point Cloud để xác định khoảng cách (depth) của các bounding box phát hiện được.

1.3 Các phương pháp đánh giá bài toán

Đề tài này tập trung vào hai mục tiêu chính:

1. Đánh giá khả năng phát hiện đối tượng 2D từ ảnh RGB và ảnh RGBD

2. Đánh giá khả năng xác định khoảng cách (độ sâu) từ camera đến đối tượng bằng dữ liệu Point Cloud.

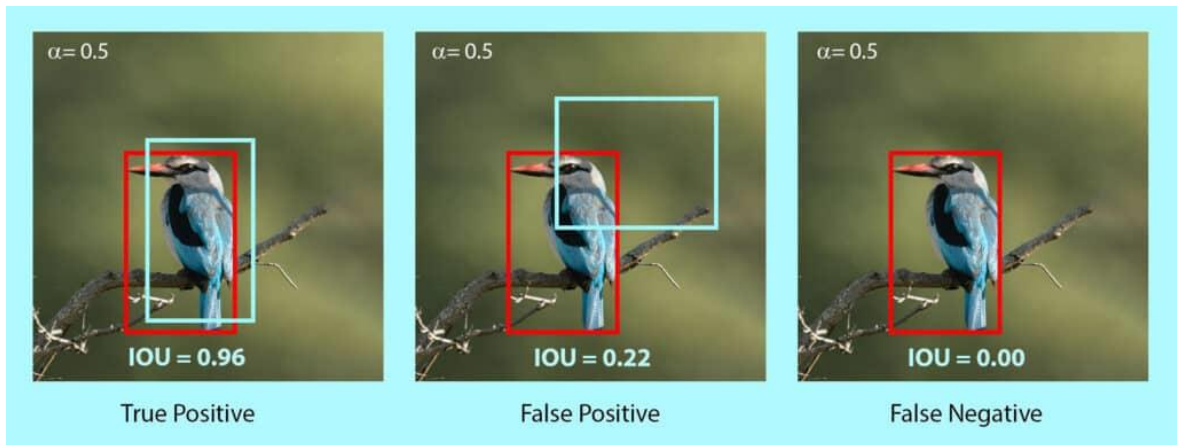
Với mỗi mục tiêu, nhóm sử dụng các chỉ số đánh giá phù hợp:

1.3.1 Phát hiện đối tượng

1.3.1.1 AP (Average Precision)

AP (Average Precision) là thước đo tóm tắt hiệu suất của mô hình phát hiện đối tượng trên tập kiểm tra, dựa trên mối quan hệ giữa Precision và Recall.

- Confusion Matrix trong phát hiện đối tượng thường bao gồm:
 - True Positive (TP): Dự đoán chính xác đối tượng ($IoU > \text{ngưỡng}$)
 - False Positive (FP): Dự đoán sai hoặc dư thừa
 - False Negative (FN): Bỏ sót đối tượng



Hình 1.3. Hình ảnh về TP, FP, FN

Nguồn: [Mean Average Precision \(mAP\) in Object Detection](#)

- *Precision*: Cho biết trong các đối tượng mà mô hình phát hiện, tỷ lệ đúng thực sự là bao nhiêu.

$$Precision = TP / (TP + FP) \quad (1.1)$$

- *Recall*: Cho biết trong tất cả đối tượng thực tế, mô hình tìm ra được bao nhiêu phần trăm.

$$Recall = TP / (TP + FN) \quad (1.2)$$

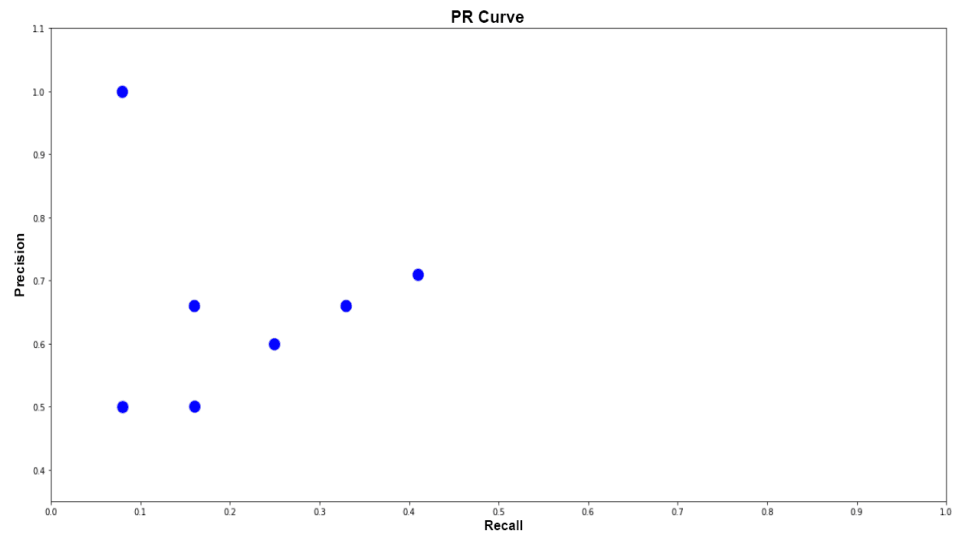
Để tính AP cho một class thì ta thực hiện như sau:

- Bước 1: Gom lại tất cả bounding box của class đó trên tập test
- Bước 2: Sắp xếp theo confidence giảm dần

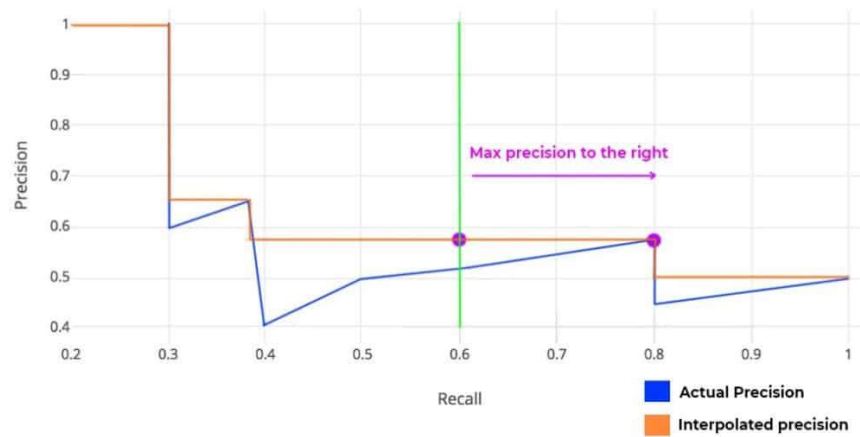
- Bước 3: Đánh nhãn từng box là TP (True Positive) hoặc FP (False Positive)
 - Nếu bounding box khớp với groundtruth ($\text{IoU} > \text{threshold}$ và chưa bị matched): TP
 - Nếu bounding box không khớp hoặc đã khớp nhưng bị trùng (duplicate): FP
- Bước 4: Tính Precision và Recall tại từng mốc
- Bước 5: Vẽ biểu đồ Precision-Recall (PR curve)
- Bước 6: Tính diện tích dưới đường cong PR (AP) sử dụng 11 point interpolation

Preds.	Conf.	Matches	Cumulative TP	Cumulative FP	Precision	Recall
	0.92	TP	1	0	$1/(1+0) = 1$	$1/16 = 0.08$
	0.91	FP	1	1	$1/(1+1) = 0.5$	$1/16 = 0.08$
	0.88	TP	2	1	$2/(2+3) = 0.66$	$2/16 = 0.16$
	0.86	FP	2	2	0.5	0.16
	0.77	TP	3	2	0.6	0.25
	0.63	TP	4	2	0.66	0.33
	0.58	TP	5	2	0.71	0.41

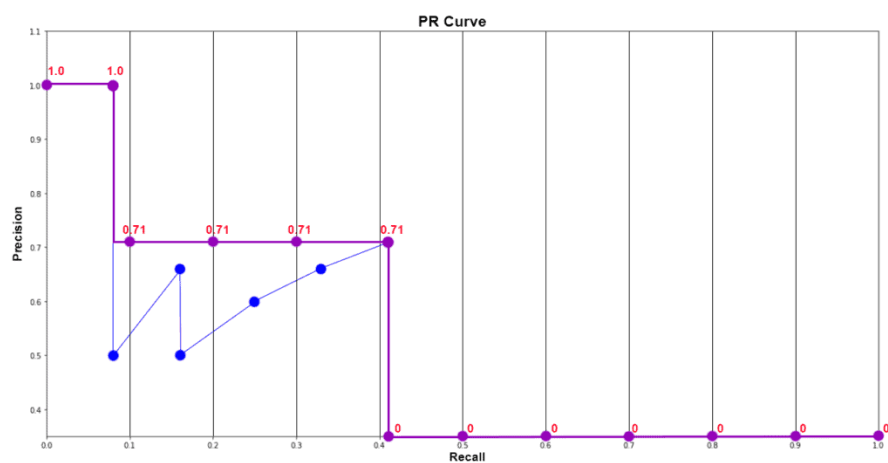
Hình 1.4. Hình ảnh về bảng tổng hợp tính Precision và Recall



Hình 1.5. Hình ảnh vẽ các Precision và Recall



Hình 1.6. Sử dụng 11-point interpolation



Hình 1.7. Kết quả của bảng AP

1.3.1.2 mAP (Mean Average Precision)

- Mean Average Precision (mAP) theo từng lớp (class-wise)

$$mAP = \frac{1}{N_{classes}} \sum_{i=1}^{N_{classes}} AP_i \quad (1.3)$$

- mAP@[.5:.95] theo chuẩn COCO (nhiều mức IoU)

$$mAP@[.5:.95] = \frac{1}{10} \sum_{IoU=0.5}^{0.95} AP_{IoU} \quad (\text{step} = 0.05) \quad (1.4)$$

1.3.2 Xác định khoảng cách

Trong bài toán này, nhóm sử dụng tọa độ (X, Y, Z) của điểm Point Cloud đã được hiệu chỉnh (calibration) và chiếu về tọa độ camera. Sau khi crop vùng dữ liệu Point Cloud theo bounding box 2D dự đoán. Nhóm tính khoảng cách đến đối tượng bằng giá trị trung bình của khoảng cách Euclidean:

$$d = \sqrt{X^2 + Y^2 + Z^2} \quad (1.5)$$

Tuy nhiên, do bộ dữ liệu KITTI không cung cấp ground-truth độ sâu cho từng đối tượng, các chỉ số lỗi như MAE, RMSE không thể được tính chính thức trong khóa luận này.

1.4 Tầm quan trọng

1.4.1 Tính mới

Đề tài có tính mới ở cách kết hợp dữ liệu RGB và LiDAR (Point Cloud) để giải quyết bài toán phát hiện đối tượng kèm xác định khoảng cách trong không gian 3D. Cụ thể, đề tài xây dựng pipeline chiếu hình học để tạo ảnh RGBD đồng bộ, phát hiện bằng Yolov8n, và kết hợp kết quả 2D với Point Cloud để tính toán khoảng cách thực tế. Ngoài ra, đề tài thử nghiệm hướng mở rộng Early Fusion với module Adapter gồm 2 giai đoạn:

1. Học và nén thông tin depth từ 4 kênh RGBD về 3 kênh thông qua vài block Convolution và skip connection nhằm biểu diễn thông tin độ sâu dưới dạng ảnh màu.
2. Sử dụng ảnh biểu diễn trên làm đầu vào để huấn luyện và dự đoán với backbone YoLov8n gốc (không thay đổi cấu trúc backbone).

Hướng này khai thác thông tin depth ngay trong backbone tiêu chuẩn và cho thấy tiềm năng tăng độ chính xác phát hiện ở các điều kiện moderate và hard.

1.4.2 Tính hữu dụng

Bài toán phát hiện đối tượng và xác định khoảng cách đối tượng từ ảnh RGB kết hợp LiDAR có tính ứng dụng cao trong hệ thống xe tự hành, hỗ trợ lái thông minh và robot di động. Việc biết được vị trí 2D trên ảnh cùng với khoảng cách thực tế giúp hệ thống hiểu và lập kế hoạch di chuyển an toàn, tránh va chạm hoặc nhận diện vật thể trong không gian 3D. Đề tài cũng góp phần xây dựng pipeline đơn giản, khả thi và dễ dàng mở rộng trong các bài toán thị giác máy tính thực tế đòi hỏi thông tin về độ sâu.

1.5 Giới hạn nghiên cứu

- Đề tài chỉ phát hiện đối tượng 2D trên ảnh RGB và ước lượng khoảng cách (độ sâu) từ LiDAR, không làm phát hiện 3D Bounding Box (kích thước, góc phương vị).
- Sử dụng thông tin calibration có sẵn từ KITTI, không giải quyết tự động căn chỉnh camera – LiDAR cho hệ thống mới.
- Khoảng cách được xác định từ giá trị tọa độ X, Y, Z của LiDAR đã được căn chỉnh theo camera và đã được crop theo bounding box 2D, không dùng mô hình deep regression để dự đoán chi tiết.
- Không thực hiện phân đoạn đối tượng (Instance Segmentation 3D) trong Point Cloud, chỉ crop thô theo bounding box 2D.
- Phần mở rộng so sánh độ chính xác phát hiện (RGB vs RGBD) chỉ ở mức thử nghiệm ban đầu, chưa tối ưu YOLO hoặc huấn luyện trên tập lớn.
- Không nghiên cứu tốc độ suy luận (FPS) hoặc triển khai real-time trên thiết bị nhúng.
- Không so sánh khoảng cách với ground truth thật vì KITTI không cung cấp độ sâu chuẩn riêng theo cho từng bounding box, chỉ minh họa pipeline tính khoảng cách từ Point Cloud.
- Sử dụng 3 lớp chính để sử dụng và đánh giá: Car, Presiden, Cyclist cho xuyên xuất khóa luận.

1.6 Bố cục khóa luận

Khóa luận này bao gồm 5 chương: Chương 1 giới thiệu bài toán, dữ liệu được sử dụng, phạm vi nghiên cứu và phương pháp đánh giá. Chương 2 trình bày lý thuyết phục vụ phát triển đề tài này gồm xử lý ảnh, point cloud, hiệu chỉnh camera và mạng neuron. Ở chương 3 khảo sát và nói về các công trình liên quan đến bài toán gồm các mô hình phổ biến cho bài toán phát hiện đối tượng. Chương 4 trình bày giải pháp đề xuất, bao gồm phân tích dữ liệu, mô hình được sử dụng và các thực nghiệm liên quan. Ở chương cuối cùng tổng kết kết quả, nêu ứng dụng minh họa, hạn chế và hướng phát triển về đề tài.

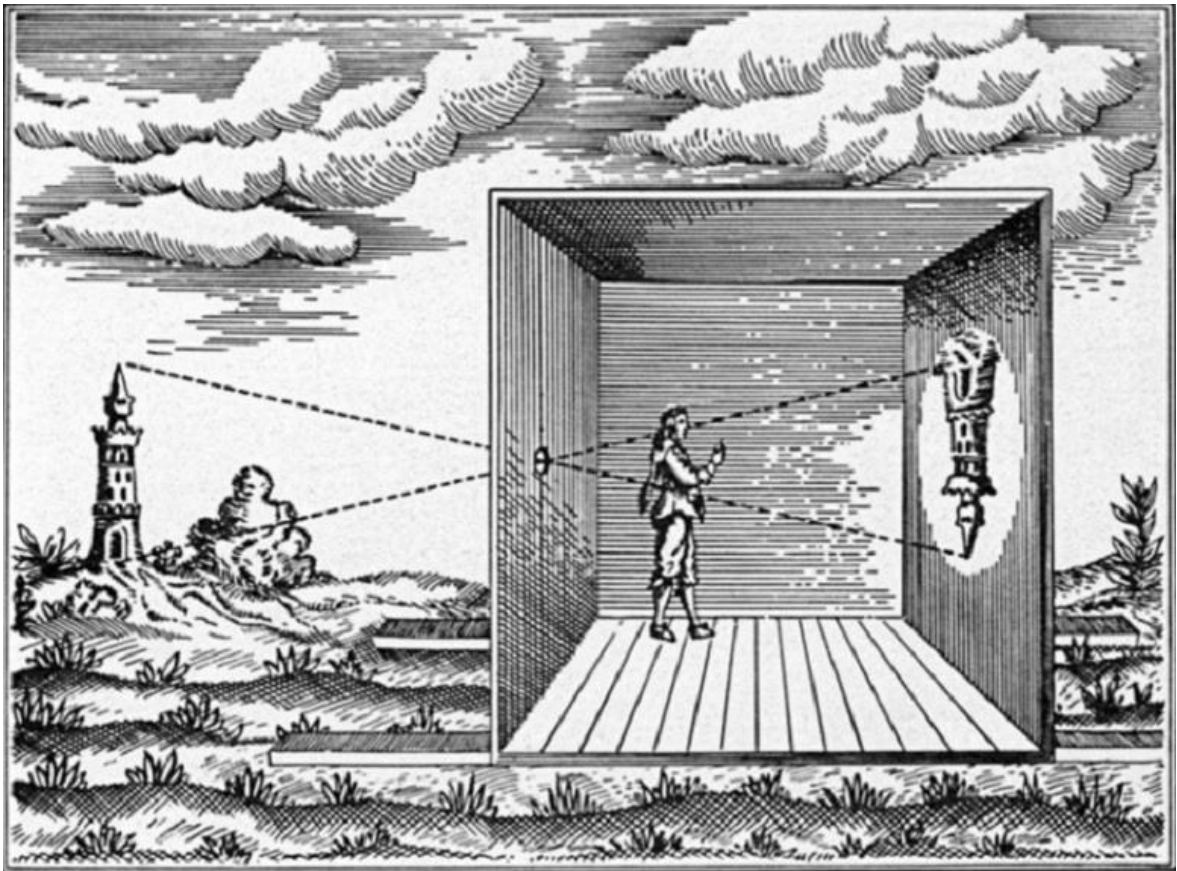
Chương 2. Lý thuyết nền tảng

2.1 Quá trình hình thành ảnh

2.1.1 Camera Model PinHole

2.1.1.1 Mô hình Pinhole Camera

Mô hình Pinhole Camera [1] là một mô hình lý tưởng hóa của camera, giả định rằng ánh sáng đi qua một lỗ nhỏ (Pinhole) tại tâm camera, chiếu lên mặt phẳng để tạo ra ảnh 2D từ đối tượng 3D.



Hình 2.1. Hình ảnh về lý thuyết Camera PinHole

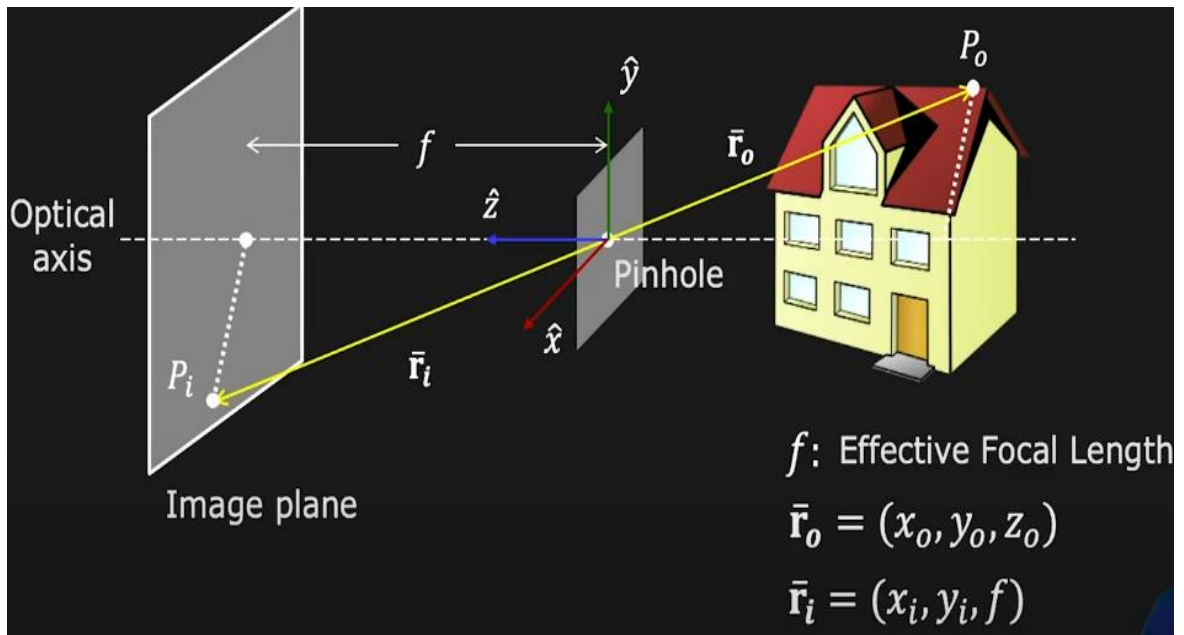
Nguồn: [Make Your Own Camera Obscura](#) | [The Village at Black Creek](#)

2.1.1.2 Các thành phần chính

1. Tâm camera (Camera Center): Điểm mà các tia sáng hội tụ, thường là góc của hệ tọa độ camera
2. Mặt phẳng ảnh (Image Plane): Nơi mà ảnh 2D được tạo ra, cách tâm camera một khoảng gọi là tiêu cự
3. Hệ tọa độ thế giới (World Coordinate): Hệ tọa độ 3D cố định trong không gian

4. Hệ tọa độ camera (Camera Coordinate): Hệ tọa độ 3D gắn với camera, trục z hướng ra ngoài (trục quang học)

5. Hệ tọa độ ảnh (Image Coordinate): Hệ tọa độ trên mặt phẳng, với tọa độ (u, v)



Hình 2.2. Hình ảnh về các thành phần chính trong camera pinhole

Nguồn: [Pinhole and Perspective Projection | Image Formation](#)

2.1.1.3 Ma trận nội tại và ngoại tại

Ma trận ngoại tại $[R|t]$ mô tả vị trí, hướng của camera so với hệ tọa độ thế giới.

- R : Ma trận quay (hướng camera)
- t : là vector tịnh tiến (vị trí camera trong thế giới)

Vai trò: Ma trận ngoại tại giúp chuyển đổi tọa độ của một điểm từ hệ tọa độ thế giới sang hệ tọa độ của camera.

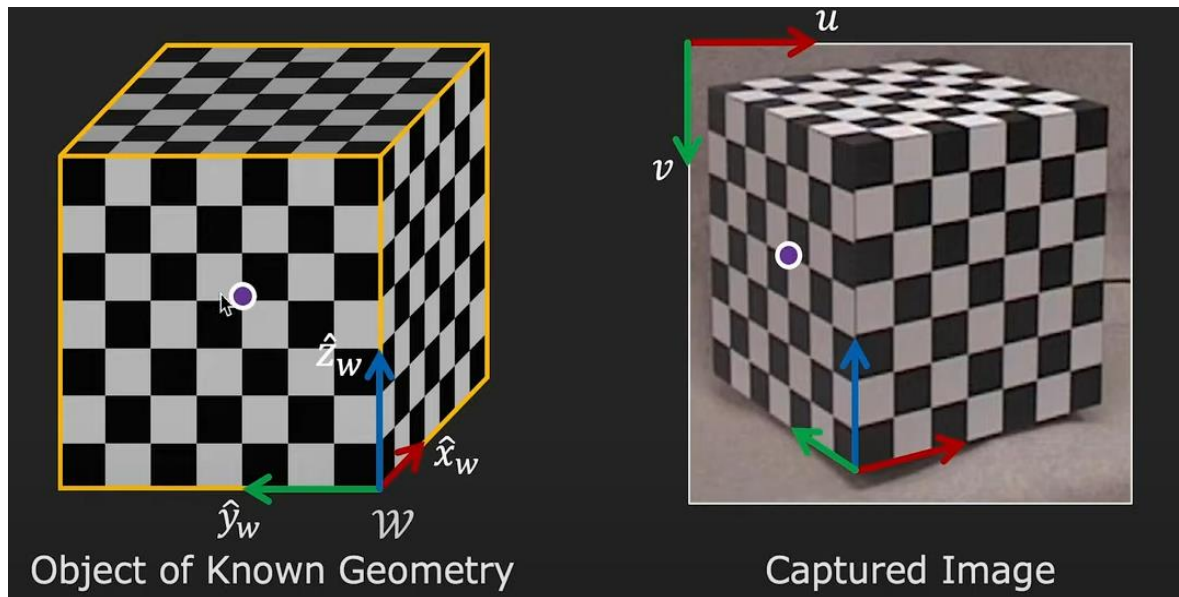
Ma trận nội tại K mô tả đặc tính bên trong của camera bao gồm.

- f : Tiêu cự quyết định ảnh phóng to hay thu nhỏ như thế nào.
- (o_x, o_y) : Tâm ảnh, thường là điểm nằm chính giữa của ảnh.

Vai trò của ma trận nội tại K giúp chiếu điểm từ hệ tọa độ camera (3D) lên ảnh 2D (u, v).

2.1.1.4 Ma trận chiếu

Ma trận chiếu P là sự kết hợp của ma trận nội tại và ngoại tại. Nó có vai trò giúp chiếu trực tiếp một điểm 3D từ hệ tọa độ thế giới lên ảnh 2D.



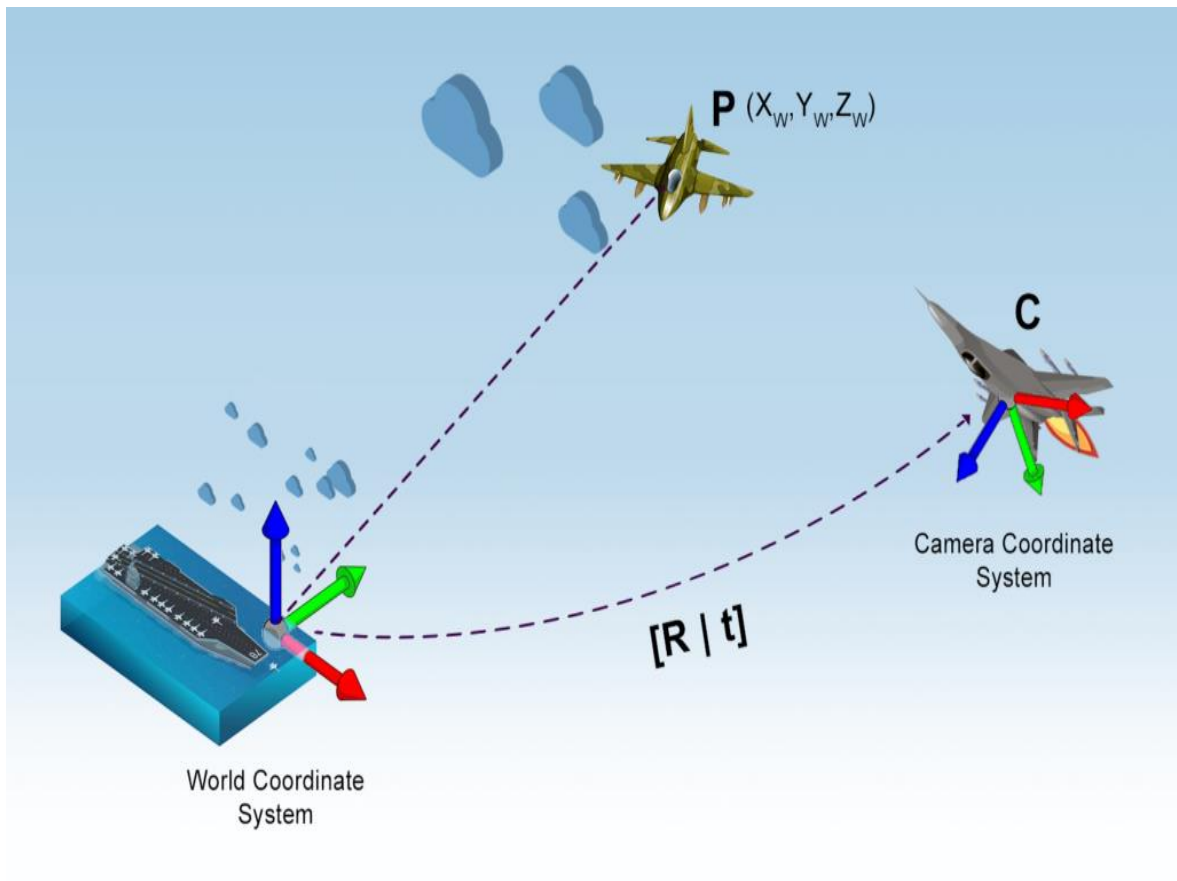
Hình 2.3. Hình ảnh minh họa về cách ma trận chiếu giúp chuyển đổi từ 3D sang ảnh 2D

Nguồn: [Camera Calibration](#) | [Camera Calibration](#)

2.1.2 Các hệ trục tọa độ thế giới

2.1.2.1 Hệ tọa độ thế giới

Định nghĩa: Là hệ tọa độ 3D cố định, độc lập với camera và sử dụng cho thế giới thực với tọa độ được kí hiệu là (X_w, Y_w, Z_w) . Được dùng để mô tả vị trí và hướng của vật thể.



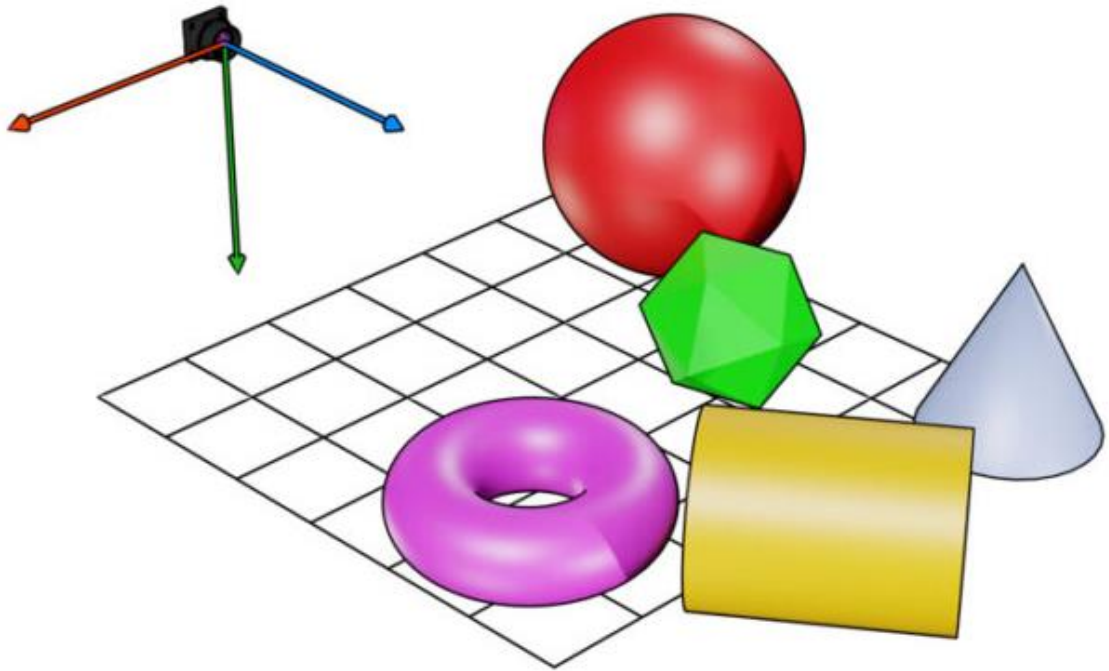
Hình 2.4. Hình ảnh minh họa về tọa độ thế giới

Nguồn: [Matrix Transformations and Coordinate Systems with Python | Sigmoidal](#)

2.1.2.2 Hệ tọa độ camera

Định nghĩa: Là hệ tọa độ 3D gắn với Camera có gốc tại tâm quang học, dùng để mô tả vị trí và hướng tương đối với camera.

Ký hiệu: (X_c, Y_c, Z_c)



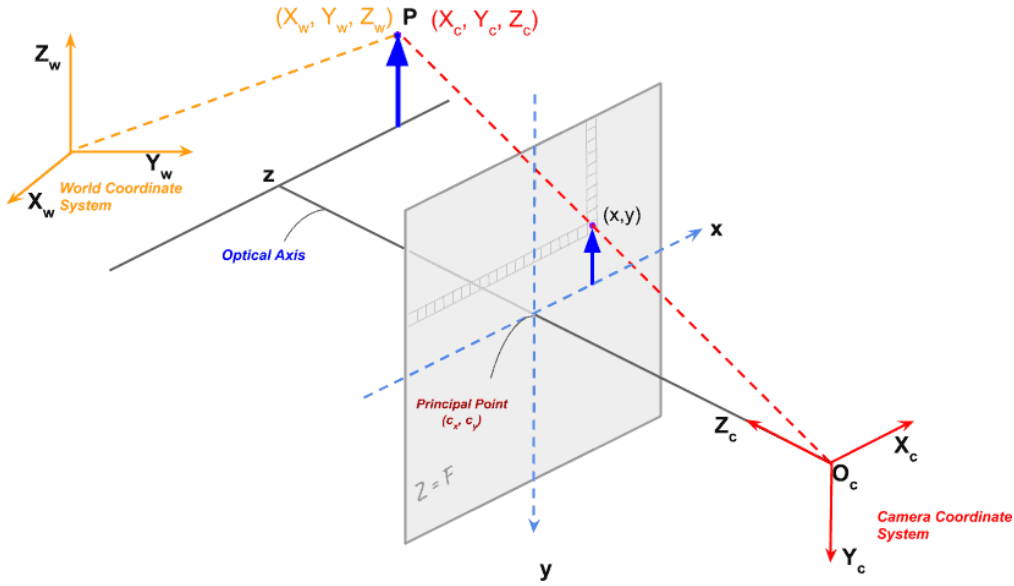
Hình 2.5. Hình ảnh minh họa về tọa độ của camera

Nguồn: [Matrix Transformations and Coordinate Systems with Python | Sigmoidal](#)

2.1.2.3 Hệ tọa độ ảnh

Định nghĩa: Là hệ tọa độ 2D dùng để mô tả vị trí của điểm ảnh trên mặt phẳng ảnh (image plane), sau khi được ánh xạ từ hệ tọa độ camera thông qua phép chiếu phối cảnh (perspective projection).

Tọa độ thường được ký hiệu là: (x, y)



Hình 2.6. Hình ảnh minh họa về tọa độ ảnh

Nguồn: [Geometry of Image Formation | LearnOpenCV #](#)

2.1.3 Chuyển đổi hệ tọa độ

2.1.3.1 Hệ tọa độ thế giới sang hệ tọa độ camera

Mục tiêu: Chuyển đổi một điểm trong không gian thế giới thực thành hệ tọa độ gắn với camera thông qua ma trận ngoại tại [2] [3].

Công thức:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \mathbf{R} \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} + \mathbf{t} \quad (2.1)$$

Hoặc

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = [\mathbf{R} | \mathbf{t}] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (2.2)$$

2.1.3.2 Hệ tọa độ ảnh sang hệ tọa độ mặt phẳng ảnh

Mục tiêu: Chuyển từ hệ tọa độ ảnh sang pixel u, v thông qua ma trận nội tại.

Công thức:

$$\begin{bmatrix} u' \\ v' \\ w' \end{bmatrix} = \mathbf{K} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} \quad (2.3)$$

2.2 Hiệu chỉnh camera

2.2.1 Hiệu chỉnh camera

Là quá trình xác định các thông số bên trong và ngoài của máy ảnh [4]. Nhằm mục đích mô hình hóa chính xác cách camera ghi nhận hình ảnh 3D sang thành ảnh 2D.

2.2.2 Hiệu chỉnh một camera

Nội dung của phần này được tham khảo và tổng hợp từ [4] [5]

1. Image Plane to Image Sensor Mapping

Khi một điểm P trong không gian 3D được chiếu xuống mặt phẳng ảnh (Image Plane), nó có tọa độ liên tục (x, y) tính theo milimét. Tuy nhiên, cảm biến của camera ghi nhận ảnh dưới dạng lưới pixel rời rạc (u,v).

Quá trình chuyển đổi tọa độ “thước đo vật lý” trên image plane sang tọa độ pixel trên sensor bao gồm các bước sau:

- Bước 1: Với điểm tọa độ trong hệ camera (X_c, Y_c, Z_c) dùng tam giác đồng dạng của mô hình PinHole:

$$x = f \frac{X_c}{Z_c}, \quad y = f \frac{Y_c}{Z_c} \quad (2.4)$$

Với f là tiêu cự hiệu dụng (tính theo mm)

- Bước 2: Chuyển từ milimét sang pixel: Giả sử cảm biến có mật độ pixel (pixel/ mm) mỗi chiều là m_x, m_y , cho phép quy đổi:

$$u' = m_x x, \quad v' = m_y y \quad (2.5)$$

2. Hệ tọa độ đồng nhất (Homogeneous Coordinates)

Sử dụng tọa độ đồng nhất giúp biểu diễn các phép biến đổi tuyến tính (quay, tịnh tiến) trong không gian dưới dạng ma trận:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = [R | t] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (2.6)$$

Điều này giúp chuẩn hóa và gộp nhiều phép biến đổi vào một phép nhân ma trận duy nhất. Nhằm thuận tiện hơn cho việc tính toán và lập trình.

3. Ma trận ngoại tại

Ma trận ngoại tại mô tả vị trí và hướng của camera trong không gian thế giới thông qua:

- R : ma trận quay 3×3
- t : vectơ tịnh tiến 3×1

Công thức chuyển đổi:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = R \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} + t \quad (2.7)$$

Hoặc viết gọn trong hệ đồng nhất

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = [R | t] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (2.8)$$

4. Ma trận nội tại

Ma trận nội tại K mô tả các đặc tính vật lý bên trong của camera, như tiêu cự, vị trí tâm ảnh:

$$K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (2.9)$$

Trong đó:

- f_x, f_y : Tiêu cự theo trục x và y , tính bằng pixel
- c_x, c_y : Tọa độ pixel tâm quang học
- γ : Độ lệch trục giữa các trục x và y của cảm biến

5. Mô hình hóa méo ảnh

Camera thực tế luôn có lỗi quang học, gây ra méo ảnh hay biến dạng hình học ảnh đầu ra.

Có hai dạng méo chính:

- Méo xuyên tâm (radial): Ảnh bị “phình” hoặc “lõm” do ống kính cong

$$\begin{aligned} x_{radial} &= x \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \\ y_{radial} &= y \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \end{aligned} \quad (2.10)$$

- Méo tiếp tuyến (tangential): Do sai lệch trục của thấu kính

$$\begin{aligned} x_{tangential} &= 2p_1 xy + p_2 (r^2 + 2x^2) \\ y_{tangential} &= p_1 (r^2 + 2x^2) + 2p_2 xy \end{aligned} \quad (2.11)$$

Tọa độ sau khi méo:

$$x_d = x_{(radial)} + x_{(tangential)}, y_d = y_{(radial)} + y_{(tangential)} \quad (2.12)$$

2.2.3 Hiệu chỉnh hai camera

Định nghĩa: Là quá trình xác định đồng thời các tham số nội tại (intrinsic) và ngoại tại (extrinsic) của hai camera trong hệ thống stereo [6]. Mục tiêu là hiệu chỉnh lại hình ảnh sao cho các điểm tương ứng trong hai ảnh sẽ nằm cùng một hàng ngang (epipolar line).

Mục tiêu: Sau khi hiệu chỉnh, hai ảnh từ camera trái và phải sẽ được chuyển đổi hình học sao cho:

- Các điểm tương ứng nằm cùng một dòng ngang.
- Việc tìm kiếm điểm tương ứng (matching) trong ảnh stereo trở nên dễ dàng hơn.
- Từ đó có thể áp dụng các thuật toán stereo matching để tái dựng bản đồ disparity (khoảng cách lệch). Dựa vào đó suy ra bản đồ độ sâu (depth map).

Quy trình hiệu chỉnh:

1. Hiệu chỉnh từng camera (calibration riêng lẻ):

Sử dụng ảnh chụp từ một bảng hiệu chuẩn (ví dụ ảnh bàn cờ) để xác định các tham số nội tại của từng camera:

- Ma trận nội tại: gồm tiêu cự, tâm ảnh
- Các hệ số biến dạng ống kính

2. Hiệu chỉnh hệ stereo (stereo calibration):

Xác định các tham số mô tả mối quan hệ hình học giữa hai camera:

- Ma trận xoay (R): Biểu diễn sự khác biệt về hướng nhìn giữa hai camera.
- Vector tịnh tiến (T): Biểu diễn khoảng cách không gian giữa hai camera.
- Ma trận thiết yếu (Essential Matrix- E) và ma trận cơ bản (Fundamental Matrix -F):

- Ma trận E hoạt động trong không gian 3D, ánh xạ điểm trong ảnh trái sang ảnh phải thông qua ma trận nội tại.
- Ma trận F hoạt động trong không gian ảnh 2D, ánh xạ điểm ảnh trái sang đường epipolar trong ảnh phải.

Kết quả:

Sau khi hiệu chỉnh, hai ảnh có thể được làm phẳng, sao cho các đường epipolar là các hàng ngang. Điều này giúp giảm bài toán tìm điểm tương trong ảnh stereo xuống 1D, tăng tốc độ và độ chính xác của việc tái dựng độ sâu.

2.3 Khái niệm LiDAR

2.3.1 LiDAR

LiDAR (Light Detection and Ranging): Là công nghệ cảm biến từ xa sử dụng *laser* để đo độ cao và khoảng cách đến các đối tượng như mặt đất, rừng và tòa nhà, ... từ đó tạo ra bản đồ không gian 2D hoặc mô hình 3D của môi trường xung quanh. LiDAR hoạt động bằng cách sử dụng nguồn sáng như tia *cực tím khả kiến* hoặc *gần hồng ngoại*. LiDAR sẽ gửi ra một xung ánh sáng (*Pulse*), sau đó chờ đợi sự trở về (*Return*) [7] [8] [9] và thời gian mà xung laser mất để quay trở lại được sử dụng để tính toán khoảng cách tới đối tượng theo **công thức**:

$$d = \frac{t \times c}{2} \quad (2.13)$$

Với:

c = tốc độ ánh sáng (299,792,458 m/s)

t = thời gian di chuyển (travel time)

d = khoảng cách (distance)

Một số đặc điểm nổi bật:

- Cho phép xây dựng mô hình PC với độ chính xác cao.
- Hoạt động *độc lập* với điều kiện ánh sáng tự nhiên (có thể hoạt động tốt cả ban ngày lẫn ban đêm).
- Được ứng dụng rộng rãi trong nhiều lĩnh vực như hệ thống thông tin địa lý (GIS), ô tô tự lái, robotics, khảo sát và viễn thám.

2.3.2 Các kiểu LiDAR

Phân loại theo ứng dụng và môi trường thu thập:

• **LiDAR địa hình (Topographic LiDAR):** Sử dụng tia laser gần hồng ngoại để khảo sát mặt đất, đánh giá độ cao và xây dựng mô hình số độ cao (DEM) hay mô hình số bề mặt (DSM).

• **LiDAR Metric:** Sử dụng ánh sáng xanh để đo đặc độ sâu trong các môi trường dưới nước (đo độ sâu lòng sông hoặc biển).

• **LiDAR trên cạn (Terrestrial LiDAR):** Được lắp đặt cố định trên mặt đất, dùng để đo đạc các tòa nhà, đặc điểm tự nhiên và thiết lập mô hình 3D của các di tích lịch sử.

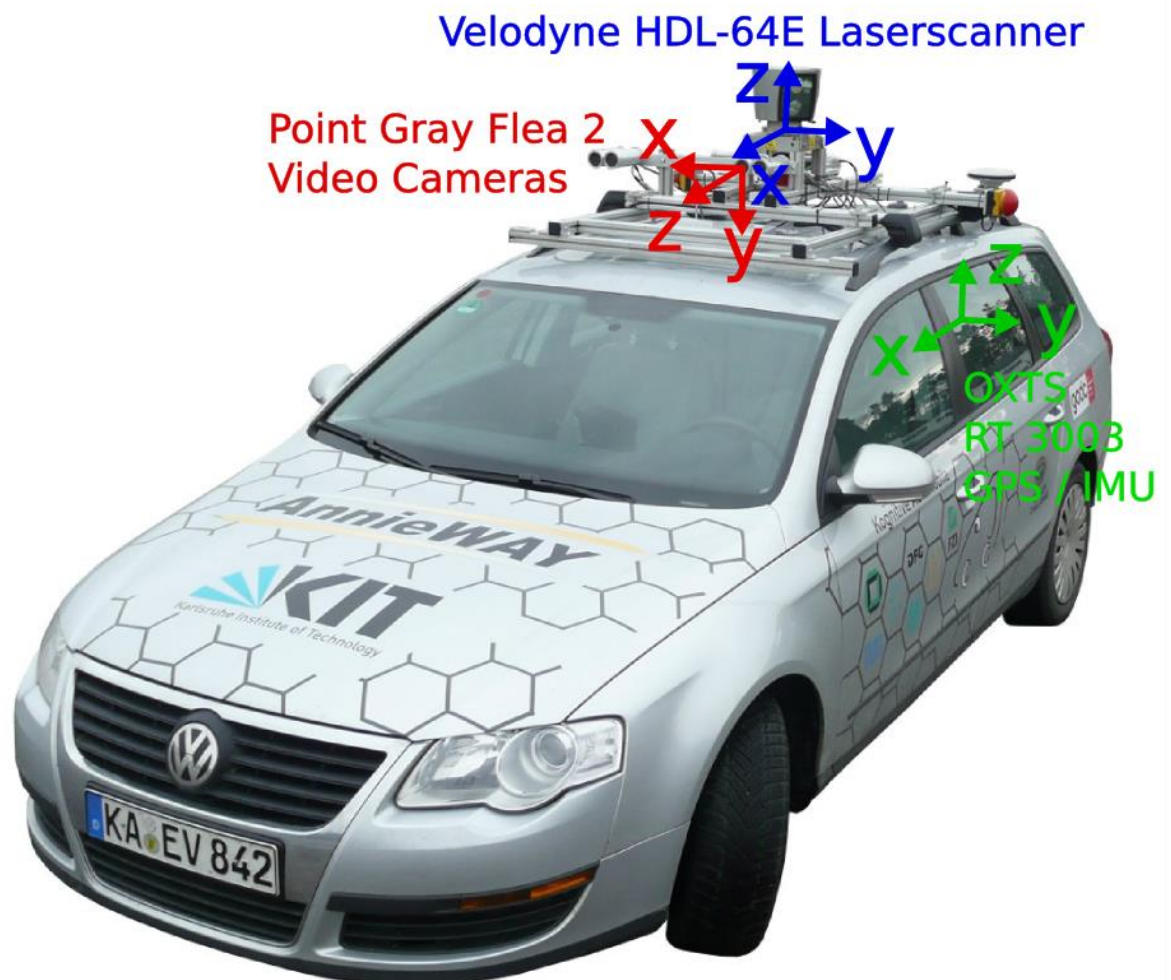
• **LiDAR trên không (Airborne LiDAR):** gắn trên máy bay, UAV hoặc Drone. Tích hợp các thành phần như cảm biến LiDAR, thiết bị định vị GPS và đơn vị đo vận tốc quán tính (IMU) để xác định chính xác vị trí, hướng và độ cao trong quá trình khảo sát quy mô rộng.

Phân loại theo công nghệ cảm biến và phương pháp quét:

- LiDAR quay cơ học:
 - Sử dụng dãy laser được sắp xếp theo chiều dọc, quay 360° để quét môi trường xung quanh.
 - Hạn chế: mật độ PC giảm theo khoảng cách.
- LiDAR Hybrid - Solid - State:
 - Kết hợp gương quay đa giác và gương dao động để quét theo cả chiều ngang và dọc.
 - Ưu điểm: Có thể quét khung hình bằng một chùm laser duy nhất nhưng đòi hỏi tốc độ quét rất cao.
- Công nghệ MEMS (Micro-Electro-Mechanical Systems):
 - Sử dụng hương quét micrometer siêu nhỏ, điều khiển bằng thanh cân treo.
 - Nhược điểm: giới hạn về góc quay và độ bền, dẫn đến trường nhìn hẹp và biến dạng ở biên.
- LiDAR ASIC:

- Tích hợp hàng trăm kênh laser trên một chip, giúp giảm kích thước, chi phí vận hành và tăng độ phân giải cũng như độ chính xác của dữ liệu thu thập (PC).
- Cân bằng giữa hiệu suất và độ tin cậy.

→ Loại LiDAR được sử dụng trong bộ dataset của KITTI là **Velodyne HDL-64E**, đây là một loại LiDAR quay cơ học với một đầu quét laser 3D xoay tròn với 64 chùm tia và thu được khoảng 1.3 triệu điểm mỗi giây. Được trang bị trên nóc xe Volkswagen Passat B6 cùng với hệ IMU/GPS và các camera.



Hình 2.7. Hình ảnh về tên của LiDAR và Camera được dùng để thu thập

Nguồn: [The KITTI Vision Benchmark Suite](#)

2.3.3 Cấu tạo LiDAR

2.3.3.1 Các thành phần chính

Nội dung của phần này được tham khảo và tổng hợp từ [8].

- Bộ phát Laser (Laser Scanner):
 - Chức năng: Phát ra các xung laser.
 - Các thông số quan trọng: độ chính xác, mật độ điểm (resolution), phạm vi quét và tốc độ quét.
- Bộ Thu (Receiver):
 - Chức năng: Ghi nhận ánh sáng phản xạ từ các đối tượng xung quanh và truyền tín hiệu về cho hệ thống xử lý.
- Bộ Xử Lý (Processor):
 - Chức năng: Tính toán thời gian bay (TOF / Time of Flight) của các xung laser và chuyển đổi kết quả thành dữ liệu dạng đám mây điểm (PC) để xây dựng mô hình 3D.

2.3.3.2 Hệ thống định vị và định hướng

GNSS (Global Navigation Satellite System)[10]:

- Cung cấp thông tin vị trí, vĩ độ, kinh độ và độ cao chính xác của hệ thống.

IMU (Inertial Measurement Unit):

- Xác định hướng, tốc độ và gia tốc của nền tảng di động (máy bay, drone, ô tô).
- Giúp hiệu chỉnh các sai lệch do rung lắc và góc quay không vuông (off nadir).

2.3.3.3 Các thành phần hỗ trợ khác

Máy tính và hệ thống lưu trữ dữ liệu: Xử lý và lưu trữ dữ liệu thu thập được, thường sử dụng định dạng dữ liệu LAS.

Hệ thống điều khiển: Quản lý quá trình quét và đồng bộ giữa các thành phần của hệ thống.

2.3.4 Nguyên tắc hoạt động của LiDAR

- **Bước 1:** Phát xạ xung laser (Pulse) [10]:

Hệ thống LiDAR phát ra một chùm tia laser ngắt quãng theo hướng xác định.

- **Bước 2:** Phản xạ (Return):

Khi tia laser va chạm với đối tượng (đất, cây cối, tòa nhà, con người, ...), nó sẽ phản xạ trở lại hệ thống.

- **Bước 3:** Ghi nhận và Tính toán khoảng cách

Thời gian bay (Time-of-Flight – **TOF**): Thời gian từ lúc phát ra đến khi nhận lại tia laser.

Do tia laser phải đi đến đối tượng và quay về nên thời gian tính được được chia đôi. Quá trình này sẽ được lặp lại hàng triệu lần mỗi giây và thu thập được dữ liệu dạng “PC” mô tả chi tiết không gian 3D.

Hệ thống cũng cần hiệu chỉnh các yếu tố như rung lắc, góc phát xung không vuông (*off nadir*) để đảm bảo độ chính xác cao.

- **Bước 4:** Xử lý và Hiển thị dữ liệu

Dữ liệu PC được xử lý bởi bộ xử lý (*processor*) và có thể kết hợp với dữ liệu hình ảnh (panchromatic, multispectral) để tạo ra các mô hình số bề mặt (*DSM*) hoặc địa hình (*DEM*) với chi tiết cao.

Hệ thống phân loại và tô màu dữ liệu LiDAR dựa trên đặc tính nhận dạng (theo tiêu chuẩn của *American Society for Photogrammetry and Remote Sensing*) để mô phỏng cảnh quan thực tế (màu xanh cho cây xanh, màu nâu cho đất, màu xám cho công trình, ...).

2.4 Point Cloud

2.4.1 Khái niệm Point Cloud

Point Cloud là tập hợp các điểm dữ liệu trong không gian 3 chiều, mỗi điểm được xác định bởi tọa độ (X, Y, Z) và có thể kèm theo thông tin phụ như cường độ ánh sáng hoặc màu sắc [11].

2.4.2 Thư viện Open3D

Các thư viện để trực quan hóa dữ liệu điểm như Point Cloud Library, Cloud Compare, MeshLab, Matlab, Autodesk Recap và đặc biệt là **Open3D** [12].

Open3D là thư viện dùng để xử lý dữ liệu 3D bằng C++ và Python. Hỗ trợ đọc/ghi các định dạng phổ biến (pcd, ply, xyz...), thao tác và trực quan hóa Point Cloud, xử lý voxel, pháp tuyến, crop, clustering, segmentation...

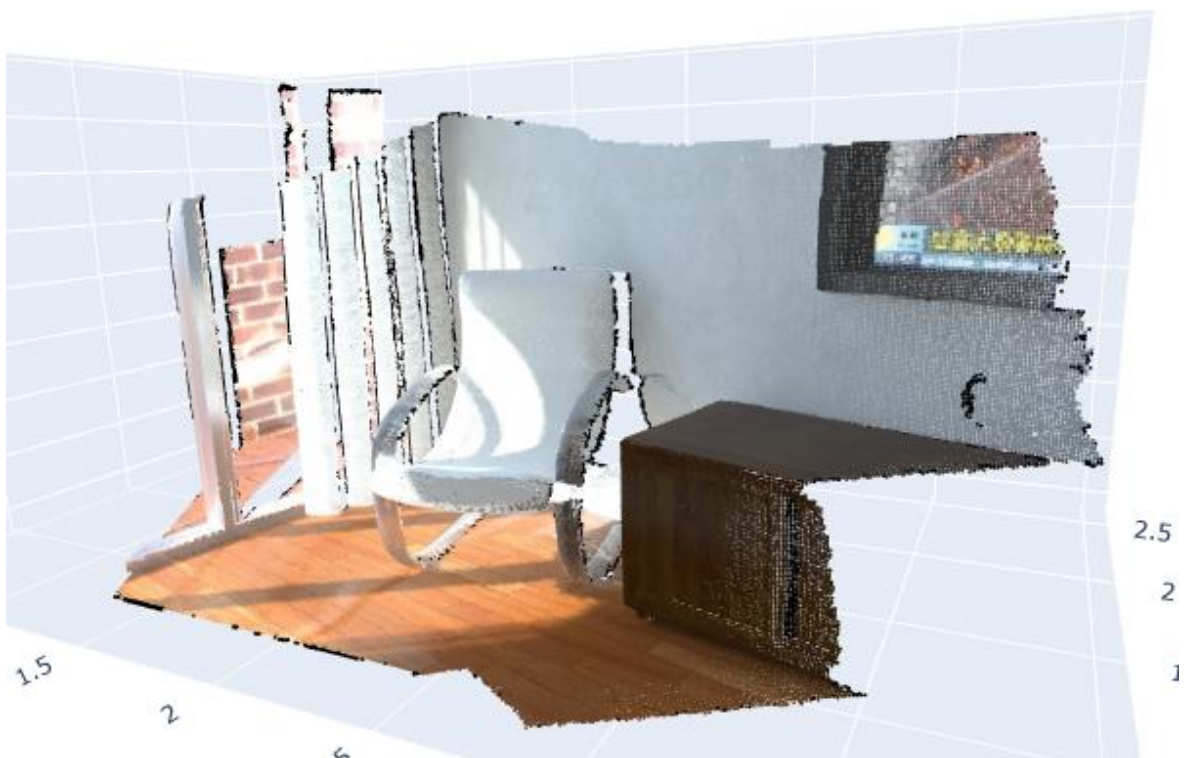
Cấu trúc dữ liệu được **Open3D** sử dụng:

- Voxel Grid: được tạo hoặc bắt nguồn từ các Point Cloud. Voxel là các điểm ảnh 3 chiều của hình ảnh 2D, viết tắt của điểm ảnh thể tích.
- Octree: cấu trúc phân chia không gian 3D thành 8 phần (octants) để quản lý dữ liệu hiệu quả
- Meshes: lưới tam giác hoặc đa giác dùng trong đồ họa 3D.

Các định dạng dữ liệu phổ biến trong xử lý PC

- *.PLY*: định dạng lưu trữ dữ liệu 3D dưới dạng tập hợp đa giác. Có thể chứa cả dữ liệu PC và Mesh.
- *.PCD*: định dạng chuyên dụng cho dữ liệu PC.
- *.XYZ*: biểu diễn các điểm trong không gian với tọa độ xyz ở định dạng văn bản.

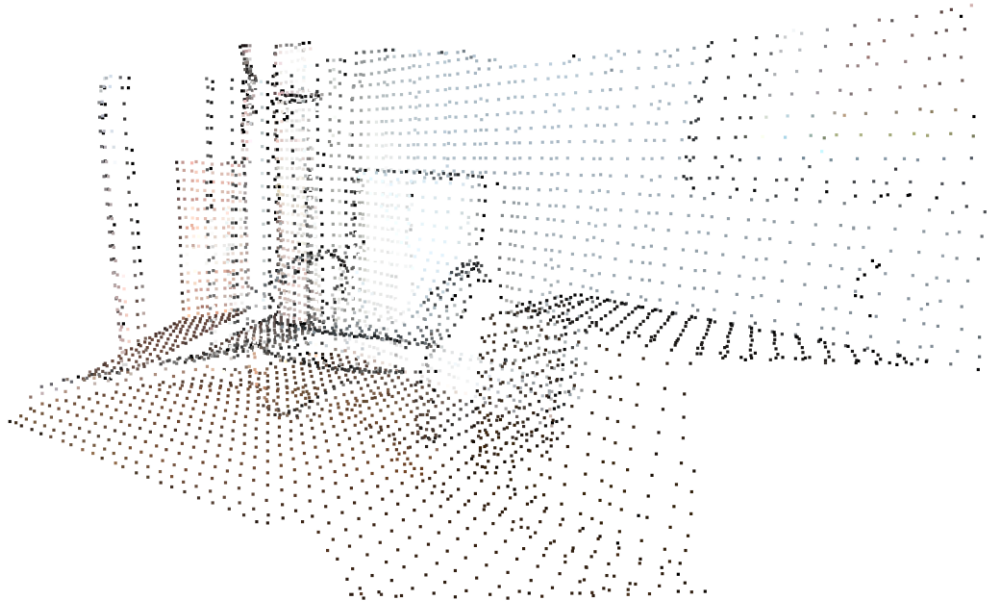
2.4.3 Trực quan hóa dữ liệu PC



Hình 2.8. Hình ảnh sau khi trực quan hóa dữ liệu Point Cloud

2.4.4 Cách biểu diễn Voxel

Open3D hỗ trợ **downsampling** PC (Voxel downsampling) [12] [13] bằng cách chia không gian thành các voxel – mỗi voxel chứa điểm, được đại diện bằng một điểm trung bình. Giúp giảm dữ liệu nhưng vẫn giữ được cấu trúc tổng thể.



Hình 2.9. Hình ảnh sau khi một file Point Cloud được áp dụng Voxel downsampling

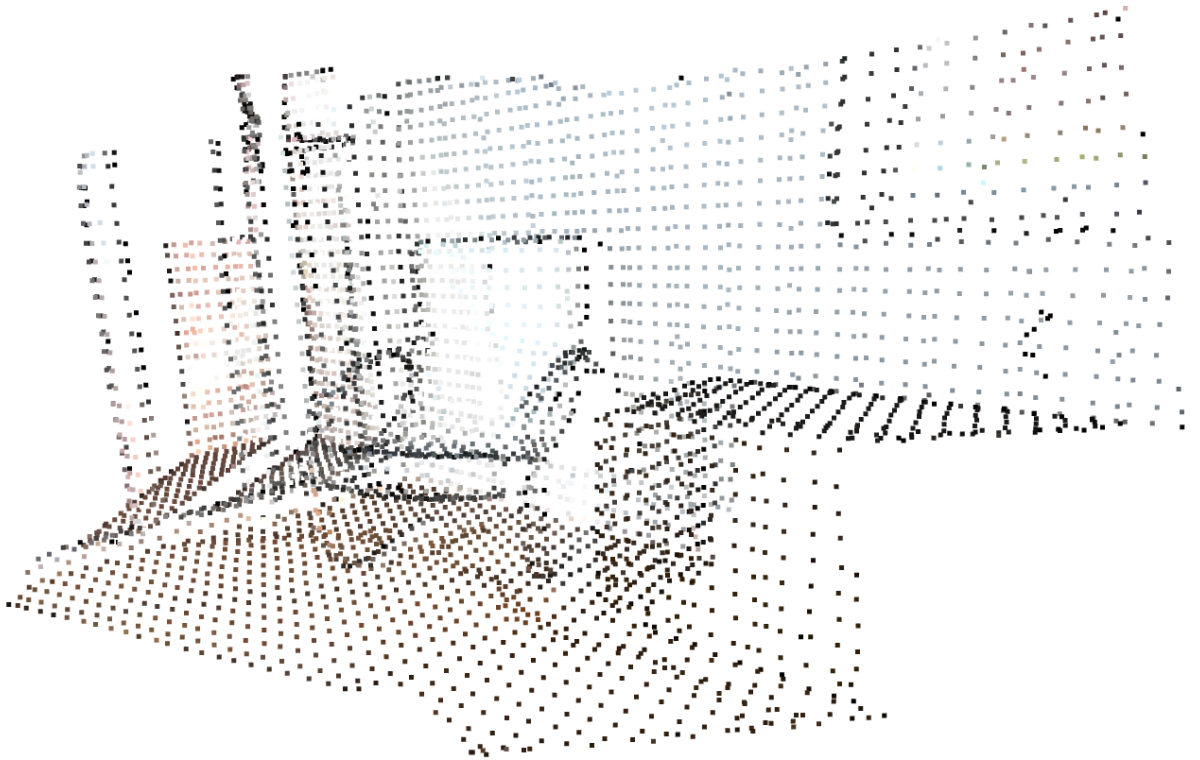
2.5 Các kỹ thuật xử lý PC cơ bản

Nội dung của phần này được tham khảo và tổng hợp từ [13][14].

2.5.1 Kỹ thuật lấy mẫu

- Mục đích của lấy mẫu (Sampling):
 - Giảm tải dữ liệu: dữ liệu PC gốc thường có đến hàng triệu điểm, gây quá tải cho bộ nhớ và tính toán. Lấy mẫu giúp giảm số điểm xuống mức vừa đủ nhưng vẫn giữ cấu trúc chung của đối tượng.
 - Tăng tốc độ xử lý: Với số điểm ít hơn, các bước tiền xử lý (như ước lượng pháp tuyến, đăng ký, tìm lân cận) thực hiện nhanh hơn và ít tiêu tốn tài nguyên hơn
 - Giữ được đặc trưng hình học: Mục tiêu là giữ lại những điểm đại diện cho hình dáng, chi tiết quan trọng, để vẫn cho kết quả xử lý (phân đoạn, phân cụm, tái dựng bề mặt...) gần tương đương với dữ liệu đầy đủ.
- Voxel Grid (Downsampling theo lưới voxel):
 - Nguyên lý: Chia không gian 3D thành lưới ô vuông (voxel) đều nhau có kích thước R . Mỗi voxel chứa một tập con các điểm gốc. Với mỗi voxel có ít nhất một điểm, sinh ra duy nhất một điểm đại diện bằng cách lấy trung bình tọa độ của tất cả điểm trong voxel đó.

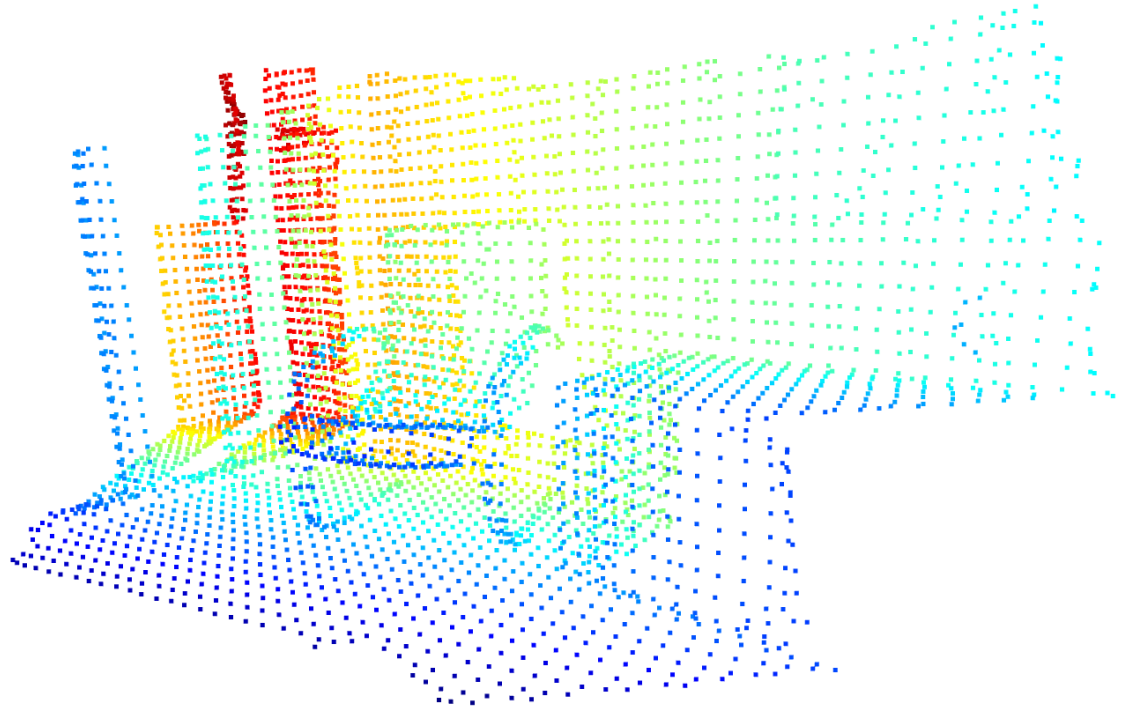
- Xây dựng tọa độ lưới voxel:
 - Tính miền tọa độ
 - Kích thước lưới
 - Mã hóa chỉ số voxel: Mỗi index là 1 voxel duy nhất



Hình 2.10. Hình ảnh sau Point Cloud áp dụng Voxel Grid

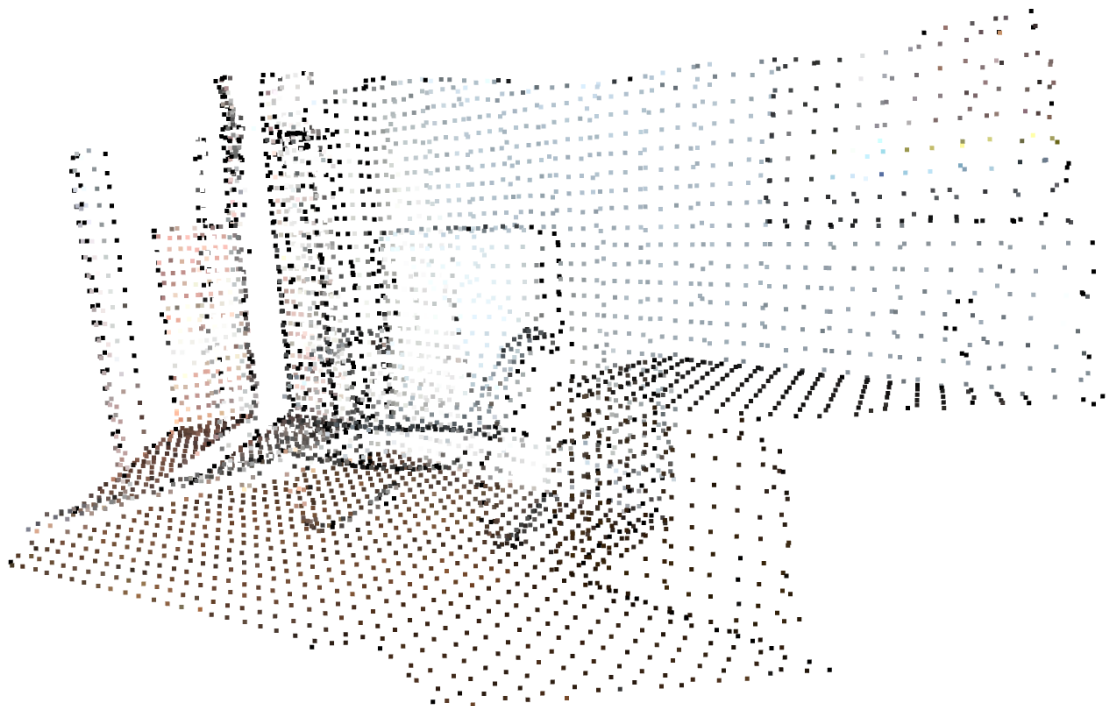
➔ **Nhược điểm:** Nếu mật độ gốc không đồng đều, vùng thưa quá có thể mất chi tiết, vùng dày cũng chỉ còn một điểm đại diện.

- **Centroid Point:** Với mỗi ô voxel, tính tọa độ trung bình của tất cả điểm chứa trong ô đó và làm điểm đại diện



Hình 2.11. Hình ảnh sau Point Cloud áp dụng Centroid Point

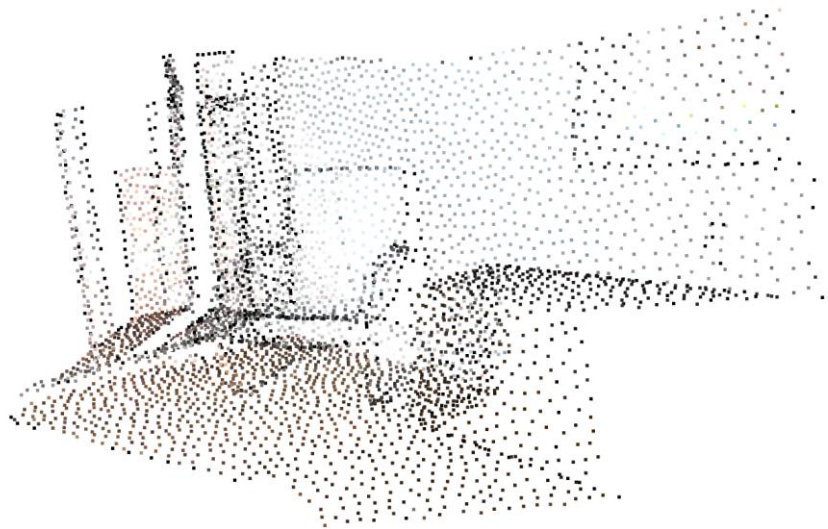
- Center Point: với mỗi voxel (như trên), chọn điểm nằm ở giữa (center) của voxel làm đại diện, thay vì trung bình



Hình 2.12. Hình ảnh sau Point Cloud áp dụng Center Point

- Furthest Point Sampling (FPS): Lấy mẫu tuần tự sao cho mỗi điểm mới luôn là điểm xa nhất so với tất cả điểm đã chọn, đảm bảo phân bố đều theo hình dạng gốc.

- Thuật toán (Heuristic):
 - Bước 1: Chọn ngẫu nhiên 1 điểm ban đầu.
 - Bước 2: Tính khoảng cách giữ điểm hiện tại được chọn đến tất cả các điểm còn lại.
 - Bước 3: chọn điểm xa nhất so với điểm trước đó.
 - Bước 4: Lặp lại bước 2 và 3 cho đến khi chọn được m điểm.
- Ưu điểm: Duy trì sự phân bố đồng đều hơn qua toàn bộ vùng dữ liệu, đặc biệt với PC có mật độ không đồng đều. Giữ chi tiết vùng thưa lẫn vùng dày.



Hình 2.13. Hình ảnh sau Point Cloud áp dụng Furthest Point Sampling

2.5.1.1 Kỹ thuật Radius Outlier

Mục tiêu: Loại bỏ các “điểm ngoại lai” – những điểm không thuộc về cấu trúc chính của dữ liệu, thường do nhiễu từ cảm biến hoặc lỗi trong quá trình thu thập dữ liệu.

Phương pháp: Dựa trên mật độ điểm

- Một điểm được xem là hợp lệ nếu có đủ số lượng điểm lân cận trong một bán kính xác định.
- Nếu điểm có quá ít điểm lân cận, nó sẽ bị xem là nhiễu và bị loại bỏ.

Cách hoạt động: Thuật toán **ROR** (*Radius Outlier Removal*) hoạt động theo các bước sau:

- Bước 1: Xác định tham số

- Bán kính (r): Là khoảng cách tối đa để tìm các điểm lân cận xung quanh một điểm.
- Số lượng lân cận tối thiểu ($\text{min_neighbors: } k$): Là số lượng điểm tối thiểu cần trong bán kính (r) để một điểm được xem là hợp lệ.
- Bước 2: Duyệt qua từng điểm
 - Với mỗi điểm P_i trong PC thuật toán sẽ tính toán khoảng cách từ P_i đến tất cả các điểm khác.
 - Xác định tập hợp các điểm lân cận dựa trên khoảng cách Euclidean
- Bước 3: Kiểm tra điều kiện “ngoại lai”
 - Đếm số lượng điểm trong N_i (số lân cận).
 - Nếu $|N_i| < K$, điểm P_i được coi là “ngoại lai” và sẽ được bỏ đi.
 - Nếu $|N_i| \geq K$, điểm P_i sẽ được giữ lại.
- Bước 4: Tạo PC mới
 - Sau khi kiểm tra tất cả các điểm một PC mới được tạo ra, chỉ chứa các điểm không bị coi là ngoại lai.

2.5.1.2 Kỹ thuật Statistical Outlier

SOR (Statistical Outlier Removal) là kỹ thuật lọc nhiễu dựa trên “*Phân tích thống kê*” của khoảng cách giữa các điểm và lân cận chúng.

Cơ sở lý thuyết:

- SOR dựa trên ý tưởng rằng khoảng cách trung bình từ một điểm đến các lân cận gần nhất của nó phản ánh mật độ cục bộ.
- Nếu khoảng cách trung bình của một điểm vượt quá một ngưỡng, điểm đó sẽ được cho là “ngoại lai”.

Các tham số chính:

- Số lượng k lân cận: Số điểm lân cận gần nhất (k -nearest neighbors) được xem xét cho mỗi điểm.

- Hệ số ngưỡng chuẩn hóa (stddev_mul): Hệ số nhân của độ lệch chuẩn để xác định ngưỡng.

Cách hoạt động:

- Bước 1: Xác định tham số
 - k: Số lượng lân cận gần nhất
 - stddev_mul: Hệ số ngưỡng (vd: stddev_mul= 1.0, nghĩa là các điểm có khoảng cách trung bình vượt quá 1 độ lệch chuẩn sẽ bị loại).
- Bước 2: Tính khoảng cách lân cận
 - Với mỗi điểm P_i trong point cloud, tìm kiếm k điểm lân cận gần nhất.
 - Tính khoảng cách trung bình từ P_i đến k lân cận gọi là d_i .
- Bước 3: Phân tích thống kê
 - Thu thập tất cả các khoảng cách trung bình $d_1, d_2, d_3, \dots, d_n$ từ toàn bộ PC.
 - Tính trung bình và độ lệch chuẩn của tập d_i .
 - Xác định ngưỡng threshold
- Bước 4: Loại bỏ ngoại lai
 - Nếu $d_i > \text{threshold}$ thì coi là ngoại lai và được loại bỏ.
 - Nếu $d_i \leq \text{threshold}$ thì giữ lại
- Bước 5: Tạo PC mới
 - PC mới chỉ chứa các điểm không coi là ngoại lai.

2.5.2 Kỹ thuật Surface normal

Một vector vuông góc với mặt phẳng.

Một mặt phẳng có thể được xác định bởi:

- Vector vuông góc (a, b, c)
- Một điểm trên mặt phẳng (X, Y, Z)

Phương trình mặt phẳng từ vector vuông góc và điểm trong: $ax+by+cz+d = 0$

$$n = [n_x, n_y, n_z]^T = \frac{[a, b, c]^T}{\|[a, b, c]^T\|} \quad (2.14)$$

*Về mặt lý thuyết:

- Chọn 3 điểm bất kì: Lấy ba điểm trên bề mặt, xác lập phương trình mặt phẳng chung rồi tính véc-tơ vuông góc.

- Tuy nhiên trong thực tế PC có hàng ngàn đến hàng triệu điểm, việc chọn ba điểm “đẹp” là rất khó và dễ dẫn đến sai số.

*Thực tế:

- Không có điểm nào là hoàn hảo trong một mặt phẳng - vector riêng
- Thu thập một tập hợp các điểm
- Tính toán các vector riêng và giá trị riêng(*eigenvalues*) ($\lambda_1, \lambda_2, \lambda_3$)
- Giá trị ít bị điều khiển nhất là vector chuẩn (*normal vector*): $n = \lambda_3$

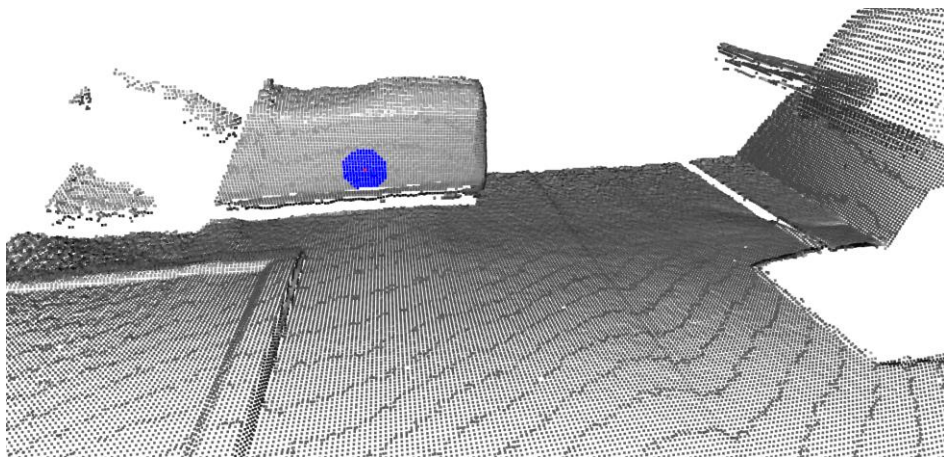
Surface variance = Curvature

$$c = \frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3} \quad (2.15)$$

2.5.3 Tìm kiếm trên PC

2.5.3.1 Tìm kiếm lân cận K-Nearest Neighbors - KNN

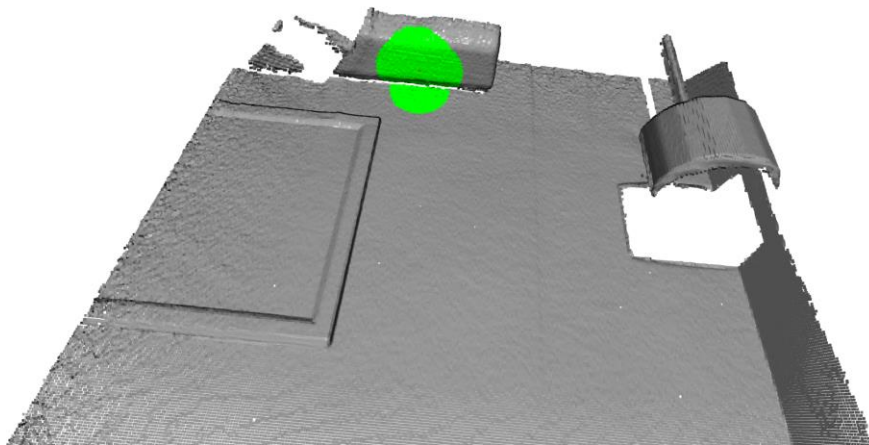
Với mỗi điểm mỏ neo (anchor point), tìm K điểm gần nhất dựa trên khoảng cách Euclid



Hình 2.14. Hình ảnh Point Cloud sau khi sử dụng tìm kiếm KNN và tô màu

2.5.3.2 Radius Nearest Neighbors

Với mỗi điểm mỏ neo, tìm tất cả các điểm nằm trong một bán kính cho trước



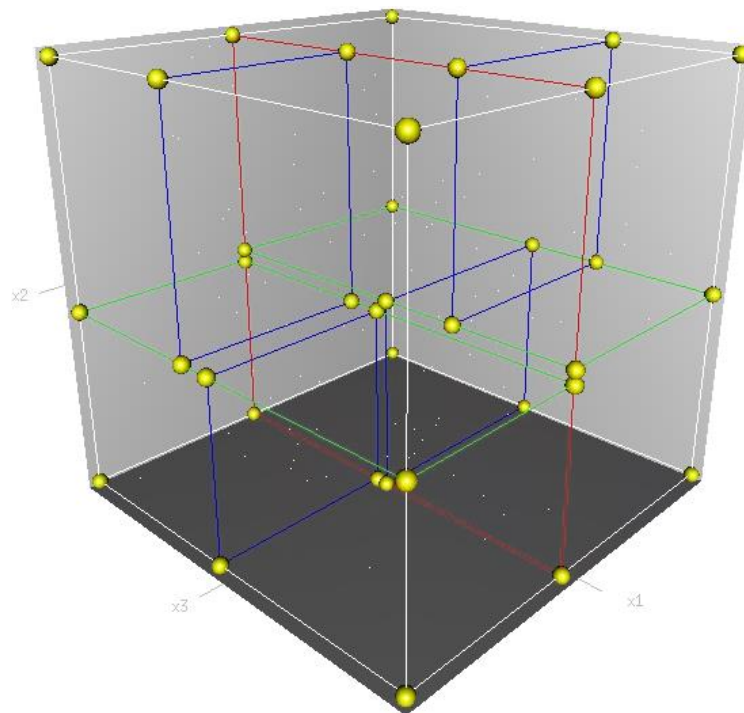
Hình 2.15. Hình ảnh Point Cloud sau khi sử dụng tìm kiếm Radius Nearest Neighbors và tô màu

2.5.4 Cấu trúc dữ liệu trên PC

2.5.4.1 K Demensional Tree

KDTree [15] là một cấu trúc cây nhị phân dùng để tổ chức các điểm trong không gian K chiều.

Nó chia không gian thành các vùng bằng cách luân phiên cắt dọc theo các trục tọa độ (X, Y, Z) tại mỗi mức của cây.



Hình 2.16. Hình ảnh minh họa về K Demensional Tree

Mỗi nút trong cây lưu trữ:

- Một điểm dữ liệu (tọa độ)
- Một mặt phẳng chia không gian thành 2 phần (trái và phải)

Mục đích chính là để tăng tốc các truy vấn không gian như tìm kiếm hàng xóm gần nhất (*KNN*) hoặc truy vấn phạm vi (*Radius Seach*).

Cách xây dựng KDTree:

- Chọn một trục (x, y, hoặc z) tại mỗi mức (thường luân phiên hoặc chọn trục có độ phân tán lớn nhất).
- Sắp xếp các điểm theo trục đó và chọn điểm giữa (median) làm nút gốc.
- Chia tập dữ liệu thành hai tập con (trái và phải) dựa trên mặt phẳng cắt qua điểm median.
- Lặp lại đệ quy cho mỗi tập con cho đến khi số điểm trong một vùng nhỏ hơn ngưỡng (thường là 1 điểm).

2.5.4.2 OCTree

Octree là một cấu trúc cây phân cấp (*hierarchical tree*) chia không gian 3D thành các khối lập phương (*cube*) nhỏ hơn, gọi là voxel, thông qua việc phân chia đệ quy [16]. Mỗi nút trong cây đại diện cho một vùng không gian 3D, và mỗi nút có thể được chia thành tối đa 8 nút con (do đó gọi là "*oct*" - tám).

Ý tưởng:

- Bắt đầu với một khối lập phương bao quanh toàn bộ không gian (gọi là root node).
- Nếu một nút chứa quá nhiều dữ liệu (ví dụ: quá nhiều điểm trong đám mây điểm) hoặc cần độ phân giải cao hơn, nó được chia thành 8 khối nhỏ hơn (gọi là nút con), mỗi khối chiếm 1/8 thể tích của nút cha.
- Quá trình chia này tiếp tục đệ quy cho đến khi đạt được điều kiện dừng (ví dụ: số điểm trong mỗi nút nhỏ hơn ngưỡng, hoặc đạt độ sâu tối đa).

Cấu trúc:

- **Root node:** Đại diện cho toàn bộ không gian 3D.

• **Internal nodes:** Các nút trung gian, đại diện cho các khối lập phương được chia nhỏ.

• **Leaf nodes:** Các nút lá, chứa dữ liệu thực tế (như điểm, màu sắc, hoặc thông tin khác) và không được chia thêm.

Cách hoạt động:

• Khởi tạo:

- Xác định một khối lập phương bao quanh toàn bộ dữ liệu (bounding box).
- Gán tất cả các điểm (hoặc dữ liệu không gian) vào nút gốc.

• Chia đệ quy:

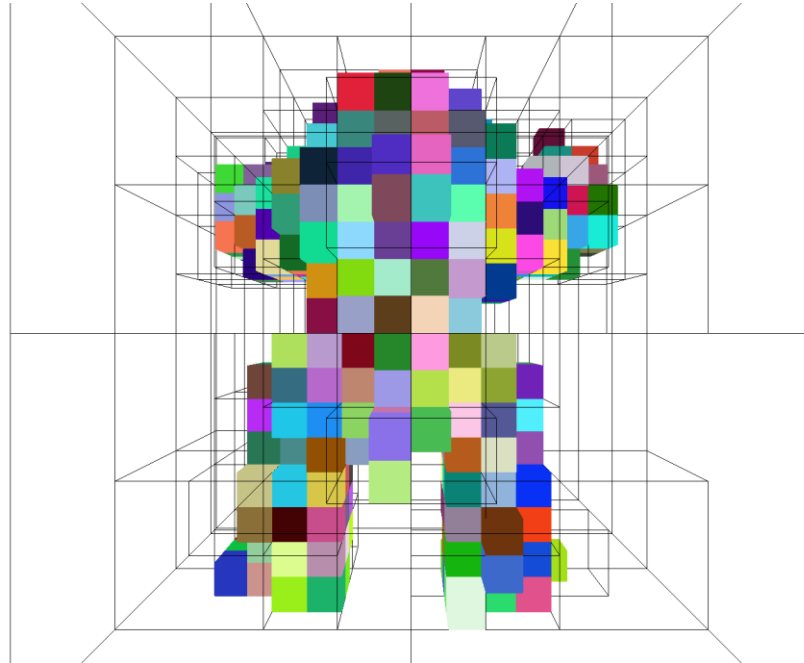
- Nếu số lượng điểm trong một nút vượt quá ngưỡng (hoặc theo tiêu chí khác), chia khối lập phương thành 8 khối nhỏ hơn bằng cách chia đôi mỗi chiều (X, Y, Z).
- Phân bổ các điểm vào các nút con tương ứng dựa trên vị trí không gian của chúng.

• Điều kiện dừng:

- Số điểm trong một nút nhỏ hơn ngưỡng.
- Đạt độ sâu cây tối đa.
- Kích thước voxel nhỏ hơn một giá trị tối thiểu.



Hình 2.17 Hình ảnh minh họa về Point Cloud trước khi OCTree



Hình 2.18. Hình ảnh minh họa về Point Cloud sau khi OCTree

2.5.4.3 Cấu trúc dữ liệu giúp tìm kiếm hiệu quả

Bảng 2.1. So sánh cấu trúc dữ liệu giúp tìm kiếm hiệu quả

	Cấu trúc	Nguyên lý	Ưu điểm
KDTree	Đơn giản, có sẵn trong nhiều thư viện	Chia đôi theo trục lần lượt ($X \rightarrow Y \rightarrow Z \rightarrow X \rightarrow \dots$) cho đến khi mỗi lá chứa ≤ 1 điểm hoặc đạt chiều sâu tối đa	Tìm KNN/radius search nhanh (tránh duyệt toàn bộ điểm)
Octree	Dễ dàng kết hợp với voxelization Hỗ trợ traversal, locate leaf node	Chia không gian theo lưới $2 \times 2 \times 2$ đệ quy; mỗi nút có 8 con	Xử lý dữ liệu không đồng đều (dense vs sparse)

2.6 Các kỹ thuật xử lý PC nâng cao

Nội dung của phần này được tham khảo và tổng hợp từ [17].

2.6.1 Registration

Registration là quá trình căn chỉnh hai hoặc nhiều đám mây điểm (PC) sao cho chúng khớp với nhau trong cùng một hệ tọa độ. Điều này rất cần thiết nếu chúng ta có dữ liệu từ nhiều cảm biến hoặc góc quét khác nhau.

Nếu không căn chỉnh đúng sẽ làm cho dữ liệu bị lệch dẫn đến sai sót trong việc tái tạo 3D hoặc phát hiện vật thể.

Tại sao cần căn chỉnh trong xử lý dữ liệu PC?

- Kết hợp dữ liệu từ nhiều góc độ khác nhau
 - PC thường được thu thập từ các thiết bị như LIDAR hoặc camera 3D, mỗi lần quét chỉ cho một phần góc độ do góc quét hạn hẹp.
 - Để tạo mô hình 3D hoàn chỉnh, chúng ta cần ghép các PC này lại với nhau. Căn chỉnh chúng để đảm bảo các thành phần riêng lẻ khớp với nhau một cách chính xác.
- Xử lý sự khác biệt về hướng
 - Mỗi lần LiDAR hoặc Camera 3D quét thì chúng có thể được thực hiện từ vị trí và góc độ khác nhau, dẫn đến các PC nằm ở hệ tọa độ khác nhau.
 - Căn chỉnh giúp di chuyển và xoay PC nằm đúng vị trí tương ứng với nhau.
- Tăng độ chính xác và giảm sai số
 - Nếu không căn chỉnh, các PC này sẽ không khớp với nhau, dẫn đến mô hình 3D bị méo mó hoặc sai lệch.
 - Thuật toán như ICP sẽ giúp tối ưu hóa sự khớp nối, giảm sai số giữa các điểm tương ứng của hai hay nhiều PC.
- Xử lý cho dữ liệu lớn và phức tạp
 - Trong các ứng dụng như quét thành phố hoặc môi trường lớn, căn chỉnh là bước quan trọng để tổ chức dữ liệu từ hàng triệu điểm thành một mô hình thống nhất.

2.6.2 Các phương pháp Registration

2.6.2.1 Thuật toán ICP

Là thuật toán dùng để căn chỉnh hai PC: Một PC nguồn là P và một PC đích là Q . Mục tiêu là tìm cách nào đó để xoay và di chuyển P để nó khớp với Q một cách tốt nhất (gần trùng với Q hoặc trùng với Q).

- Nguyên lý hoạt động:

- **Tìm cặp điểm gần nhất:** Với mỗi điểm trong PC nguồn (P), thuật toán sẽ xác định điểm gần nhất trong PC đích (Q) dựa trên khoảng cách Euclidean. Các cặp điểm này được coi là tương ứng tạm thời (tức là cặp điểm này có thể sẽ bị thay đổi).
- **Tính toán biến đổi:** Dựa trên các cặp điểm tương ứng, ICP sẽ tính toán ma trận quay (Rotation) và Vector tịnh tiến (Translation) nhằm giảm thiểu tổng sai số giữa các cặp điểm. Sai số thường được định nghĩa là *tổng bình phương khoảng cách*.
- **Áp dụng biến đổi:** Ma trận quay và vector tịnh tiến được áp dụng để cập nhật vị trí của đám mây nguồn (P) đưa nó gần hơn với đám mây đích (Q).
- **Lặp lại:** Các bước trên thực hiện lặp đi lặp lại, mỗi lần cập nhật các cặp điểm tương ứng và ma trận biến đổi, cho đến khi sai số E hội tụ (nhỏ hơn ngưỡng cho trước) hoặc đạt số lần lặp tối đa.
- **Ưu điểm:**
 - Dễ triển khai và tính toán đơn giản.
 - Phù hợp với PC có cấu trúc rõ ràng và ít nhiễu.
- **Nhược điểm:**
 - Nhạy cảm với nhiễu và ngoại lai dẫn đến lỗi sẽ tăng mạnh vì khoảng cách Euclidean được tính trực tiếp.
 - Kém hiệu quả với bề mặt không đồng nhất vì nó không tận dụng được thông tin cấu trúc bề mặt.

2.6.2.2 Registration nâng cao

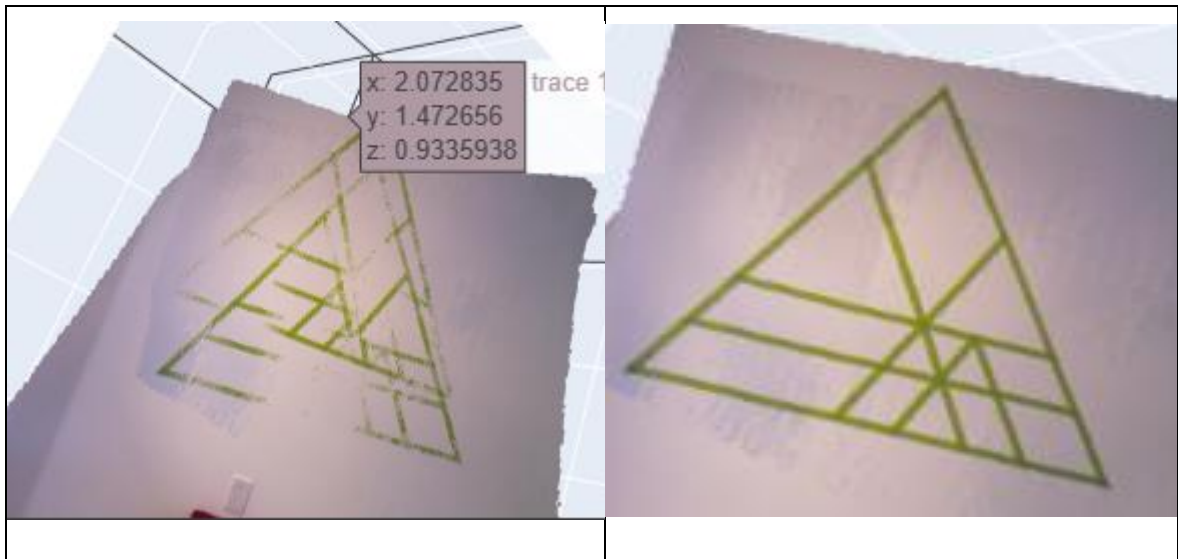
- **Colored Point Cloud Registration:**
 - Là kỹ thuật căn chỉnh tập hợp điểm PC bằng cách sử dụng cả thông tin tọa độ X, Y, Z và thông tin màu sắc như RGB, HSV.
 - Mục tiêu là ghép các PC từ các góc nhìn khác nhau hoặc từ cảm biến khác nhau thậm chí là từng thời điểm khác nhau thành một hệ tọa độ

chung. Tương tự như các phương pháp registration khác nhưng bổ sung thêm màu sắc để tăng độ chính xác.

- Sự khác biệt so với ICP truyền thống chỉ dùng X, Y, Z :
 - ICP truyền thống: Chỉ dùng tọa độ không gian X, Y, Z để tính khoảng cách Euclidean giữa các điểm gần nhất, sau đó tối ưu hóa chuyển động (R và t) để căn chỉnh hai PC. Dẫn đến sẽ không hiệu quả nếu các vật thể vô tình cùng chung hình dạng nhưng khác đặc trưng như màu sắc, ...
 - Colored ICP: Ngoài sử dụng khoảng cách không gian, thì Colored ICP còn xem xét khoảng cách giữa các màu sắc giữa các điểm. Dẫn đến hàm loss được điều chỉnh để cân bằng giữa độ tương đồng không gian và màu sắc. Điều này giúp cải thiện độ chính xác khi căn chỉnh các PC có thông tin màu sắc.
- Ưu điểm:
 - Phân biệt vật thể tốt hơn.
 - Tăng độ chính xác.
- Nhược điểm:
 - Ảnh hưởng bởi nhiều màu sắc
 - Tăng độ phức tạp của thuật toán do sử dụng thêm hàm loss cho màu sắc

Bảng 2.2. Bảng so sánh kết quả giữa ICP và Colored ICP

Sử dụng ICP truyền thống (Point to Plane)	Sử dụng Colored ICP
--	---------------------



- Global Registration:

- Hai phương pháp ICP và Colored ICP đều thuộc nhóm local registration, yêu cầu một estimate khởi tạo đủ gần. Khi ta không có phép biến đổi khởi tạo đủ gần, thuật toán local có thể kẹt tại cực tiểu cục bộ. Khi đó ta cần đến Global Registration để tìm alignment “toàn cục” ngay từ đầu.

- Global registration tìm phép biến đổi ban đầu để căn chỉnh hai PC source và target mà không cần khởi tạo trước. Kết quả thường dùng làm input cho các bước local refinement.

- Khái niệm: Global registration sẽ tìm vị trí và hướng xoay tổng thể để ghép hai hoặc nhiều PC lại mà không cần biết vị trí gần đúng ban đầu.

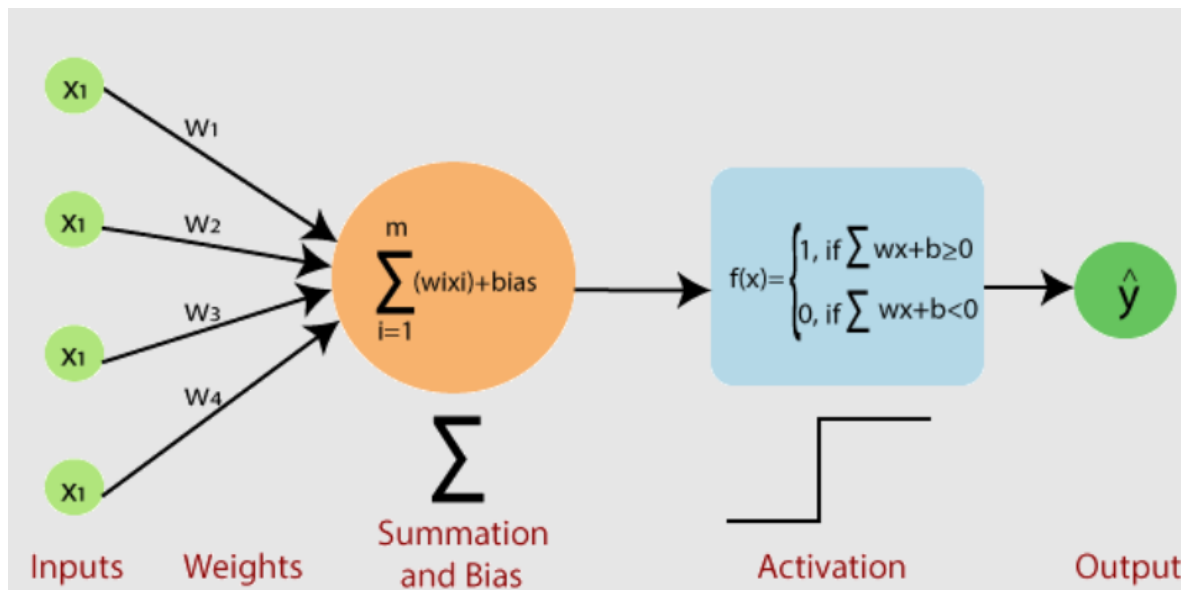
- Ưu điểm: Phù hợp khi PC ban đầu cách xa nhau hoặc chưa được căn chỉnh.

2.7 Perceptron

2.7.1 Khái niệm

Perceptron là mô hình dạng nơ-ron nhân tạo đơn giản nhất được phát minh bởi Frank Rosenblatt. Mô hình này mô phỏng quá trình ra quyết định của một nơ-ron sinh học, giúp phân loại các mẫu đầu vào nhị phân nghĩa là dự đoán một trong hai lớp có thể xác định

2.7.2 Cấu trúc và nguyên lý hoạt động



Hình 2.19. Cấu trúc và nguyên lý hoạt động của Perceptron

Nguồn: [Bài 7: Thuật toán Perceptron - Trí tuệ nhân tạo](#)

Bao gồm: vector đầu vào x_i , vector trọng số w tương ứng, và một ngưỡng kích hoạt (bias)

Đầu ra công thức:

$$y = f(w_i \cdot x_i + b) \quad (2.16)$$

Trong đó:

- $w_i \cdot x_i$ là tích vô hướng giữa vector trọng số và vector đầu vào
- b là bias
- f là hàm kích hoạt, thường là hàm ngưỡng:

$$f(x) = \begin{cases} 1 & \text{nếu } \sum w_i \cdot x_i + b \geq 0 \\ 0 & \text{nếu } \sum w_i \cdot x_i + b < 0 \end{cases} \quad (2.17)$$

2.7.3 Ưu điểm

- Dễ hiểu, dễ dàng cài đặt.
- Là nền tảng lý thuyết cho các mạng neuron phức tạp hơn.

2.7.4 Nhược điểm

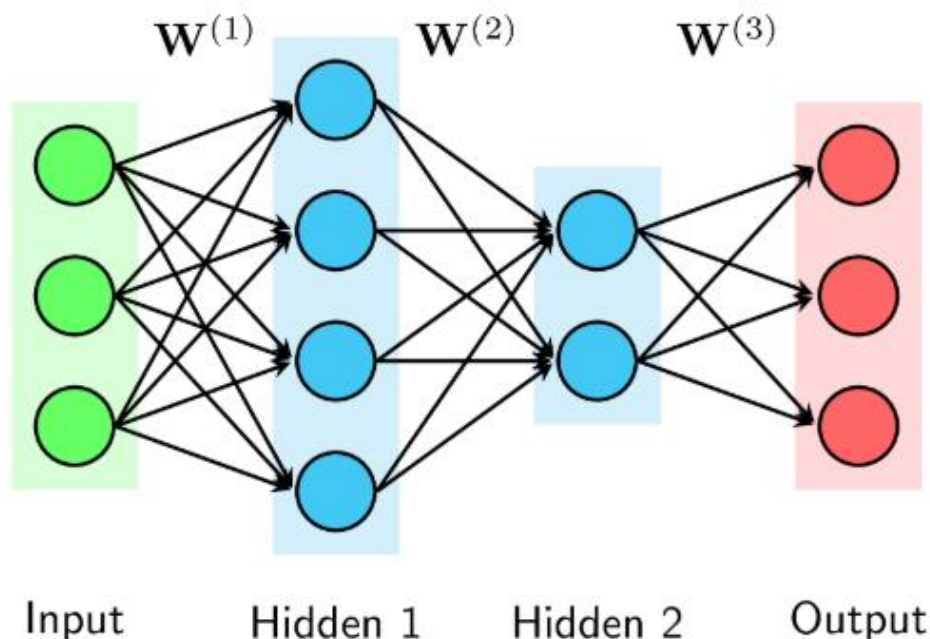
- Chỉ xử lý được bài toán tuyến tính phân biệt được.
- Không giải được các bài toán như XOR hay là các bài toán phi tuyến tính.
- Mô hình khá đơn giản không đủ khả năng trích xuất đặc trưng phức tạp từ dữ liệu.

2.8 Mạng neuron đa tầng

2.8.1 Khái niệm

Multilayer Perceptron (MLP) là mạng nơ-ron có cấu trúc nhiều lớp, trong đó có ít nhất một lớp ẩn giữa đầu vào và đầu ra, các kết nối giữa các tầng là đầy đủ (Fully Connected). Là cải tiến của Perceptron đơn tầng nhằm giải quyết các bài toán phân loại phi tuyến như bài toán XOR.

2.8.2 Cấu trúc và nguyên lý hoạt động



Hình 2.20. Ảnh từ mô tả về Multilayer Perceptron

Nguồn: [Machine Learning cơ bản](#)

MLP được huấn luyện bằng thuật toán lan truyền ngược kết hợp với phương pháp tối ưu hóa như gradient descent. Cơ chế này tính toán đạo hàm của hàm mất mát theo từng trọng số, sau đó cập nhật ngược từ đầu ra về đầu vào. Đây là yếu tố then chốt giúp mạng học được các mối quan hệ phi tuyến trong dữ liệu.

2.9 Mạng neuron tích chập

2.9.1 Khái niệm

CNN (Convolutional Neural Network) là một kiến trúc mạng nơ-ron chuyên xử lý dữ liệu không gian như ảnh hoặc video. Thay vì dùng kết nối đầy đủ thì CNN sử dụng các lớp tích chập (convolutional layers) để học các đặc trưng cục bộ trong dữ liệu. CNN được ứng dụng nhiều trong nhận dạng khuôn mặt, phân loại động vật ...

2.9.2 Phép toán Convolution

Phép tích chập (convolution) thực hiện bằng cách trượt một kernel (bộ lọc) qua ảnh và tính tổng tích từng phần tử

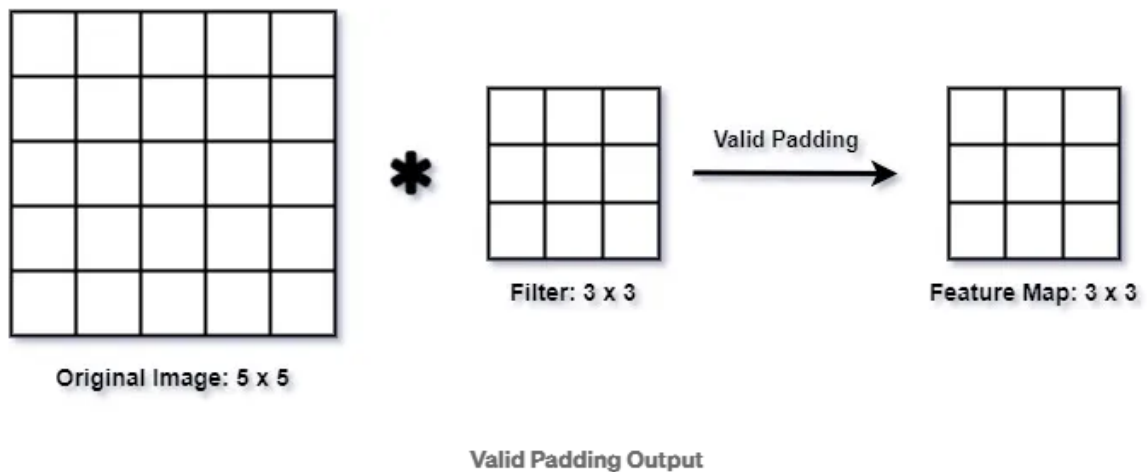
Input		Kernel		Output																	
<table border="1" style="border-collapse: collapse;"> <tr><td>0</td><td>1</td><td>2</td></tr> <tr><td>3</td><td>4</td><td>5</td></tr> <tr><td>6</td><td>7</td><td>8</td></tr> </table>	0	1	2	3	4	5	6	7	8	*	<table border="1" style="border-collapse: collapse;"> <tr><td>0</td><td>1</td></tr> <tr><td>2</td><td>3</td></tr> </table>	0	1	2	3	=	<table border="1" style="border-collapse: collapse;"> <tr><td>19</td><td>25</td></tr> <tr><td>37</td><td>43</td></tr> </table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Hình 2.21. Hình ảnh minh họa về cơ chế của Convolution

2.9.3 Phép toán Padding

Padding là kỹ thuật thêm các hàng / cột giá trị 0 xung quanh biên ảnh để giữ nguyên kích thước đầu ra sau tích chập. Có hai loại padding phổ biến:

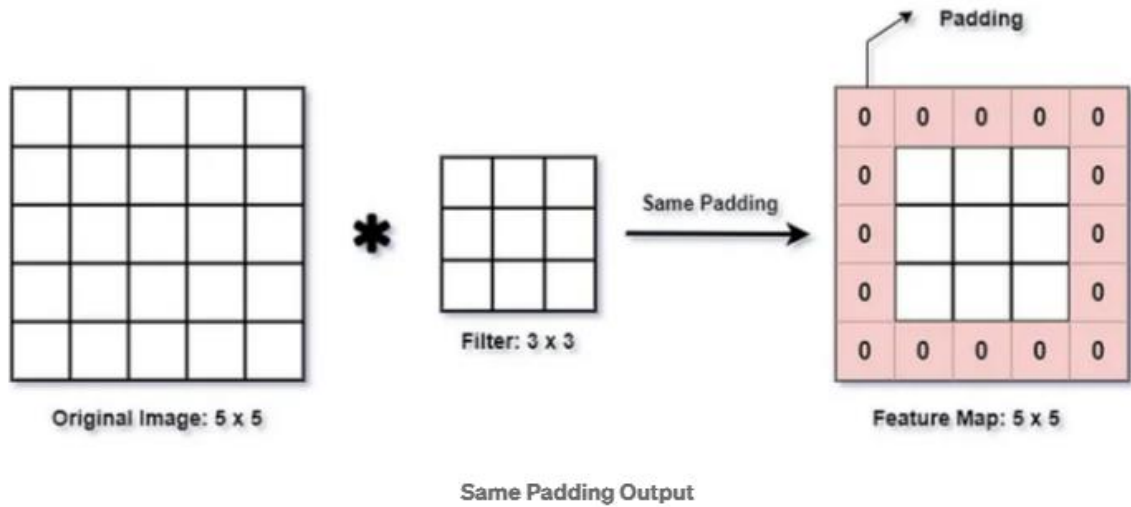
- Valid Padding: Không thêm gì (output nhỏ hơn input)



Hình 2.22. Hình ảnh minh họa về Valid Padding

Nguồn: [Padding in Neural Networks: Why and How? | by Ashish Sanjay Raut | GoPenAI](#)

- Same Padding: Thêm đủ để giữ nguyên kích thước



Hình 2.23. Ảnh minh họa về Same Padding

Nguồn: [Padding in Neural Networks: Why and How? | by Ashish Sanjay Raut | GoPenAI](#)

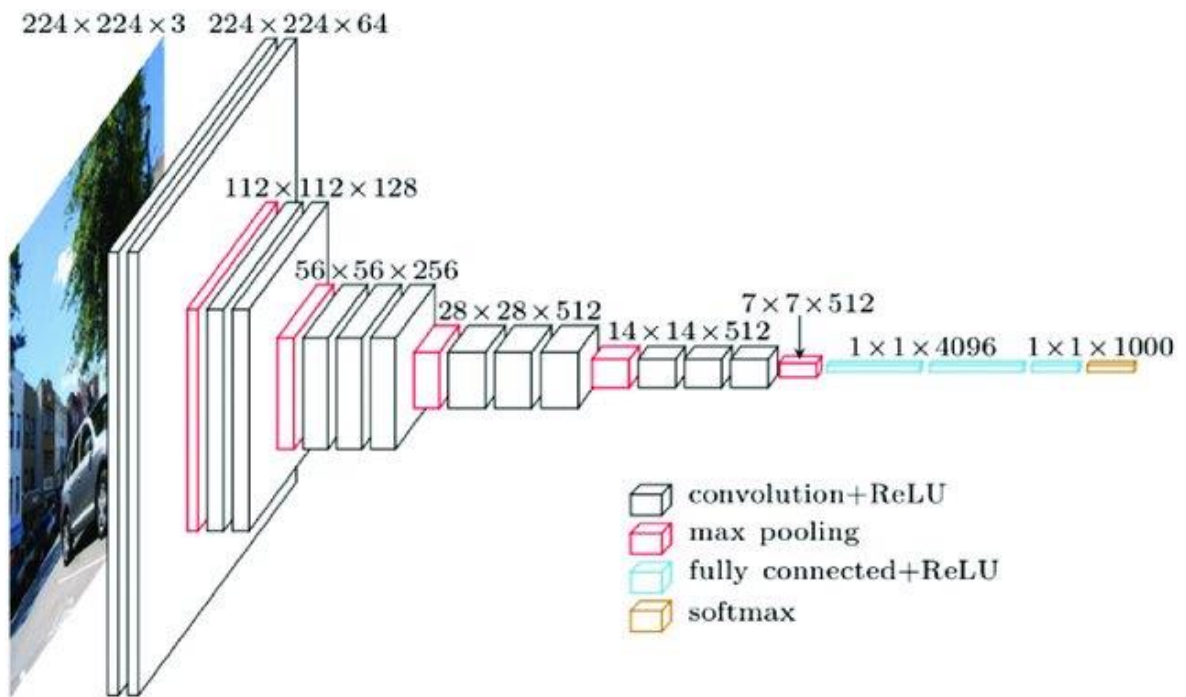
2.9.4 Phép toán Stride

Stride là bước nhảy của kernel khi quét qua ảnh. Stride lớn hơn 1 sẽ làm giảm kích thước đầu ra, đồng thời giúp giảm tính toán và làm cho việc trích xuất các đặc trưng tổng quát hơn.

2.10 Một số kiến trúc mạng neuron tích chập hiện đại

2.10.1 Mạng VGG

VGG là mạng CNN sâu được đề xuất bởi nhóm Oxford năm 2014. Đặc trưng chính của VGG là sử dụng nhiều lớp tích chập nhỏ 3x3 liên tiếp và xen kẽ với các lớp pooling. VGG đơn giản nhưng rất mạnh mẽ trong việc trích xuất đặc trưng không gian, được sử dụng rộng rãi trong thị giác máy tính.



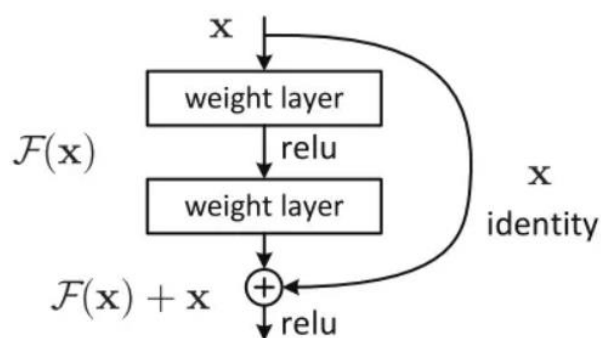
Hình 2.24. Hình ảnh về kiến trúc VGG-16

Nguồn: [\(PDF\) A review of deep learning in the study of materials degradation](#)

2.10.2 Mạng ResNet

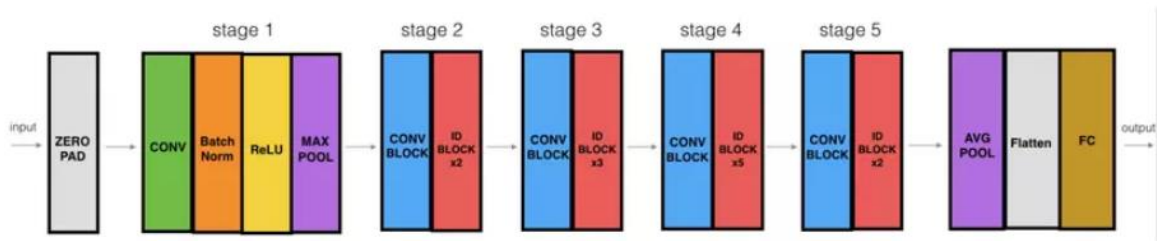
ResNet giải quyết vấn đề Vanishing Gradients (Suy giảm gradient). Bằng cách sử dụng kết nối tắt (Skip connection), ResNet cho phép tín hiệu gradient đi xuyên qua một hay nhiều lớp mà không bị yếu dần đi (Chống lại việc đạo hàm bằng 0).

Mỗi khối Residual Block thực hiện như sau:



Hình 2.25. Sơ đồ của một khối Residual Block

Nguồn: [Giới thiệu mạng ResNet](#)



Hình 2.26. Kiến trúc mạng ResNet-50

Nguồn: [Giới thiệu mạng ResNet](#)

2.10.3 Một số kiến trúc khác

- Inception: Sử dụng nhiều kích thước kernel song song để trích xuất đặc trưng đa cấp.
- EfficientNet: Tối ưu hóa đồng thời chiều sâu, rộng và độ phân giải.
- MobileNet: Kiến trúc nhẹ, tối ưu cho các thiết bị di động nhờ tích chập depthwise separable.

2.11 Bài toán phát hiện đối tượng

2.11.1 Giới thiệu về bài toán

Bài toán phát hiện đối tượng (Object detection) là một trong những nhiệm vụ cốt lõi trong thị giác máy tính (Computer Vision), với mục tiêu xác định sự hiện diện, vị trí, và nhãn lớp của các đối tượng có trong một ảnh đầu vào. Khác với bài toán phân loại ảnh (image classification) vốn chỉ đưa ra nhãn cho toàn bộ ảnh, hay bài toán phân đoạn (segmentation) vốn cần định vị chi tiết từng pixel, bài toán phát hiện đối tượng nhắm đến việc xác định một hoặc nhiều bounding box (hộp giới hạn) cùng với nhãn lớp tương ứng cho mỗi đối tượng có trong ảnh.

2.11.2 Mục tiêu

Một hệ thống phát hiện đối tượng sẽ có đầu ra là:

- Một tập các bounding box có tọa độ hình chữ nhật (x_{min} , y_{min} , x_{max} , y_{max}).
- Một tập các nhãn lớp tương ứng với các đối tượng được phát hiện.
- (Tùy chọn) Một tập xác suất tin cậy (confidence scores) biểu thị độ tin cậy cho từng dự đoán.

2.11.3 Các hướng tiếp cận chính

Các phương pháp phát hiện đối tượng hiện đại có thể được chia thành hai nhóm chính:

- Nhóm hai giai đoạn (two -stage dectectors): Đại diện tiêu biểu là R-CNN, Fast R-CNN, Faster R-CNN. Hướng tiếp cận này tách biệt hai bước:
 - 1) Đề xuất vùng nghi ngờ (Region Proposal).
 - 2) Phân loại và tinh chỉnh bounding box.
- Nhóm một giai đoạn (one -stage dectectors): Đại diện tiêu biểu là YOLO. Nhóm tiếp cận này trực tiếp dự đoán bounding box và lớp từ toàn bộ ảnh đầu vào trong một bước duy nhất, với ưu điểm tốc độ nhanh, phù hợp với thời gian thực.

Trong khóa luận này, chúng tôi lựa chọn mô hình thuộc nhóm one -stage cụ thể là YOLOv8 vì sự cân bằng tốt giữa hiệu năng và tốc độ suy luận.

2.11.4 Ứng dụng thực tiễn

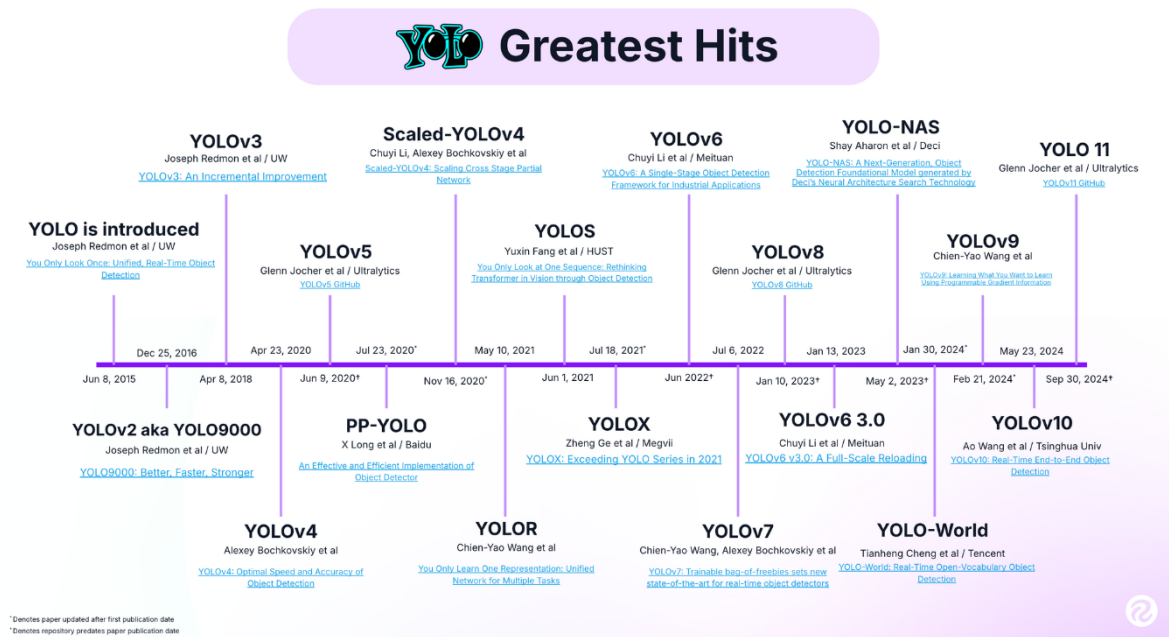
Phát hiện đối tượng là nền tảng cho nhiều bài toán thị giác ứng dụng như:

- Hệ thống xe tự lái
- Giám sát an ninh
- Nhận diện vật thể trong công nghiệp (robot, y tế, công nghiệp, ...)

Chương 3. Các công trình liên quan

3.1 Sự xuất hiện của YOLO và các phiên bản nổi bật

YOLO (You Only Look One) là một mô hình phát hiện đối tượng theo thời gian thực được giới thiệu bởi Joseph Redmon và các người cộng sự của mình vào năm 2015. Dưới đây là hình ảnh về sự phát triển YOLO qua các năm [18] [19] [20] [21].



Hình 3.1. Mô tả quá trình phát triển của YOLO từ 2015 đến hiện tại

Nguồn: [What is YOLO? The Ultimate Guide \[2025\]](#)

Giới thiệu sơ lược về một số phiên bản YOLO nổi bật cũng như những ưu điểm và cải tiến đáng kể của các phiên bản YOLO mới so với các phiên bản YOLO tiền nhiệm.

- YOLOv2 là phiên bản cải tiến của YOLOv1 được phát triển vào năm 2016 sử dụng thêm các kỹ thuật như chuẩn hóa dữ liệu theo lô, anchor boxes giúp cải thiện độ chính xác. Loại bỏ đi tầng Fully Connected giúp giảm tham số tăng tốc độ xử lý.
- YOLOv3 là phiên bản phát triển tiếp tục của YOLOv2 vào năm 2016 bằng cách sử dụng một mạng mới là DarkNet-53 làm backbone để trích xuất đặc trưng, sử dụng Multi - scale Prediction dự đoán ở 3 cấp độ (13x13, 26x26, 52x52) nhằm cải thiện đối tượng nhỏ.

- YOLOv4 được giới thiệu vào năm 2020 mô hình này tiếp tục nâng cao độ chính xác cũng như là hiệu suất nhờ áp dụng những kỹ thuật như CSPDarknet-53 + Mish, PANet (Path Aggregation NetWork), SPP (Spatial Pyramid Pooling), SAM (Spatial Attention Module). Thông qua đó giúp YOLOv4 có khả năng phát hiện đa dạng và chính xác cao hơn so với YOLOv3.
- YOLOv5 được phát triển vào 2020 được viết lại bằng Pytorch, backbone CSPDarknet-53 được sửa đổi thêm SPPF, cải tiến PANet thành CSP PAN, tự động tối ưu hóa anchor (AutoAnchor). Từ đó giúp mô hình cải thiện độ chính xác tốt hơn và hiệu suất nhanh hơn
- YOLOv6 được giới thiệu vào năm 2022 sử dụng kiến trúc mới tên là EfficientRep dựa trên RepVGG để tăng tốc inference, gồm các khối RepBlock, ERBlock, và SimSPPF. Sử dụng Anchor free để tăng tốc độ cho Non-Max Suppression, ngoài ra còn sử dụng Decoupled head giúp phân nhiệm vụ từ đó mô hình dự đoán tốt hơn
- YOLOv7 được giới thiệu vào năm 2022 sử dụng kiến trúc E-ELAN để cải thiện khả năng trích xuất đặc trưng mà vẫn giữ tốc độ suy luận. Mô hình mở rộng đồng thời chiều sâu và chiều rộng giúp tối ưu hiệu quả mạng. Bên cạnh đó, YOLOv7 áp dụng nhiều kỹ thuật Bag-of-Freebies để tăng độ chính xác mà không làm chậm quá trình suy luận
- YOLOv8 ra mắt vào năm 2023 bởi Ultralytics, đánh dấu bước chuyển mình lớn nhất trong toàn bộ dòng mô hình YOLO khi lần đầu tiên áp dụng triệt để thiết kế anchor-free và decoupled head. Việc loại bỏ anchor boxes giúp đơn giản hóa quá trình huấn luyện và suy luận, đồng thời tăng tốc độ xử lý nhờ giảm đáng kể chi phí tính toán cho Non-Max Suppression (NMS). Ngoài ra sử dụng decoupled head tách hai nhiệm vụ phân loại và hồi quy giúp mô hình học ổn định hơn. Về backbone sử dụng khối C2f (Concat-to-Fusion) thay thế cho các khối CSP truyền thống trong backbone, nhằm giảm số lượng tham số nhưng vẫn giữ nguyên khả năng truyền đặc trưng.

Ngoài ra YOLO còn phát triển rất nhiều những nhánh nhỏ, to khác nhau nhằm phục vụ một số tác vụ nhất định hay kích thước đầu vào cụ thể. Tuy nhiên trong đề

tài này chúng tôi lựa chọn mô hình YOLOv8 để nghiên cứu và áp dụng. Để nói chi tiết thì chúng tôi sẽ giải thích kỹ càng hơn về YOLOv8 và các phiên bản của nó thông qua chỉ mục 3.4 ở bên dưới đây.

3.2 Mô hình YOLO

3.2.1 Định nghĩa về YOLO

YOLO (You Only Look One) [18] [22] là một mô hình mạng CNN dành cho nhiệm vụ nhận diện, phát hiện và phân loại đối tượng. YOLO được tạo ra từ việc kết hợp các convolutional layers và connected layers.

Trong đó:

- Convolutional layers sẽ trích xuất ra các đặc trưng của ảnh.
- Fully – connected layers sẽ dự đoán ra xác suất đó và tọa độ của đối tượng.

3.2.2 Đặc điểm

YOLO xử lý toàn bộ ảnh trong một lần truyền (single pass), khác với các mô hình như R-CNN hay Fast R-CNN đòi hỏi nhiều bước. YOLO giúp tăng tốc độ và phù hợp cho ứng dụng thời gian thực. Ngoài ra YOLO sử dụng linear regression để dự đoán các thông tin ở mỗi ô vuông. Do đó ở layer cuối cùng chúng ta sẽ không có sử dụng bất kì hàm kích hoạt nào cả.

3.2.3 Input

Input tensor của YOLO là một mảng 3D đại diện cho ảnh đầu vào có định dạng $[H, W, C]$.

Trong đó:

- H là chiều cao
- W là chiều rộng
- C là số kênh màu (thường là 3 cho RGB)

3.2.4 OutPut

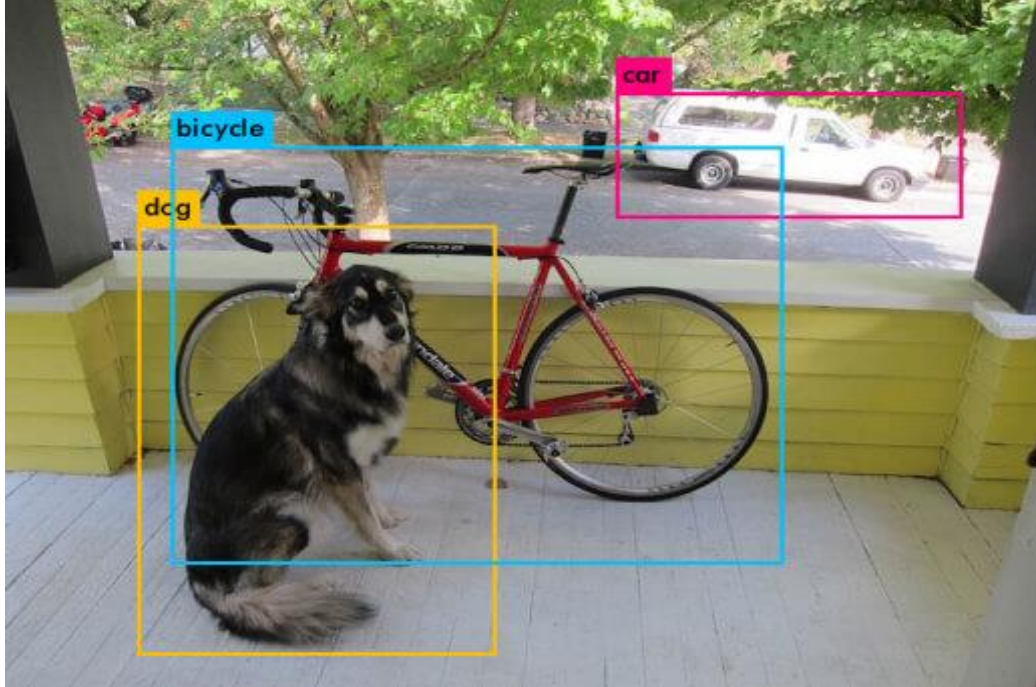
Output tensor của YOLO là một mảng 3D với kích thước:

$$[S, S, (B \times 5 + C)] \quad (3.1)$$

Trong đó:

- S là kích thước ô lưới ($S \times S$)

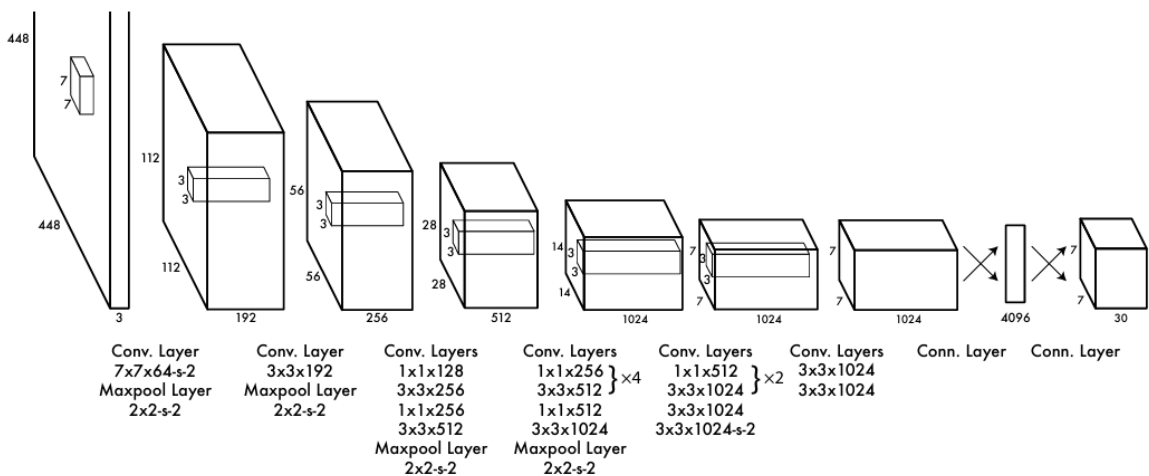
- B x 5: Bao gồm x, y (tọa độ trung tâm), w (chiều rộng), h (chiều cao) và object score.
- C: Xác suất thuộc các lớp (ví dụ: C=0.92 thuộc lớp dog)



Hình 3.2. Ảnh OutPut mô phỏng của YOLO

Nguồn: [YOLO — Intuitively and Exhaustively Explained | by Daniel Warfield](#)
[| Intuitively and Exhaustively Explained | Medium](#)

3.2.5 Kiến trúc YOLO



Hình 3.3. Kiến trúc mô hình YOLO: 24 convolutions

Nguồn: arxiv.org

YOLO là một kiến trúc được thiết kế lại từ đầu (from scratch) sử dụng kiến trúc mạng CNN gồm 24 tầng convolutions và 2 tầng FullyConnected. Với mục tiêu là tối ưu về tốc độ và đơn giản hóa quá trình phát hiện đối tượng (Object detection) trong một forward pass duy nhất.

Trong đó 24 tầng convolutions có mục tiêu là để trích xuất những đặc trưng (features):

- Các bộ lọc 3x3 đi kèm với stride 1 và padding 1: Dùng để mở rộng vùng nhìn (receptive field) theo chiều sâu của mạng điều này cho phép mô hình nắm bắt tốt hơn bối cảnh rộng hơn. Ngoài ra việc dùng stride và padding giúp cho không gian đặc trưng được bảo toàn không bị thiếu chi tiết nhất là ở rìa.
- Các bộ lọc 1x1 có vai trò tinh chỉnh và kết hợp các đặc trưng cục bộ, làm tăng thêm chiều sâu mô hình mà không tăng nhiều chi phí tính toán.
- Xen kẽ là tầng subsampling cụ thể là max pooling nhằm giảm kích thước chiều không gian và giữ lại các đặc trưng nổi bật quan trọng nhất.

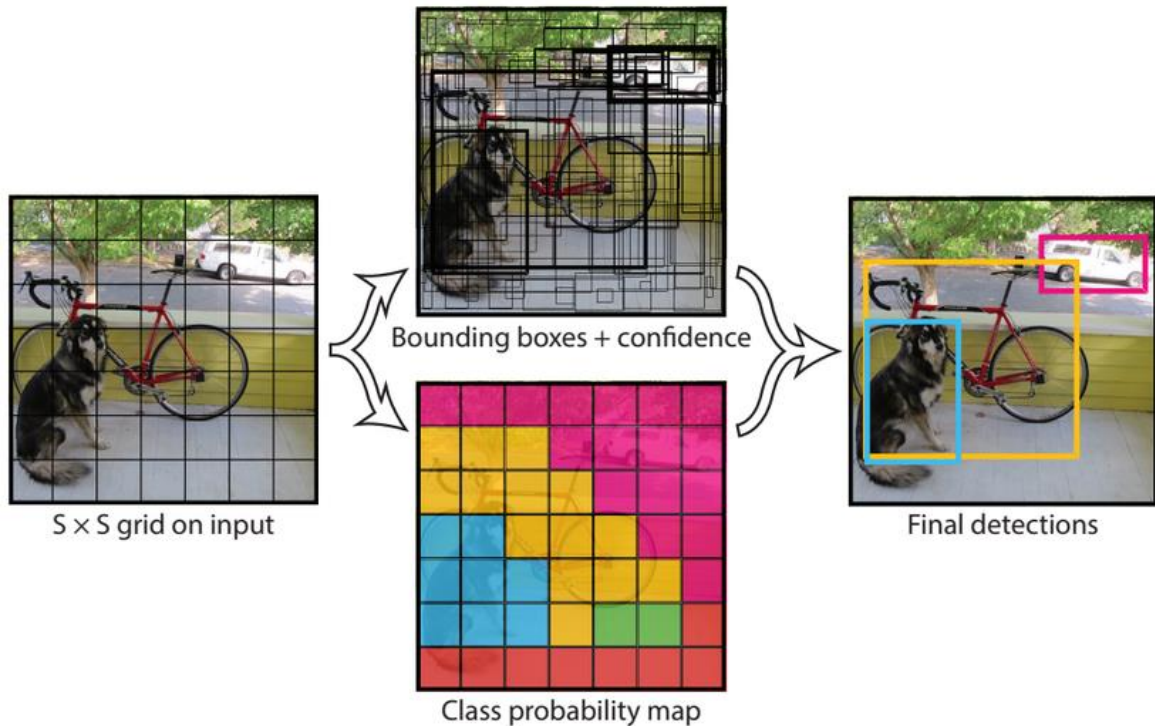
Ngoài ra còn có 2 tầng Fully Connected đóng vai trò như là trung tâm tổng hợp và dự đoán các đặc trưng:

- Dự đoán ra các bounding box và các class tương ứng trên từng ô lưới.
- Và vì đây là một bài toán hồi quy (regression) nên không sử dụng activation function ở lớp đầu ra output mà trực tiếp sử dụng các đặc trưng để đưa ra dự đoán.

Tuy nhiên, YOLOv1 vẫn có những hạn chế như là độ chính xác không cao với các vật thể nhỏ, bị chồng lấp do chia ảnh thành lưới 7x7, dẫn đến mỗi ô trách nhiệm cho một vùng ảnh lớn. Điều này thì dần được cải thiện ở các phiên bản sau như là YOLOv2, YOLOv3, ..., YOLOv8, ... bằng những cơ chế như thêm nhiều tầng đặc trưng có kích thước khác nhau, hay sử dụng anchor box, anchorfree ...

3.2.6 Cách hoạt động

- Đầu vào: hình ảnh được đưa về 448x448.
- Chia lưới hình ảnh: Hình ảnh 448x448 được chia thành lưới 7x7. Mỗi ô lưới chịu trách nhiệm dự đoán đối tượng có tâm nằm trong ô đó.
- CNN backbone: Đảm nhiệm phần trích xuất đặc trưng với các tầng convolution và pooling (cụ thể Maxpooling).
- Đầu ra: tensor dạng $[S, S, (5 \cdot n_boundingbox + n_class)]$.



Hình 3.4. Hình ảnh quá trình hoạt động của YOLO

Nguồn: [Tìm hiểu về YOLO trong bài toán real-time object detection](#)

3.2.7 Hàm lỗi

YOLO sử dụng một hàm mất tổng hợp gồm 3 thành phần lỗi sau:

1) Classification Loss:

Mục đích: Đo sai lệch giữa xác suất của lớp được dự đoán và lớp ground truth của đối tượng trong mỗi ô lưới **có chứa đối tượng**.

Công thức:

$$L_{\text{class}} = \sum_{i=0}^{S^2} I_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (3.2)$$

Nguồn: [YOLO — Intuitively and Exhaustively Explained | by Daniel Warfield](#)
[| Intuitively and Exhaustively Explained | Medium](#)

2) Localization Loss

Mục đích: Đo sai lệch giữa tọa độ bounding box được dự đoán và ground-truth box, bao gồm:

- Center coordinates: $x, y \rightarrow$ tương đối trong ô lưới (giá trị từ 0–1)
- Width & height $\rightarrow$ sử dụng căn bậc hai: $\sqrt{w} + \sqrt{h}$ để tránh ưu tiên cho các object lớn

Công thức:

Localization Loss:

$$L_{\text{box}} = L_{\text{confidence}} + L_{\text{coordinates}} \quad (3.3)$$

Loss coordinates:

$$L_{\text{coordinates}} = \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B II_{ij}^{\text{obj}} l_i \quad (3.4)$$

$$l_i = (\sqrt{w_i} - \sqrt{\widehat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\widehat{h}_i})^2 + (x_i - \widehat{x}_i)^2 + (y_i - \widehat{y}_i)^2 \quad (3.5)$$

3) Confidence Loss

Mục đích: Đo sự khác biệt giữa:

- Objectness score được dự đoán: $P(\text{obj}) * IoU$
- Và ground-truth (1 nếu có object, 0 nếu không có object)
- Tính cho cả:
 - Grid cell có object $\rightarrow$ yêu cầu dự đoán confidence gần 1.
 - Grid cell không có object $\rightarrow$ yêu cầu dự đoán confidence gần 0

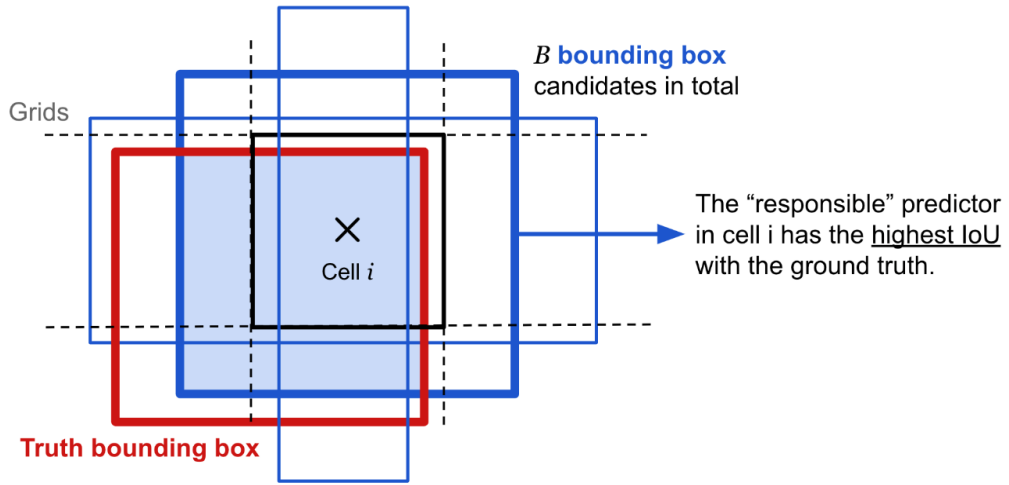
Công thức Confidence Loss:

$$L_{\text{confidence}} = \sum_{i=0}^{S^2} \sum_{j=0}^B II_{ij}^{\text{obj}} (c_i - \hat{c}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B II_{ij}^{\text{noobj}} (c_i - \hat{c}_i)^2 \quad (3.6)$$

3.3 Anchor Boxes

YOLOv2 là bước tiến lớn trong thể hệ YOLO khi đưa vào sử dụng Anchor Boxes [23] thay vì để cho mô hình tự dự đoán ngẫu nhiên không có hình dạng mẫu trước làm cho việc dự đoán bounding box khó tối ưu khi huấn luyện như phiên bản trước.

Vậy thực tế Anchor Boxes này làm gì mà khiến cho việc mô hình không còn tự dự đoán ngẫu nhiên bounding box nữa? Về mặt ý tưởng thì Anchor Boxes sẽ như là một khuôn mẫu định sẵn ép những dự đoán bounding box của mô hình theo chiếc khuôn đó và dự đoán. Hình ảnh dưới đây sẽ là ví dụ cho việc chúng ta thiết lập khuôn mẫu cho bounding box dự đoán:



Hình 3.5. Hình ảnh liên quan đến Anchor Boxes

Nguồn: [Khoa học dữ liệu](#)

Dựa vào hình trên ta sẽ thấy rằng các ô được tô màu xanh dương đậm thể hiện các khuôn mẫu bounding box cho một đối tượng trên một ô lưới của feature map có chứa điểm mid point của đối tượng. Với các khuôn mẫu này khi dự đoán bounding box xung quanh đối tượng thì mô hình sẽ căn cứ vào những ô có hình dạng kích thước như này cho một đối tượng cụ thể điều này giúp mô hình học dự đoán chính xác hơn các hộp bounding sao cho phù hợp với khuôn mẫu tăng tốc độ dự đoán.

Tuy nhiên vẫn sẽ có trường hợp ngoại lệ đó là vô tình 2 hay nhiều điểm mid point đều tập trung vào một ô lưới thì sẽ khiến cho việc mô hình khó dự đoán chính xác được lớp nào như hình dưới đây:

Anchor box example



Hình 3.6. Mô tả việc bị trùng lặp mid point

Nguồn: [Khoa học dữ liệu](#)

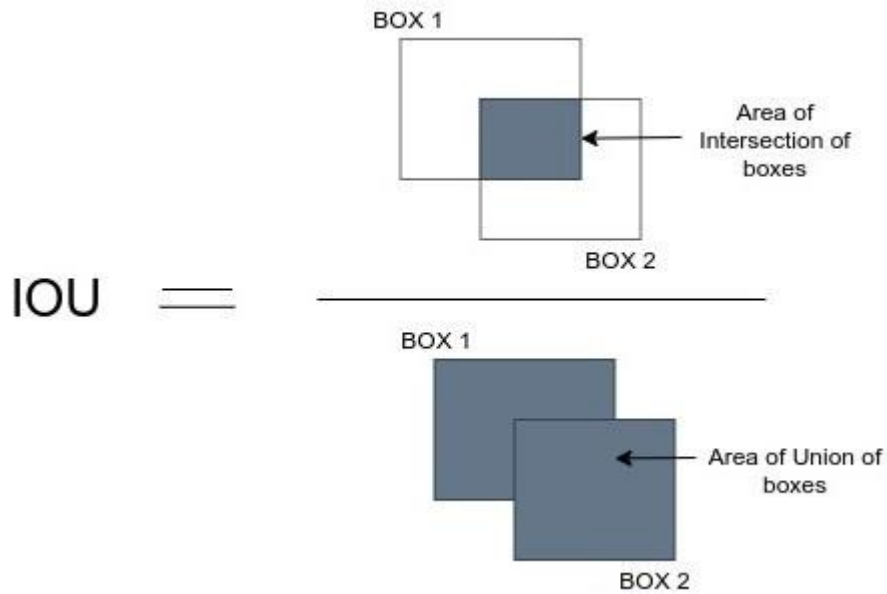
Do việc mid point của đối tượng xe và người cùng chung một điểm dẫn đến sẽ có thể khiến cho mô hình cân nhắc trước khi đưa ra dự đoán tên lớp sao cho phù hợp với đối tượng được phát hiện.

3.4 IoU

IoU (Intersection over Union) là chỉ số đo lường mức độ sự trùng khớp giữa vùng được phát hiện (predict bounding box) và vùng nhãn thật (growth truth bounding box)

Công thức tính:

$$IoU = \frac{\text{Diện tích giao nhau giữa 2 bounding box}}{\text{Diện tích hợp nhất giữa 2 bounding box}} \quad (3.7)$$



Hình 3.7. Hình ảnh về cách tính IoU

Nguồn: [Tìm hiểu và triển khai thuật toán Non Maximum Suppression](#)

Giá trị IoU nằm trong khoảng $[0, 1]$, IoU càng cao thì có nghĩa là vùng dự đoán càng khớp với vùng thật. Trong việc huấn luyện và đánh giá phát hiện đối tượng, IoU là tiêu chí quan trọng để quyết định xem một phát hiện được coi là đúng (true positive) hay không.

3.5 Non-max suppression (NMS)

Thuật toán YOLO sau khi chạy sẽ dự báo ra rất nhiều bounding box trên một bức ảnh tùy vào số lượng của class cũng như số lượng bounding box cần thiết để xác định ra đối tượng mà ta mong muốn chính vì thế mà sẽ có những đối tượng (object) sẽ bị chồng chéo lên nhau điều đó làm giảm khả năng phán đoán chính xác cụ thể vật thể đó ở vị trí nào. Do đó YOLO sử dụng NMS (Non-max suppression) để giảm bớt số lượng những khung được sinh ra một cách rất đáng kể.



Hình 3.8. Hình ảnh mô tả về cách thức Non-Max Suppression hoạt động

Nguồn: [Non-Maximum Suppression with OpenCV and Python - The Python Code](#)

Ở ví dụ trên ta thấy rõ ban đầu có 4 bounding box được sinh ra bao quanh chiếc xe sau đó áp dụng Non-max Suppression thì chỉ còn 1 chiếc bounding box duy nhất.

Vậy các bước Non-max Suppression đã làm là gì ? Đó là:

- Bước 1: Trước tiên chúng ta sẽ làm là giảm bớt số lượng các bounding box đi bằng cách lọc bỏ toàn bộ những bounding box có xác suất chứa vật thể nhỏ hơn ngưỡng nào đó, thông thường là 0.5.
- Bước 2: Đối với các bounding còn lại nếu mà chúng overlap nhau, thì NMS sẽ chọn ra một bounding box có xác suất (confidence) chứa vật thể là lớn nhất. Sau đó tính toán các chỉ số giao nhau (IoU) với các bounding box còn lại.

Nếu giá trị IoU lớn hơn hoặc bằng ngưỡng IoU đã xác định trước (ví dụ $\text{IoU} > 0.5$) thì lúc đó NMS nhận định rằng có sự chồng lấn giữa các bounding box còn lại vậy thì ở đây NMS loại bỏ những bounding box thừa đi chỉ lấy lại bounding box có điểm tin cậy cao nhất. Lúc này ta thu về được một bounding box duy nhất.

3.6 YOLOv8

3.6.1 Input

- Mô hình YOLOv8 nhận ảnh RGB làm ảnh đầu vào với kích thước chuẩn hóa về (640 x 640 x3).
- Trước khi đưa vào mô hình (backbone):
 - Ảnh được resize về kích thước 640x640.
 - Chuẩn hóa giá trị pixel về khoảng [0, 1].

- Biến đổi ảnh thành tensor: $[B, 3, 640, 640]$ trong đó:
 - B: Batch_size (số ảnh trong một lần forward).
 - 3: Số kênh màu (RGB).

3.6.2 Output

• YOLOv8 sử dụng Anchor-Free Head, do đó đầu ra sẽ không dựa vào anchor box cố định như các phiên bản cũ.

- Đầu ra của mô hình là tensor dạng: $[N, 4+1+C]$

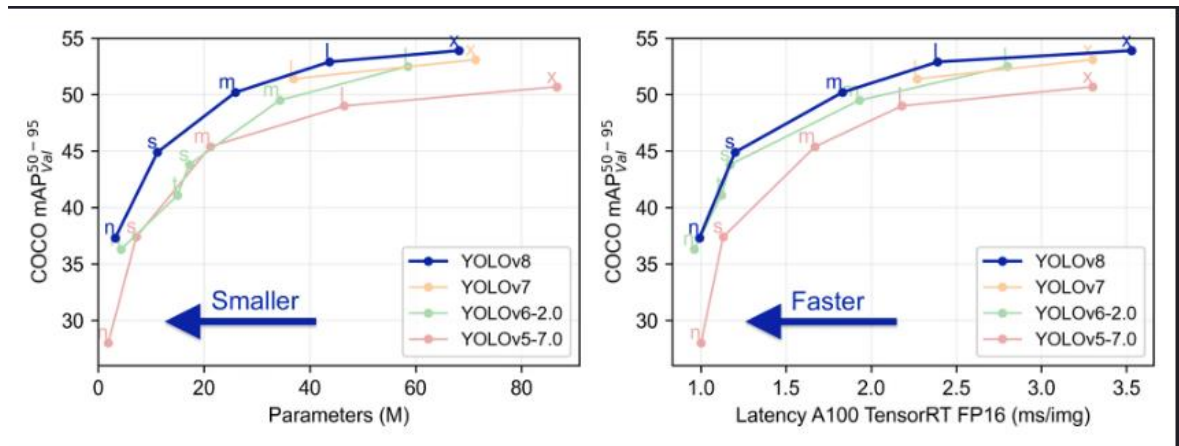
Trong đó:

- N: Là số điểm đặc trưng (grid cell) trên feature map.
- 4: Tọa độ bounding box (x_{center} , y_{center} , width, height).
- 1: Objectness score – xác suất tồn tại đối tượng tại điểm đó.
- C: Số lớp cần phân loại.

3.6.3 Kiến trúc YOLOv8

YOLOv8 [24] [25] [26] [27] có nhiều phiên bản kiến trúc khác nhau nhằm cho nhiều bài toán khác nhau như là: Object Detection, Instance Segmentation, Pose/Keypoint, Oriented Dectection, Classification.

Ở đề tài này chúng tôi sẽ sử dụng phiên bản YOLOv8n cho nhiệm vụ Object Detection (Nhận diện đối tượng) thì ở nhiệm vụ này YOLOv8 có tổng cộng năm kích cỡ khác nhau cụ thể là: YOLOv8n cho mô hình kích thước cực nhỏ (khoảng 3.2 triệu tham số), YOLOv8s cho mô hình có kích thước nhỏ, YOLOv8m cho mô hình kích thước trung bình, YOLOv8l cho kích thước lớn và cuối cùng là YOLOv8x cho kích thước cực lớn.



Hình 3.9. Hiệu năng và tham số của YOLOv8 qua từng version

Nguồn: [Explore Ultralytics YOLOv8 - Ultralytics YOLO Docs](#)

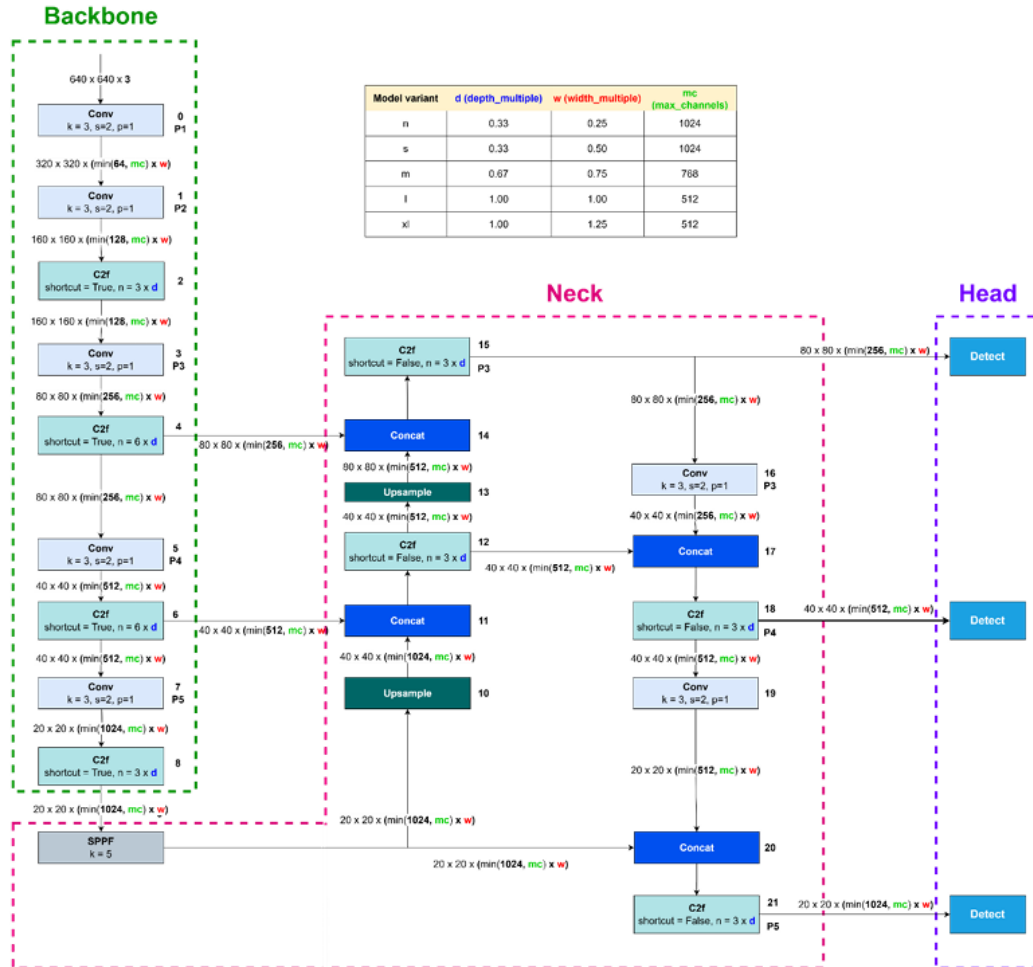
Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M) $\updownarrow$	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Hình 3.10. Bảng chi tiết tổng hợp về các phiên bản YOLOv8 trên tập COCO

Nguồn: [Explore Ultralytics YOLOv8 - Ultralytics YOLO Docs](#)

Tất cả các kích thước của mô hình YOLOv8 trong bài toán detection đều tạo từ 3 thành phần giống nhau :

- Backbone: Sử dụng backbone của YOLOv5 CSPDarknet53 với các khối C2F thay cho C3 để trích xuất các đặc trưng
- Neck: sử dụng các cơ chế có khả năng tổng hợp đặc trưng đa tỉ lệ mạnh như SPPF để tổng hợp đặc trưng
- Head: Sử dụng kiến trúc decoupled head để xử lý thông tin độc lập cho các nhiệm vụ như phát hiện, phân loại



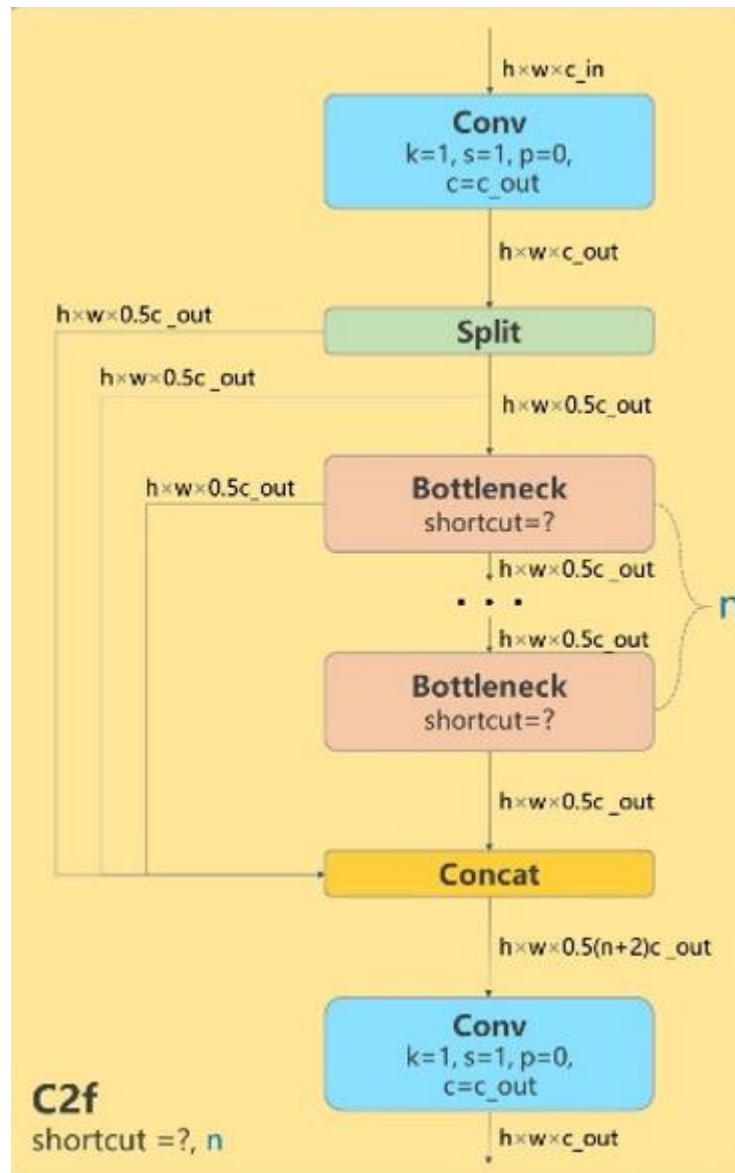
Hình 3. 11. Kiến trúc của YOLOv8

Nguồn: [Mastering All YOLO Models from YOLOv1 to YOLOv12](#)

Phần Backbone

CSPDarknet53: Backbone là phần đầu tiên của mô hình, chịu trách nhiệm trích xuất các đặc trưng hữu ích từ ảnh đầu vào. Trong YOLOv8, backbone sử dụng một kiến trúc tương tự như CSPDarknet53 đã được sửa đổi, vốn là xương sống của YOLOv5. Điểm nổi bật của trong cải tiến của YOLOv8 so với các phiên bản YOLO trước nó là sử dụng khối C2F thay cho C3.

Khối C2F (C2F Block)



Hình 3.12. Hình ảnh về cấu trúc của C2F

Nguồn: [Mastering All YOLO Models from YOLOv1 to YOLOv12](#)

Trong kiến trúc C2F, đầu vào đầu tiên được đi qua một lớp Conv để điều chỉnh số kênh. Kết quả tách ra thành nhiều nhánh, trong đó:

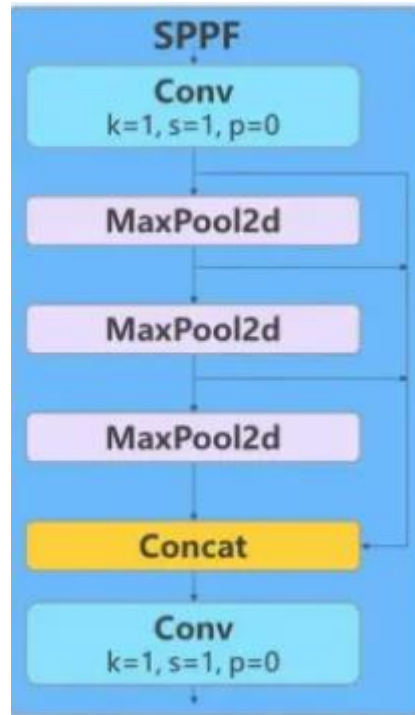
- Một phần được giữ nguyên
- Phần còn lại đi qua một chuỗi Bottleneck (có thể lặp chuỗi này n lần) với mục tiêu là trích xuất đặc trưng tốt hơn, sâu hơn.

Sau đó các nhánh sẽ được concat lại theo chiều kênh và đi qua một lớp Conv cuối để nén lại kích thước, tạo đầu ra với số kênh bằng c_{out} như thiết kế. C2F giữ nguyên sức mạnh C3 nhưng giảm thiểu chi phí tính toán đồng thời làm giàu thông tin

đầu ra nhờ chiến thuật concat rất thông minh. Đây là bước cải tiến quan trọng giúp YOLOv8 đạt hiệu năng tốt hơn với mô hình nhẹ hơn.

Phần Neck

- SPPF (Spatial Pyramid Pooling Fast): Được sử dụng ngay sau lớp tích chập cuối cùng trong backbone.



Hình 3.13. Hình ảnh về SPPF (Spatial Pyramid Pooling Fast)

Nguồn: [YOLOv8 là gì? Hướng dẫn đầy đủ](#)

Mục tiêu: Là tạo ra một biểu diễn đặc trưng cố định theo không gian cho các đối tượng có kích thước khác nhau trong ảnh mà không cần thay đổi kích thước ảnh hay mất đi thông tin không gian. SPPF giúp tăng độ tốc độ tính toán của mạng bằng cách gom các đặc trưng từ tỉ lệ ảnh khác nhau vào một feature map có kích thước cố định.

Cấu trúc của SPPF: Bao gồm một khối tích chập ban đầu, sau đó là ba lớp MaxPool2d liên tiếp. Điều đặc biệt là mỗi feature map kết quả từ MaxPool2d đều được nối (concatenated) lại ngay trước khi kết thúc khối SPPF bằng một khối tích chập khác.

Vậy với SPPF sẽ đảm bảo rằng việc tổng hợp các đặc trưng về không gian sẽ được tổng hợp tốt hơn vì nó sẽ concat với nhiều tỷ lệ đặc trưng khác nhau mà không phải thay đổi kích thước ảnh.

Phần Head

AnchorFree: Trong khi AnchorBoxes từng là một bước tiến quan trọng (từ YOLOv2). Thay vì sử dụng các Anchor Boxes như các phiên bản trước của YOLO thì ở phiên bản này YOLOv8 sử dụng AnchorFree thứ vốn đơn giản và ít phụ thuộc vào các hyperparameters và phù hợp với các ứng dụng thực tế theo thời gian thực.

Cơ chế hoạt động AnchorFree khá đơn giản: Với mỗi bounding box thật trong ảnh, mô hình sẽ xác định tâm (center point) của đối tượng sau đó ánh xạ điểm này vào Feature map. Tại vị trí tương ứng trên featuremap, mô hình được gán nhãn là có vật thể, đồng thời sẽ học cách hồi quy trực tiếp khoảng cách từ điểm đó đến bốn cạnh của bounding box, tương ứng với các giá trị (l, t, r, b) (left, top, right, bottom)

Trong quá trình huấn luyện thì để tối ưu YOLOv8 sử dụng 2 hàm mất mát chính cho việc sử dụng để hồi quy trực tiếp khoảng cách từ tâm ra bốn góc của vật thể đó là:

- CIOU (Complete Intersection over Union): Nhằm đánh giá mức độ trùng khớp giữa box dự đoán và box thật, có xét thêm khoảng cách giữa tâm và sự khác biệt về tỷ lệ khung hình.
- DFL (Distribution Focal Loss): Mục đích là mô hình hóa các giá trị khoảng cách (l, t, r, b) như một phân phối xác suất rời rạc, từ đó giúp cải thiện độ chính xác khi dự đoán tọa độ.

Cụ thể hơn về hai hàm lỗi này thì sẽ được chúng tôi đề cập chi tiết ở mục 3.4.4. Nhờ việc sử dụng cơ chế AnchorFree kèm theo các hàm lỗi tiên tiến như CIOU và DFL khiến cho mô hình YOLOv8 đạt hiệu suất cao cả về tốc độ lẫn độ chính xác trong bài toán phát hiện đối tượng.

Decouple Head: Trong các phiên bản trước đây như là YOLOv2, YOLOv3, YOLOv4 thì phần đầu ra (head) của mô hình thường sử dụng một nhánh duy nhất để dự đoán đồng thời cả classification, bounding box và objectness score. Cách tiếp cận này có ưu điểm là đơn giản, nhanh nhưng tồn tại một nhược điểm đó là dễ gây xung

đột giữa các tác vụ con. Để giải quyết vấn đề này thì YOLOv8 sử dụng kiến trúc Decouple Head – tức là tách riêng hai nhiệm vụ độc lập:

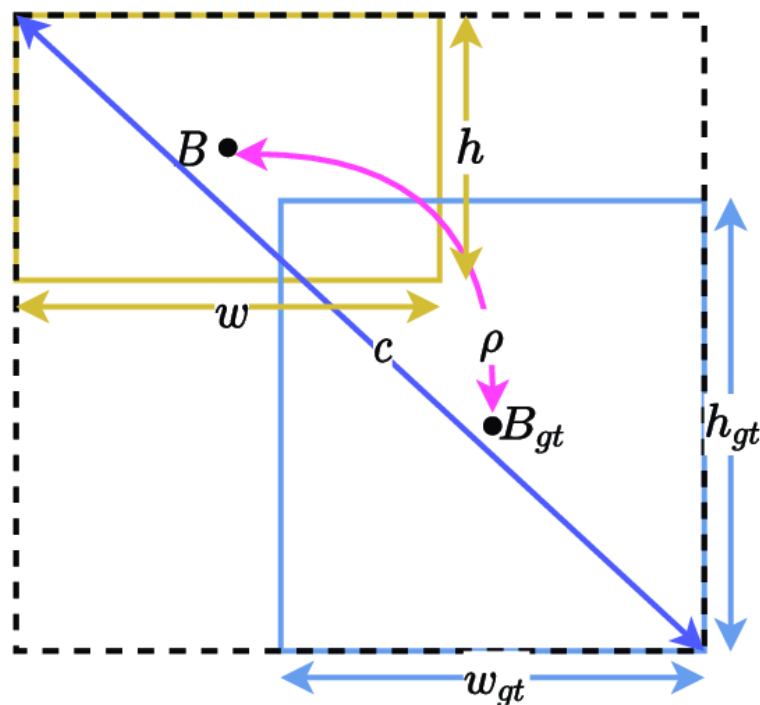
- Nhánh regression: Mục tiêu là học đặc trưng để dự đoán bounding box (tọa độ l, r, t, b)
- Nhánh classification: Mục tiêu là học đặc trưng để dự đoán objectness và class score

Việc tách riêng nhiệm vụ này giúp cho từng nhánh tập trung học sâu vào những tác vụ riêng biệt, không bị nhiễu chéo lẫn nhau trong việc cập nhật trọng số. Kết quả là mô hình sẽ hội tụ nhanh hơn, ổn định và hiệu quả hơn cho từng tác vụ nói riêng hay trong bài toán object detection (phát hiện đối tượng) nói chung.

3.6.4 Hàm lỗi

3.6.4.1 CIOU Loss

CIoU Loss là hàm lỗi dùng để huấn luyện đầu ra bounding box nghĩa là dùng để mô hình học cách dự đoán hộp chứa vật thể càng gần với bounding box ground truth từ dữ liệu huấn luyện càng tốt. Hãy cùng xem ví dụ sau để xem thực sự CIoU làm gì để cải thiện dự đoán bounding box:



Hình 3.14. Hình ảnh mô phỏng lại cách tính CIOU Loss

Nguồn: [GIoU, CIoU and DIoU: Variants of IoU and how they are better compared to IoU | by Abhishek Jain | Medium](#)

Trước tiên mô hình sẽ tính tâm của hai hộp bounding box là predict và ground truth sau đó tính khoảng cách Ecludien bình phương cho khoảng cách này. Tiếp đến tính đường chéo độ chéo đường chéo dài nhất giữa 2 hộ bounding box và bình phương nó lên. Hai việc trên giúp cho mô hình biết được rằng khoảng cách giữa 2 tâm bounding box lệch khoảng bao nhiêu. Rồi kế tiếp tính IoU là phần giao nhau giữa 2 bounding box để tính xem độ khớp giữa chúng là bao nhiêu rồi áp dụng $1 - \text{IoU}$ nếu IoU thấp thì sẽ phạt mô hình để cho nó học tốt lên. Tiếp theo là điều chỉnh tỉ lệ khung cụ thể ở bước này ta sẽ áp dụng công thức để tỉ lệ giữa chiều dài và chiều rộng của 2 bounding nếu như tỉ lệ lệch nhiều sẽ phạt để mô hình học tốt hơn sao cho tỉ lệ lệch giữa 2 bounding box là nhỏ nhất.

Công thức:

$$L_{\text{CloU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad (3.8)$$

Trong đó:

- IoU: là chỉ số đo mức độ chồng lấp giữa hai vùng: hộp dự đoán và hộp ground-truth
- $\rho(b, b^{gt})$: là khoảng cách Euclid giữa tâm hộp dự đoán và tâm hộp ground-truth
- c là độ dài đường chéo của hộp nhỏ nhất bao cả hai hộp
- v : điều chỉnh tỉ lệ khung $v = \frac{4}{\pi^2} \left(\arctan\left(\frac{w^{gt}}{h^{gt}}\right) - \arctan\left(\frac{w}{h}\right) \right)^2$
- α là trọng số cân bằng v so với IoU: $\alpha = \frac{v}{(1 - \text{IoU}) + v}$

Bảng 3.1. Mô tả chi tiết CloU Loss

Thành phần	Vai trò	Ảnh hưởng đến việc mô hình học
$1 - \text{IoU}$	Phạt nếu hộp dự đoán không giao nhiều với hộp thật	Cơ bản nhưng chưa đủ vì nếu bounding box lệch tâm hoặc tỷ lệ sai (hộp thì dài, hộp thì ngang)

$\frac{\rho^2(b, b^{gt})}{c^2}$	Phạt nếu như mô hình lệch tâm xa nhau	Bắt mô hình học vị trí một cách chính xác, tránh tình huống lệch tâm dù IoU cao
αv	Phạt nếu tỉ lệ của width/ height giữa 2 hộp bounding lệch nhau nhiều	Bắt mô hình học tỉ lệ chính xác hơn, tránh dự đoán hộp bị méo hoặc sai tỉ lệ

3.6.4.2 DFL (Distribution Focal Loss)

DFL là hàm lỗi dùng để cải thiện độ chính xác trong việc dự đoán tọa độ bounding box. DFL chuyển bài toán từ hồi quy trực tiếp (regression) sang bài toán phân loại rời rạc (discrete classification). Cụ thể ở đây là mỗi giá trị (x, y, w, h) được lượng hóa thành một phân bố xác suất trên các mốc định trước.

Ví dụ: Với tọa độ thực x là 7.6 (tương tự với y, h, w) thì DF sẽ điều hướng mô hình dự đoán gần 0.4 xác suất cho lớp 7 và 0.7 cho lớp 8. Từ phân phối xác suất này thì mô hình sẽ lấy **giá trị kỳ vọng** để ước lượng giá trị thực, đảm bảo khả năng học mượt và chính xác hơn nhất là các lớp ranh giới giữa 2 lớp.

Công thức

$$L_{DFL}(p, t) = w_l \text{CE}(p, t_l) + w_r \text{CE}(p, t_r) \quad (3.9)$$

Trong đó:

- p: phân phối xác suất đầu ra
- t: tọa độ số thực
- $t_l = \lfloor t \rfloor$ (lấy phần nguyên)
- $t_r = \lfloor t \rfloor + 1$
- $w_l = t_r - t$
- $w_r = t - t_l$
- CE: hàm cross – entropy

3.6.4.3 Binary Cross Entropy cho nhiệm vụ Objectness

Binary Cross Entropy có nhiệm vụ là đo sai lệch giữa xác suất của lớp được dự đoán và lớp ground truth của đối tượng trong mỗi ô lưới **có chứa đối tượng hoặc không chứa** từ đó giúp mô hình học cải thiện các dự đoán của các lớp

$$L_{\text{BCE}}(\hat{y}, y) = -[y \log \sigma(\hat{y}) + (1 - y) \log(1 - \sigma(\hat{y}))] \quad (3.10)$$

Trong đó:

- $\hat{y}$: giá trị đầu ra (logit) được mô hình dự đoán
- $y \in \{0,1\}$: nhãn ground truth
- $\sigma(\hat{y})$: Hàm sigmoid, chuẩn hóa logit thành xác suất

3.6.4.4 Binary Cross Entropy cho nhiệm vụ Classification

Binary Cross Entropy có nhiệm vụ là đo sai lệch giữa xác suất của lớp được dự đoán và lớp ground truth của đối tượng trong mỗi ô lưới **có chứa đối tượng** từ đó giúp mô hình học cải thiện các dự đoán của các lớp.

$$L_{\text{BCE}}(\hat{y}, y) = -[y \log \sigma(\hat{y}) + (1 - y) \log(1 - \sigma(\hat{y}))] \quad (3.11)$$

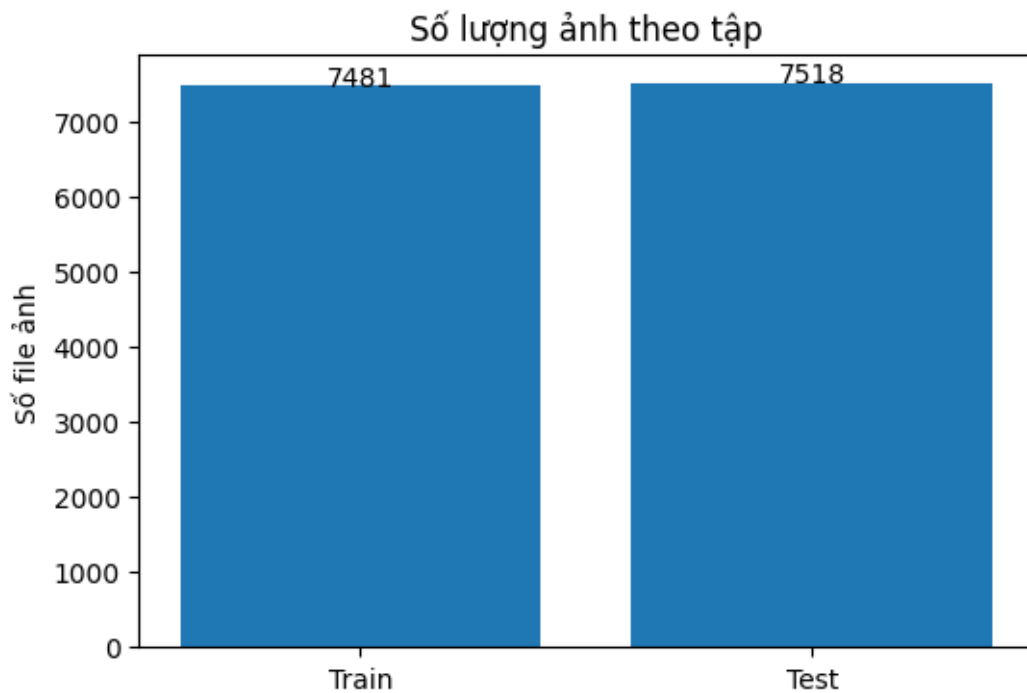
Trong đó:

- $\hat{y}$: giá trị đầu ra (logit) được mô hình dự đoán
- $y \in \{0,1\}$: nhãn ground truth
- $\sigma(\hat{y})$: Hàm sigmoid, chuẩn hóa logit thành xác suất

Chương 4. Giải pháp đề xuất

4.1 Phân tích dữ liệu

Bộ dữ liệu Object Detection Evaluation 2012 từ KITTI có tổng là 14999 ảnh. Trong đó 7481 ảnh cho tập train và 7518 ảnh cho tập test. Với 80256 đối tượng được gán nhãn. Tất cả hình ảnh đều có màu và được lưu ở dạng PNG. Đối với Point Cloud thu được từ LiDAR thì sẽ được lưu dưới dạng là .bin và cũng được chia theo tỉ lệ train test giống với tập ảnh đó là 7481 file .bin cho tập train và 7518 cho tập test.



Hình 4.1. Hình ảnh về tập phân bố tập train và test

5 ảnh ngẫu nhiên từ tập Train

000172.png



000770.png



003338.png



004157.png



005903.png



5 ảnh ngẫu nhiên từ tập Test

005255.png



003108.png



002950.png



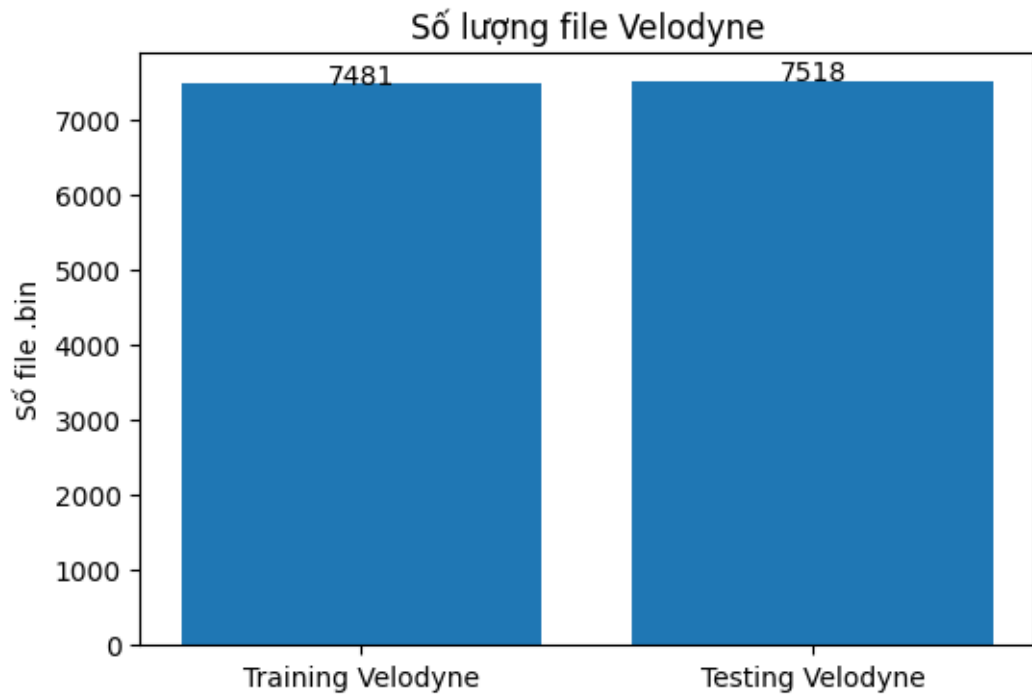
005059.png



003228.png



Hình 4.2. Hình ảnh về dữ liệu hình ảnh có trong tập train và test



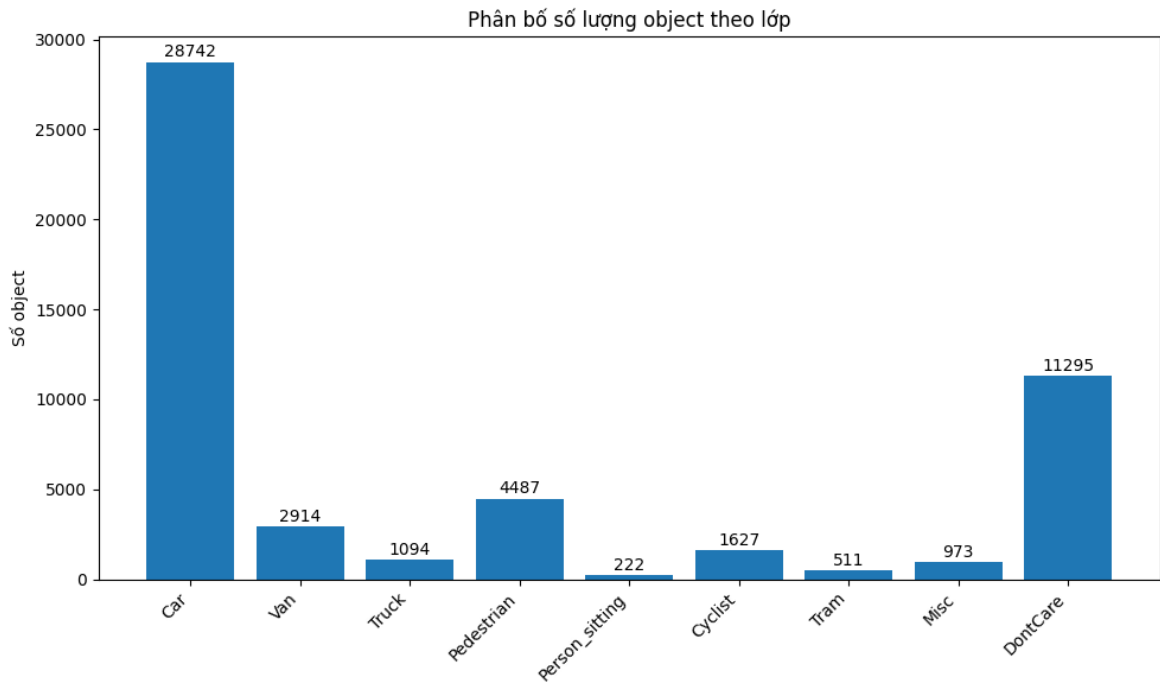
Hình 4.3. Hình ảnh phân bố dữ liệu giữa tập train và test của Point Cloud

Kích thước dữ liệu:

- Ảnh RGB (PNG) ~ 12GB
- PC (Velodyne .bin) ~ 29GB
- Calib ~ 16MB
- Label ~ 5MB

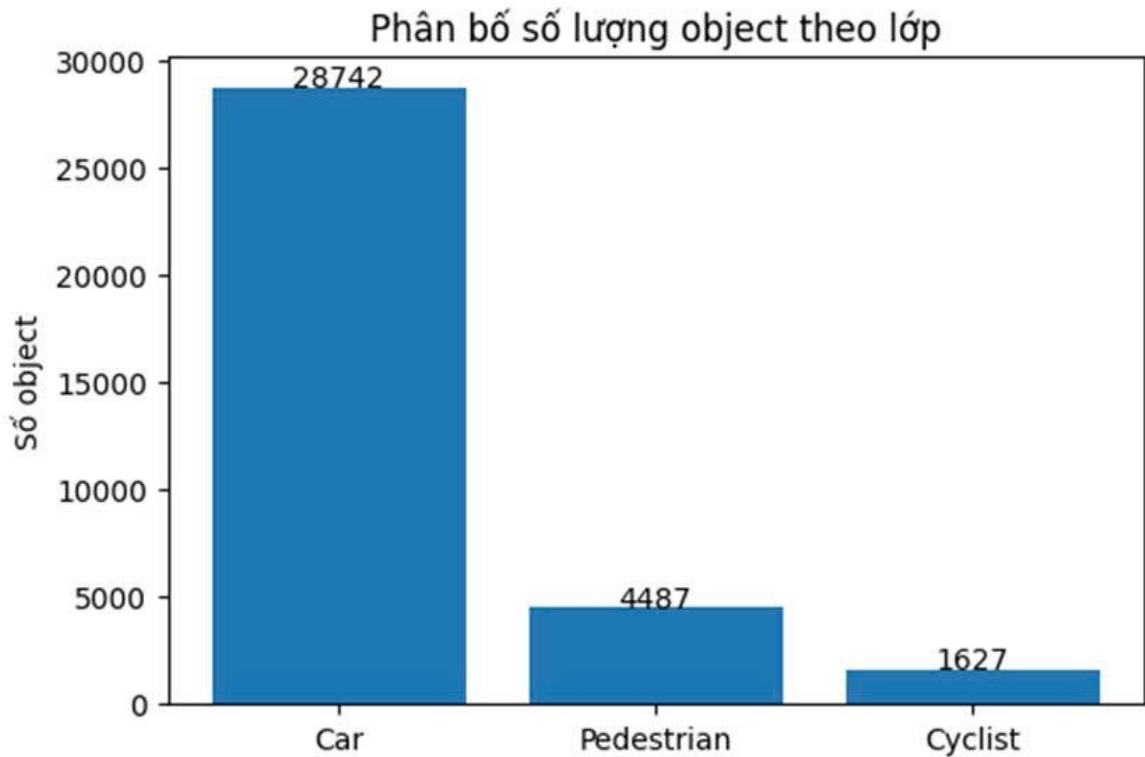
Tổng kích thước khoảng 41GB cho toàn bộ dữ liệu KITTI mà chúng tôi sẽ sử dụng để thực hiện đề tài này.

Bộ dữ liệu này có tổng cộng 8 lớp chính thức Car, Van, Truck, Pedestrian, Person_sitting, Cyclist, Tram, Don't Care, Misc



Hình 4.4. Ảnh về phân bố dữ liệu trong tập KITTI

Nhận xét chung về phân bố dữ liệu trong tập dữ liệu mà chúng tôi sử dụng đang có sự mất cân bằng giữa các lớp đặc biệt là lớp Car đang chiếm tỉ trọng rất cao trong khi đó các lớp như Person_sitting, Tram, Misc, v.v lại chiếm tỉ trọng khá thấp. Có nhiều cách để chúng ta giải quyết vấn đề mất cân bằng dữ liệu này như là loại bỏ một số lớp không chú trọng do quá nhiều dữ liệu như person_sitting, Tram, Misc và một số thứ không đáng quan tâm (don't care). Ngoài ra để thực hiện benchmark chuẩn nhất theo KITTI khuyến nghị thì chỉ nên dùng 3 lớp chính đó là Car, Cyclist, Pedestrian nên đề tài này chúng tôi sẽ sử dụng 3 lớp đó làm lớp chính cho xuyên suốt quá trình thực hiện đề tài



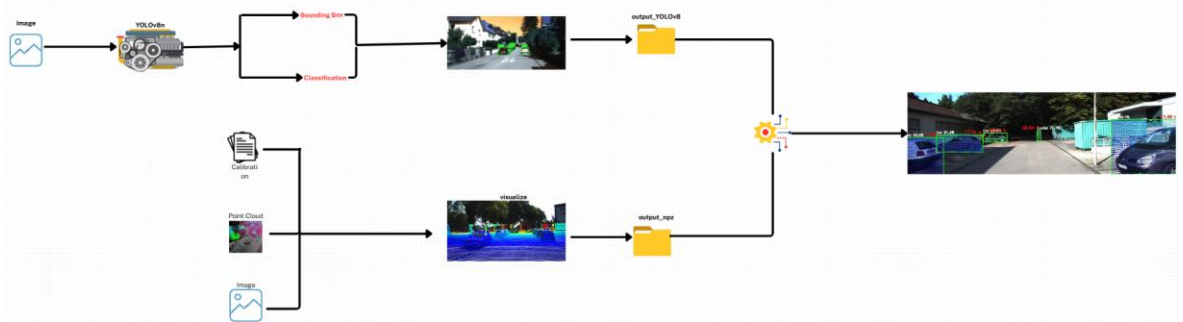
Hình 4.5. Hình ảnh về các lớp được sử dụng trong đề tài

4.2 Mô hình

4.2.1 Pipeline chính của đề tài

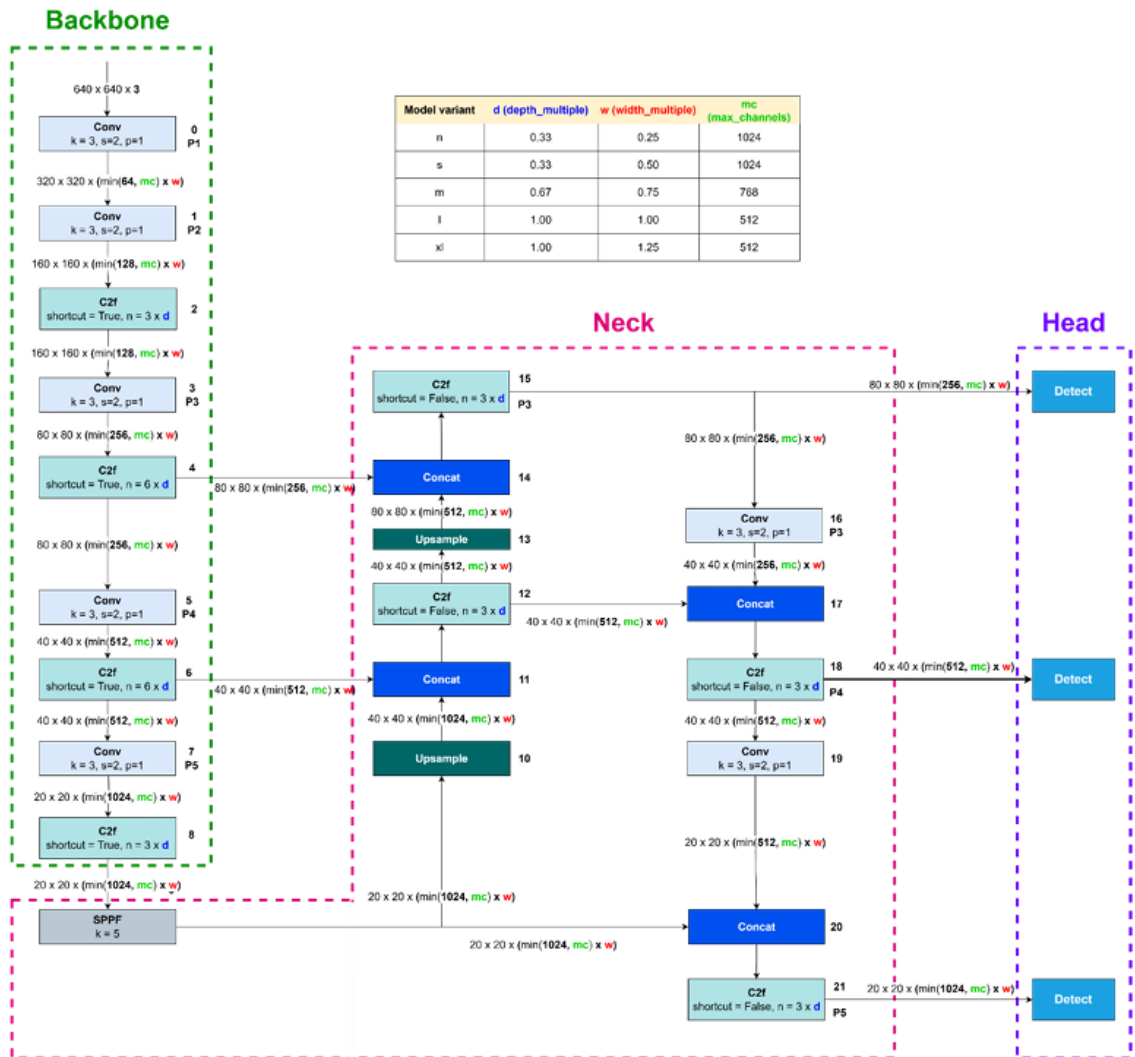
Pipeline được thiết kế để kết hợp YOLOv8 và dữ liệu Lidar (PC) nhằm phát hiện đối tượng 2D và ước tính khoảng cách từ camera đến đối tượng. Các bước thực hiện:

- Phát hiện đối tượng 2D: đầu vào là ảnh RGB được đưa vào mô hình và cho ra 2 kết quả là BBox và Classification.
- Tiền xử lý dữ liệu LiDAR: Dữ liệu PC và ma trận calib được kết hợp để chuyển các điểm 3D về tọa độ pixel.
- Kết hợp dữ liệu: kết hợp BBox từ Yolo và PC để tính khoảng cách, sau đó hiển thị lên ảnh.



Hình 4.6. Pipeline chính cho đề tài

4.2.1.1 Vẽ hình kiến trúc



Hình 4.7. Kiến trúc Yolo v8

4.2.1.2 Bảng mô tả chi tiết kiến trúc

Bảng 4.1. Backbone: trích xuất đặc trưng từ hình ảnh đầu vào.

Tầng	Tên Layer	Kiểu lớp	Kích thước đầu vào và đầu ra	Kernel/Stride/ Padding	Activation
0	Conv1 (P1)	Conv2d	$640 \times 640 \times 3 \rightarrow 320 \times 320 \times 16$	3x3/ 2/ 1	SiLU
1	Conv2 (P2)	Conv2d	$320 \times 320 \times 64 \rightarrow 160 \times 160 \times 32$	3x3/ 2/ 1	SiLU
2	C2f1	C2f	$160 \times 160 \times 32 \rightarrow 160 \times 160 \times 32$	Bottleneck ($\times 1$) + Conv (1×1 , p0)	SiLU
3	Conv3 (P3)	Conv2d	$160 \times 160 \times 32 \rightarrow 80 \times 80 \times 64$	3x3/ 2/ 1	SiLU
4	C2f2	C2f	$80 \times 80 \times 64 \rightarrow 80 \times 80 \times 64$	Bottleneck ($\times 2$)	SiLU
5	Conv4 (P4)	Conv2d	$80 \times 80 \times 64 \rightarrow 40 \times 40 \times 128$	3x3/ 2/ 1	SiLU
6	C2f3	C2f	$40 \times 40 \times 128 \rightarrow 40 \times 40 \times 128$	Bottleneck ($\times 2$)	SiLU
7	Conv5 (P5)	Conv2d	$40 \times 40 \times 128 \rightarrow 20 \times 20 \times 128$	3x3/ 2/ 1	SiLU
8	C2f4	C2f	$20 \times 20 \times 128 \rightarrow 20 \times 20 \times 256$	Bottleneck ($\times 1$, r=2.0)	SiLU
9	SPPF	SPPF	$20 \times 20 \times 256 \rightarrow 20 \times 20 \times 256$	Conv (1×1) + 3xMaxPool (5×5 , p2)	SiLU

Bảng 4.2. Neck

Tầng	Tên Layer	Kiểu lớp	Kích thước đầu vào và đầu ra	Kernel/Stride/ Padding	Activation
10	Upsample1	Upsample	$20 \times 20 \times 256 \rightarrow 40 \times 40 \times 256$	scale=2	
11	Concat1	Concat	$[40 \times 40 \times 256, 40 \times 40 \times 128 (P4)] \rightarrow 40 \times 40 \times 384$		
12	C2f5	C2f	$40 \times 40 \times 384 \rightarrow 40 \times 40 \times 384$	Bottleneck ($\times 1$) + Conv	SiLU
13	Upsample2	Upsample	$40 \times 40 \times 384 \rightarrow 80 \times 80 \times 384$	scale=2	
14	Concat2	Concat	$[80 \times 80 \times 384, 80 \times 80 \times 64 (P3)] \rightarrow 80 \times 80 \times 448$		

15	C2f6	C2f	$80 \times 80 \times 448 \rightarrow 80 \times 80 \times 448$	Bottleneck ($\times 1$) + Conv	SiLU
16	Downsample1	Conv2d	$80 \times 80 \times 448 \rightarrow 40 \times 40 \times 224$	$3 \times 3 / 2 / 1$	SiLU
17	Concat3	Concat	$[40 \times 40 \times 224, 40 \times 40 \times 384 \text{ (từ 12)}] \rightarrow 40 \times 40 \times 608$		
18	C2f7	C2f	$40 \times 40 \times 608 \rightarrow 40 \times 40 \times 608$	Bottleneck ($\times 1$) + Conv	SiLU
19	Downsample2	Conv2d	$40 \times 40 \times 608 \rightarrow 20 \times 20 \times 304$	$3 \times 3 / 2 / 1$	SiLU
20	Concat4	Concat	$[20 \times 20 \times 304, 20 \times 20 \times 256 \text{ (từ 9)}] \rightarrow 20 \times 20 \times 560$		
21	C2f8	C2f	$20 \times 20 \times 560 \rightarrow 20 \times 20 \times 560$	Bottleneck ($\times 1$) + Conv	SiLU

Bảng 4.3. Head & Detect

22	Detect P3	Detect	$80 \times 80 \times 448 \rightarrow 80 \times 80 \times (4 \text{ bbox} + 1 \text{ obj} + \text{nc cls})$	$1 \times 1 / 1 / 0$	linear $\rightarrow$ sigmoid
23	Detect P4	Detect	$40 \times 40 \times 608 \rightarrow 40 \times 40 \times (4 + 1 + \text{nc})$	$1 \times 1 / 1 / 0$	linear $\rightarrow$ sigmoid
24	Detect P5	Detect	$20 \times 20 \times 560 \rightarrow 20 \times 20 \times (4 + 1 + \text{nc})$	$1 \times 1 / 1 / 0$	linear $\rightarrow$ sigmoid

4.2.1.3 Hàm activation của tầng cuối

Nhánh bounding-box regression: không qua activation

Nhánh objectness và classification: sigmoid

4.2.1.4 Hàm loss

Bounding-box Regression loss: CIOU Loss + Distribution Focal Loss

CIOU Loss:

$$L_{\text{CIOU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad (4.1)$$

Trong đó:

- IoU: là chỉ số đo mức độ chồng lấp giữa hai vùng: hộp dự đoán và hộp ground-truth
- $\rho(b, b^{gt})$: là khoảng cách Euclid giữa tâm hộp dự đoán và tâm hộp ground-truth

- c là độ dài đường chéo của hộp nhỏ nhất bao cả hai hộp
- v : điều chỉnh tỉ lệ khung
- α là trọng số cân bằng v so với IoU: $\alpha = \frac{v}{(1-\text{IoU})+v}$

DFL:

$$L_{\text{DFL}}(p, t) = w_l \text{CE}(p, t_l) + w_r \text{CE}(p, t_r) \quad (4.2)$$

Trong đó:

- p : phân phối xác suất đầu ra
- t : tọa độ số thực
- $t_l = \lfloor t \rfloor$ (lấy phần nguyên)
- $t_r = \lceil t \rceil - 1$
- $w_l = t_r - t$
- $w_r = t - t_l$
- CE: hàm cross – entropy

Objectness & Classification loss: Binary Cross-Entropy

$$L_{\text{BCE}}(\hat{y}, y) = -[y \log \sigma(\hat{y}) + (1 - y) \log(1 - \sigma(\hat{y}))] \quad (4.3)$$

Trong đó:

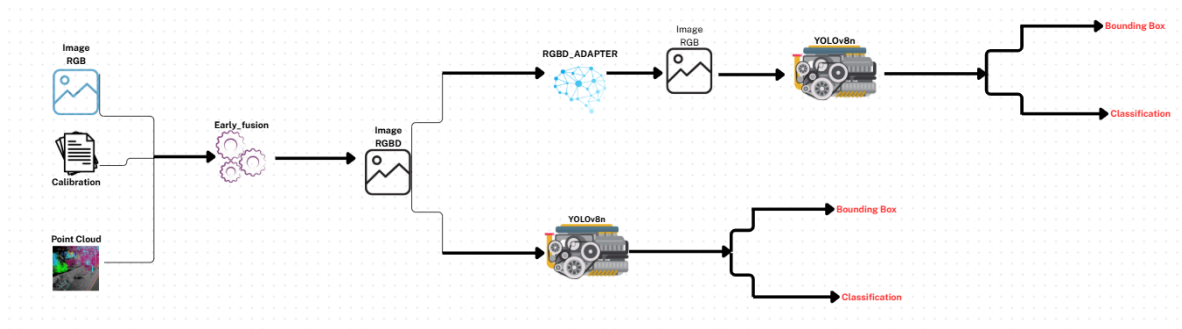
$$\sigma(\hat{y}) = 1/(1 + e^{-\hat{y}})$$

Tổng loss:

$$L = \lambda_{\text{box}} L_{\text{CIoU}} + \lambda_{\text{dfl}} L_{\text{DFL}} + \lambda_{\text{cls}} L_{\text{BCE}} + \lambda_{\text{obs}} L_{\text{BCE}} \quad (4.4)$$

Các hệ số λ được điều chỉnh trong config (mặc định box=7.5, dfl=1.5, cls=0.5, obs=0.5)

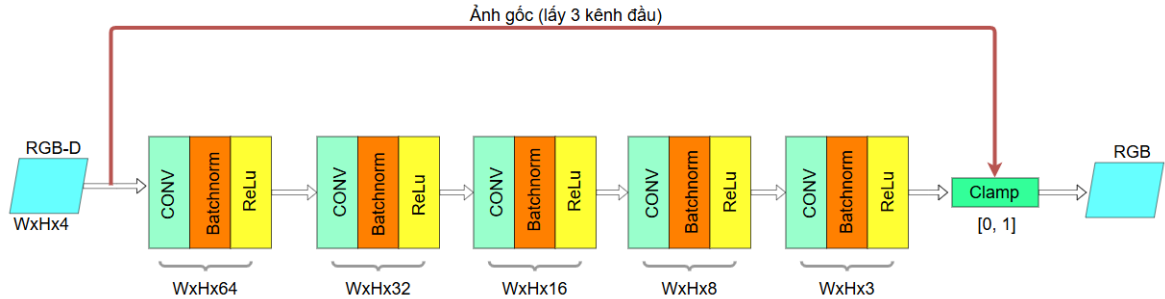
4.2.2 Pipeline hướng mở rộng của đề tài



Hình 4.8. Luồng xử lý của hướng kiểm nghiệm xem depth có thật sự mang lại thông tin hữu dụng.

Nhánh sử dụng mạng tích chập RGBD_Adapter

Mô hình kiến trúc



Hình 4. 9. Mô hình RGBD $\rightarrow$ RGB hòa trộn thêm depth rồi sinh ra ảnh gốc sao cho gần giống với ảnh thật mà mang thêm thông tin depth đã được hòa trộn bằng mạng tích chập.

Bảng mô tả chi tiết kiến trúc

Bảng 4.4. Adapter

Tầng	Tên Layer	Kiểu lớp	Kích thước đầu vào và đầu ra	Kernel/Stride/Padding	Activation
1	Conv1	Conv2d	[B, 4, H, W] $\rightarrow$ [B, 64, H, W]	3x3/ 1/ 1	Relu
	Bn1	Batchnorm2d	[B, 64, H, W]		
2	Conv2	Conv2d	[B, 64, H, W] $\rightarrow$ [B, 32, H, W]	3x3/ 1/ 1	Relu
	Bn2		[B, 32, H, W]		
3	Conv3	Conv2d	[B, 32, H, W] $\rightarrow$ [B, 16, H, W]	3x3/ 1/ 1	Relu
	Bn3	Batchnorm2d	[B, 16, H, W]		
4	Conv4	Conv2d	[B, 16, H, W] $\rightarrow$	3x3/ 1/ 1	Relu

			[B, 8, H, W]		
	Bn4	Batchnorm2d	[B, 8, H, W]		
5	Conv5	Conv2d	[B, 8, H, W] -> [B, 3, H, W]	3x3/ 1/ 1	Relu
	Bn5	Batchnorm2d	[B, 3, H, W]		
	Skip+out	+RGB skip	[B, 3, H, W] + [B, 3, H, W]		Clamp (0,1)

Hàm activation của tầng cuối

Ở đây chúng tôi không sử dụng các hàm kích hoạt tuyến tính như ReLU hay Tanh vì đầu ra ở mô hình này của chúng tôi không phải là đặc trưng (feature) mà là một bức ảnh RGB được tái cấu trúc lại khi kết hợp thông tin hỗ trợ của depth và ảnh RGB gốc nhằm từ đó giúp mô hình YOLOv8n nhận diện tốt hơn đối với các vật thể nhỏ hơn. Mà thay vào đó chúng tôi sử dụng Clamp (out ,0, 1) nhằm ép cứng giá trị pixel về đúng giá trị hợp lệ cho ảnh đã chuẩn hóa. Vì nếu dùng ReLU sẽ gây ra bất xứng và lệch độ sáng vì không có dưới hạn trên. Còn nếu dùng Tanh sẽ gây ra lệch màu, do đầu ra là $[-1, 1]$ chứ không phải $[0, 1]$.

Hàm loss, hàm cost

Đối với mô hình kiến trúc RGBD_Adapter cũng như là sinh ảnh sao cho hỗ trợ thông tin về depth cũng như không làm biến dạng, mất đi đặc trưng quan trọng của ảnh thì chúng tôi sử dụng tổng của hai hàm loss như sau để tiến hành train mô hình nhằm sinh ảnh tốt hơn thay vì làm nhiễu hoặc khiến đặc trưng bị mất mát:

SSIM: Mục đích là nhằm giữ cấu trúc giữa ảnh gốc và ảnh được sinh phải nằm trong khoảng sai lệch ít.

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (4.5)$$

Trong đó:

- μ_x, μ_y : trung bình cục bộ của ảnh x và y
- σ_x^2, σ_y^2 : là phương sai cục bộ
- σ_{xy} : hiệp phương sai giữa x và y
- C_1, C_2 : các hằng số nhỏ để tránh chia cho 0

Loss_L1: Nhằm tính toán mức sai lệch tuyệt đối giữa từng pixel.

$$\mathcal{L}_{L1}(x, y) = \frac{1}{N} \sum_{i=1}^N |x_i - y_i| \quad (4.6)$$

Trong đó:

- x: ảnh đầu vào
- y: ảnh mục tiêu
- N: tổng số pixel
- $|x_i - y_i|$: độ lệch tuyệt đối giữa từng pixel của đầu ra và gốc

Tổng Loss

$$Loss = \alpha * loss_F1 + \beta * loss_{SSIM} \quad (4.7)$$

Trong đó:

- α và β : Là hệ số điều chỉnh mức độ ưu tiên giữa tính chính xác chi tiết và tính toàn vẹn cấu trúc.

Mục tiêu của loss function tổng hợp này là đảm bảo mô hình kết hợp thông tin RGB và Depth sinh ra ảnh được phục dựng mà không mất đi đặc trưng hình ảnh ban đầu. Việc kết hợp Loss_L1 và SSIM cho phép mô hình vừa tái hiện tốt hình ảnh ban đầu vừa tận dụng thông tin từ depth để sinh ra RGB giàu đặc trưng hơn, giúp cho việc sau này nhận diện vật thể, đối tượng tốt hơn.

4.2.2.1 Nhánh sử dụng YOLOv8n thay đổi đầu vào

Trong nhánh này mô hình YOLOv8n được sử dụng được chúng tôi giữ nguyên kiến trúc backbone YOLOv8n, không thay đổi các module đặc trưng như C2F, SPPF, hay phần head dự đoán. Điểm thay đổi duy nhất đó chính là mở rộng số kênh đầu vào từ 3 (RGB) thành 4 (RGBD) nhằm đưa thêm thông tin Depth vào ảnh.

Cụ thể hơn, chúng tôi thực hiện việc ép ảnh RGB và ảnh Depth thành một tensor có shape (B, 4, H, W), sau đó cập nhật lớp đầu vào đầu tiên của backbone (Conv đầu) để chấp nhận 4 kênh thay vì 3

Toàn bộ trọng số ban đầu của YOLOv8n (được huấn luyện trước trên COCO với ảnh RGB) vẫn giữ được nguyên vẹn, chỉ khởi tạo lại trọng số lớp đầu vào.

4.3 Thực nghiệm chính

4.3.1 Thực nghiệm 1

4.3.1.1 Đánh giá Yolov8n (COCO-pretrained) trên bộ dữ liệu KITTI

Mục tiêu: Kiểm tra khả năng detection của mô hình Yolov8n đã được huấn luyện sẵn trên COCO.

Về dữ liệu chúng tôi dùng tập ảnh test của KITTI gồm 7 481 ảnh. Chỉ xét trên 3 lớp là: Car, Pedestrian và Cyclist.

Kết quả:

Bảng 4.5. Bảng so sánh mAP của các lớp

Benchmark	Easy	Moderate	Hard
Car	76.45%	56.05%	43.53%
Pedestrian	55.83%	41.60%	37.69%
Cyclist	0.42%	0.24%	0.27%

Nhận xét:

- Car: kết quả tốt trên mức Easy (~76 %) nhưng giảm dần ở Moderate và Hard.
- Pedestrian: ở mức trung bình (~56 % Easy)
- Cyclist: ở mức dưới 1% ở cả 3 cho thấy mô hình hoàn toàn không nhận diện được Cyclist trong KITTI. Vì trong COCO chỉ có “bicycle”, không có “Cyclist”.

4.3.1.2 Kết hợp dữ liệu và tính khoảng cách

Mục tiêu:

- Kết hợp dữ liệu LiDAR và ảnh KITTI để dự đoán Bbox của các đối tượng Car, Pedestrian và Cyclist trên ảnh bằng YOLOv8n.
- Thực hiện chiếu PC lên ảnh để tính được **khoảng cách từ camera đến đối tượng**.
- Trực quan hóa mối quan hệ giữa khoảng cách của các object trong LiDAR và vị trí trên ảnh 2D

Dữ liệu:

- Ảnh RGB (PNG)
- LiDAR (.bin)
- File hiệu chỉnh calibration từ KITTI (.txt)

Quy trình thực hiện:

- **Bước 1:** Dựng PC LiDAR trong hệ camera và chiếu lên ảnh để lấy tọa độ pixel.
 - Đọc calibration KITTI và lấy ba ma trận quan trọng của KITTI:
 - P2 (3x4)

- $R0_rect$ (3x3)
- $Tr_velo_to_cam$ (3x4)
- Chuyển LiDAR sang hệ tọa độ camera (camera-frame)
 - Mỗi điểm LiDAR ban đầu ở hệ tọa độ Velodyne là (X, Y, Z) .
 - Tạo tọa độ đồng nhất homogeneous để chuyển thành vector 4D: $(X, Y, Z, 1)$.
 - Áp dụng ma trận $Tr_velo_to_cam$ để đổi về camera-frame, sau đó dùng $R0_rect$ để căn chỉnh.
 - Kết quả là tập điểm 3D $(X_{cam}, Y_{cam}, Z_{cam})$ trong hệ tọa độ của camera.
- Chiếu các điểm camera-frame lên ảnh (pixel coordinates)
 - Chuẩn bị homogeneous lần nữa: $(X_{cam}, Y_{cam}, Z_{cam}, 1)$.
 - Dùng ma trận $P2$ để chiếu 3D $(X_{cam}, Y_{cam}, Z_{cam}, 1)$ xuống mặt phẳng ảnh và thu được (x', y', z') .
 - Lọc chỉ giữ $Z_{cam} > 0$ tức là chỉ lấy các điểm nằm phía trước camera.
 - Sau đó thực hiện chia cho z' để có tọa độ pixel:

$$u = \frac{x'}{z'}, \quad v = \frac{y'}{z'}$$
- Kết quả thu được là:
 - Tọa độ (u, v) trên ảnh pixel – dùng để vẽ PC.
 - Tọa độ $(X_{cam}, Y_{cam}, Z_{cam})$ – dùng để tính khoảng cách.
- **Bước 2:** Phát hiện đối tượng 2D trên ảnh bằng **YOLOv8n**
- **Bước 3:** Với mỗi BBox được phát hiện ở trên thực hiện:
 - Lọc chỉ lấy các điểm PC nằm trong BBox
 - Dùng LinearSegmentedColormap từ Matplotlib (Xanh lá → Vàng → Cam) để tô màu các điểm PC: điểm gần camera màu xanh, điểm xa màu đỏ.
 - Tính 4 kiểu khoảng cách (Min, Max, Median, Mean) dựa trên các điểm đã lọc. Khoảng cách của mỗi điểm được tính là khoảng cách Euclid từ

tâm camera đến điểm đó trong không gian 3D sau khi đã chuyển từ LiDAR sang camera-frame.

$$d = \sqrt{X_{cam}^2 + Y_{cam}^2 + Z_{cam}^2} \quad (4.8)$$

- **Bước 4:** Trực quan hóa kết quả. Hình ảnh được Object detection bằng YOLOv8n với tên class ở góc trên bên trái và giá trị distance ở góc trên bên phải của BBox.

Kết quả và nhận xét:

- Min



Hình 4. 10. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Min

- Khoảng cách là giá trị **nhỏ nhất** của các điểm LiDAR trong mỗi BBox
- Có thể bị nhiễu bởi các điểm như mặt đường

- Max



Hình 4.11. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Max

- Khoảng cách là giá trị **lớn nhất** của các điểm LiDAR trong mỗi BBox
- Không chính xác vì dễ lấy các điểm background như cây hoặc nhà ở phía sau

- Median



Hình 4.12. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Median

- Khoảng cách là giá trị **trung vị** của tất cả các depths
- Không bị ảnh hưởng bởi outlier quá lớn hoặc quá nhỏ

- Mean



Hình 4. 13. Hình ảnh về thực nghiệm 1 khoảng cách sau khi dùng Mean

- Khoảng cách là giá trị **trung bình** của tất cả các depth
- Nếu trong BBox có cả điểm rất xa hoặc rất gần thì Mean sẽ bị ảnh hưởng

4.3.2 Thực nghiệm 2

4.3.2.1 Fine-tune YOLOv8n trên KITTI

Mục tiêu: Cải thiện Detection của YOLOv8n trên cả 3 lớp bằng cách fine-tune trên tập huấn luyện của KITTI. Đặc biệt là kỳ vọng cải thiện cho 2 lớp Pedestrian và Cyclist, vốn rất thấp khi chạy trên mô hình COCO-pretrained.

Dữ liệu:

- 7481 ảnh từ tập huấn luyện của KITTI và nhãn

- Chuyển sang YOLO format
- Chia train và val với tỉ lệ 90/10

Huấn luyện:

- Load mô hình COCO-pretrained
- Thực hiện Fine-tune với các siêu tham số:
 - Epoch: 50
 - Patience: 3
 - Mixup: 0.1

Kết quả:

Bảng 4.6. Bảng so sánh mAP của các lớp

Benchmark	Easy	Moderate	Hard
Car	93.37%	84.93%	75.03%
Pedestrian	62.83%	46.37%	41.67%
Cyclist	49.42%	32.43%	28.80%

Nhận xét: Cải thiện nhiều hơn so với Yolov8n COCO

- Car: từ ~76% tăng lên 93% (Easy), Moderate và Hard cũng tăng đáng kể
- Pedestrian: từ ~56 % → ~63 % (Easy), Moderate và Hard tăng nhẹ
- Cyclist: tăng mạnh từ ~0.4 % → ~49 % (Easy). Mô hình đã có thể nhận diện được lớp Cyclist.

4.3.2.2 Kết hợp dữ liệu và tính khoảng cách

Tương tự như Thực nghiệm 1 nhưng thay đổi ở bước 2: Phát hiện đối tượng 2D trên ảnh bằng **YOLOv8n-Fine-Tuning**

Kết quả:

- Min



Hình 4.14. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Min

- Max



Hình 4.15. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Max

- Median



Hình 4.16. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Median

- Mean



Hình 4.17. Hình ảnh về thực nghiệm 2 khoảng cách sau khi dùng Mean

Nhận xét: Khoảng cách distance có thay đổi đôi chút nhưng nhìn chung không quá khác biệt so với Thực nghiệm 1.

4.3.2.3 Thực nghiệm 3

Ở phần này ta vẫn sử dụng lại **YOLOv8n-Fine-Tuning** từ Thực nghiệm 2.

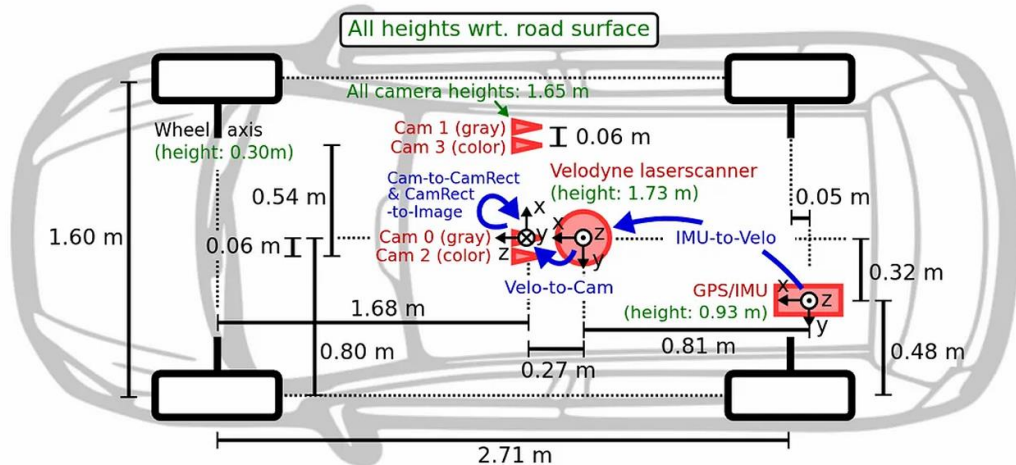
4.3.2.4 Kết hợp dữ liệu và tính khoảng cách

Mục tiêu:

- Thay vì sử dụng toàn bộ PC gốc ~120 nghìn điểm chúng ta sẽ áp dụng Voxel Centroid Downsampling để giảm số lượng điểm cần tính từ đó tăng tốc độ tính toán và giảm tải bộ nhớ.
- Cải thiện độ chính xác cho Min bằng cách lọc các phần PC dưới mặt đường đi từ đó giúp tập trung vào vật thể nhiều hơn.
- Bổ sung Highlight điểm gần nhất với giá trị (Min, Max, Median hoặc Mean) để biết được đâu là điểm được chọn để tính khoảng cách. Chúng ta sẽ biết được liệu khoảng cách đó có thật sự đúng hay không và điểm đó có nằm trên đối tượng mà ta cần tính không.

Quy trình thực hiện: (Bổ sung)

- **Bước 1:** Dựng PC LiDAR trong hệ camera và chiếu lên ảnh để lấy tọa độ pixel.
 - Bổ sung Giảm mẫu Point Cloud (**Voxel Centroid Downsampling**) trước khi
 - Chia không gian 3D thành lưới ô vuông (voxel) đều nhau có kích thước 0.1.
 - Mỗi voxel chứa một tập con các điểm gốc.
 - Với mỗi ô voxel, tính tọa độ trung bình của tất cả điểm chứa trong ô đó và làm điểm đại diện
 - Tiếp tục thực hiện Đọc calibration KITTI và các phần còn lại tương tự Thực nghiệm 1
 - Sau khi thu được tọa độ (u, v) và Tọa độ (X_{cam} , Y_{cam} , Z_{cam}) ta sẽ lọc PC theo trục Y, hãy xem ảnh dưới đây về cách thiết lập cảm biến cho tập dữ liệu KITTI để hiểu rõ:



Hình 4.18. Hình ảnh về cấu trúc của xe được KITTI sử dụng để thu thập dữ liệu

Nguồn : [understanding-kitti-dataset-part-1-hands-on-projection-matrices](https://www.cvlibs.net/datasets/kitti/eval_sequences.php)

- Phần LiDAR chính là **Velodyne laserscanner** với tọa độ Y hướng sang bên trái xe
 - Sau khi thực hiện Chuyển LiDAR sang hệ tọa độ camera thì tọa độ hiện tại của chúng ta là $(X_{cam}, Y_{cam}, Z_{cam})$ với trục Y hướng từ trên xuống
 - LiDAR đang được đặt ở trên nóc xe với chiều cao là 1.73m. vậy độ quét của LiDAR sẽ nằm trong khoảng từ 0 đến 1.73 với 0 là min (do Y hiện tại đang chiều từ trên xuống nên điểm 0 sẽ nằm ở trên) và 1.73 là max (nghĩa là phần mặt đường)
 - Chúng tôi quyết định chọn `ground_thresh = 0.9` để loại bỏ các điểm thấp hơn mức này. Nghĩa là sẽ không lấy các phần LiDAR trong khoảng 0.8m từ mặt đường trở lên, như vậy các điểm PC sẽ chỉ nằm từ phần giữa xe đến nóc xe.
- **Bước 4:** Bổ sung Highlight điểm gần nhất

Kết quả:

- Min:



Hình 4.19. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Min

- Max:



Hình 4.20. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Max

- Median



Hình 4.21. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Median

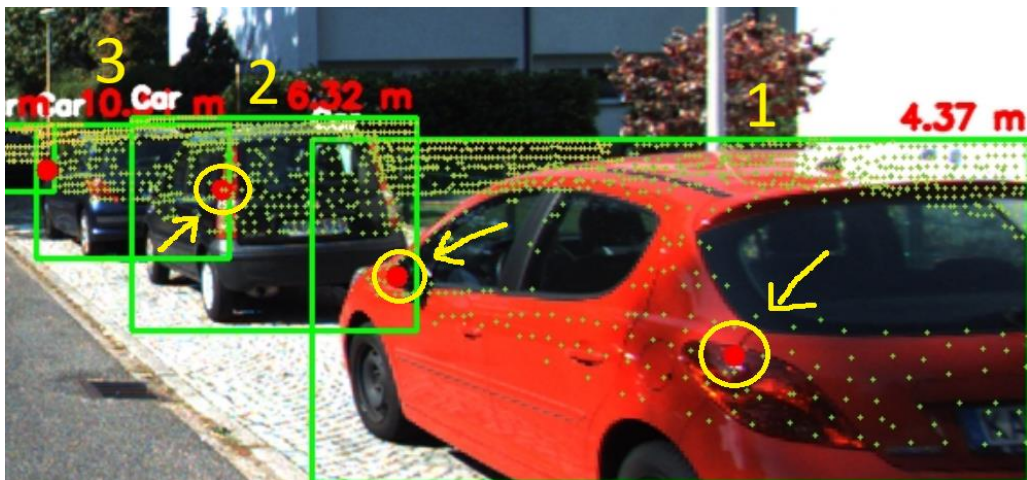
- Mean:



Hình 4.22. Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Mean

Nhận xét:

- Trước đó thì Median là phương pháp tốt nhất để tính khoảng cách vì không bị ảnh hưởng bởi nhiễu.
- Với Min trước đó thường bị ảnh hưởng chủ yếu do mặt đường, nhưng sau khi áp dụng lọc ground_thresh thì Min đang là phương pháp tối ưu nhất.
- Tuy nhiên vẫn còn một số hạn chế nhất định. Cụ thể do việc Object detection 2D nên các BBox sẽ bị đè lên nhau. Ví dụ:



Hình 4.23. Nhận xét

Có thể thấy ở xe số 1 Min vẫn cho kết quả chính xác. Nhưng ở xe số 2 do BBox chồng một phần qua xe số 1 nên bị nhiễu và lấy điểm ở phần gương của xe số 1. Tương tự ở xe số 3 cũng lấy nhiễu phần đuôi của xe số 2.

4.3.3 Thực nghiệm 4

Từ thực nghiệm này về sau chúng tôi sử dụng trọng số đã được train trên YOLOv8n thông qua dữ liệu được sinh ra từ Adapter từ thực nghiệm phụ tại chỉ mục 4.4.1 nhằm tối ưu hóa việc xác định đối tượng.

4.3.3.1 Kết hợp dữ liệu và tính khoảng cách

Mục tiêu: Để khắc phục vấn đề nhầm lẫn trong việc tính khoảng cách.

Quy trình thực hiện:(Điều chỉnh và bổ sung)

- **Bước 1:** Không cần lọc PC theo trục Y nữa.
- **Bước 3:** Với mỗi BBox được phát hiện ở trên thực hiện:
 - Chúng tôi sẽ không “Lọc và chỉ lấy các điểm PC nằm trong BBox “mà thay vào đó sẽ sử dụng mô hình Segment của YOLOv8 (yolov8n-seg.pt) để tìm mask tương ứng (có IoU cao nhất). Sau đó dùng mask này để lọc các điểm PC. Nếu không tìm được mask tương ứng thì mới lọc theo BBox.
 - Các phần còn lại thực hiện tương tự Thực nghiệm 1.

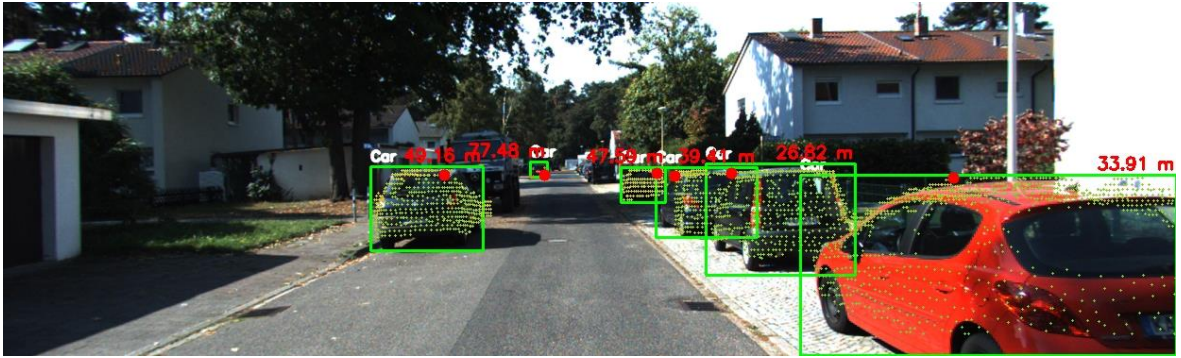
Kết quả:

- Min



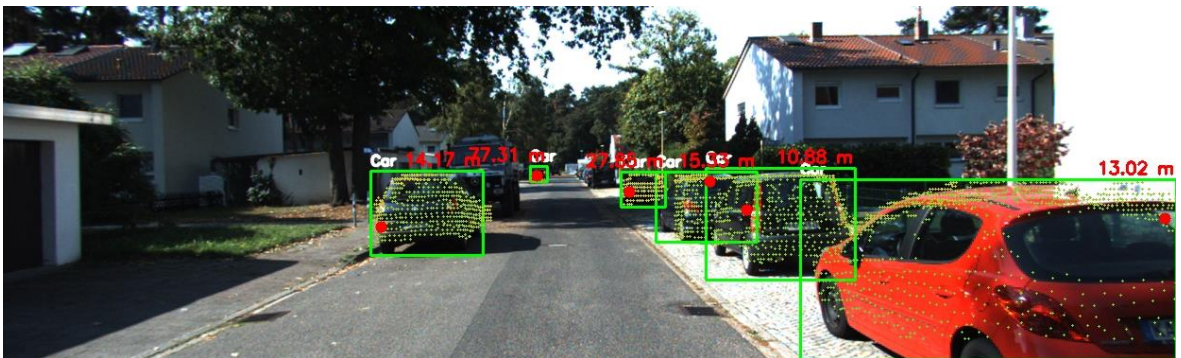
Hình 4. 24 Hình ảnh về thực nghiệm 4 khoảng cách sau khi dùng Min

- Max



Hình 4. 25 Hình ảnh về thực nghiệm 4 khoảng cách sau khi dùng Max

- Median

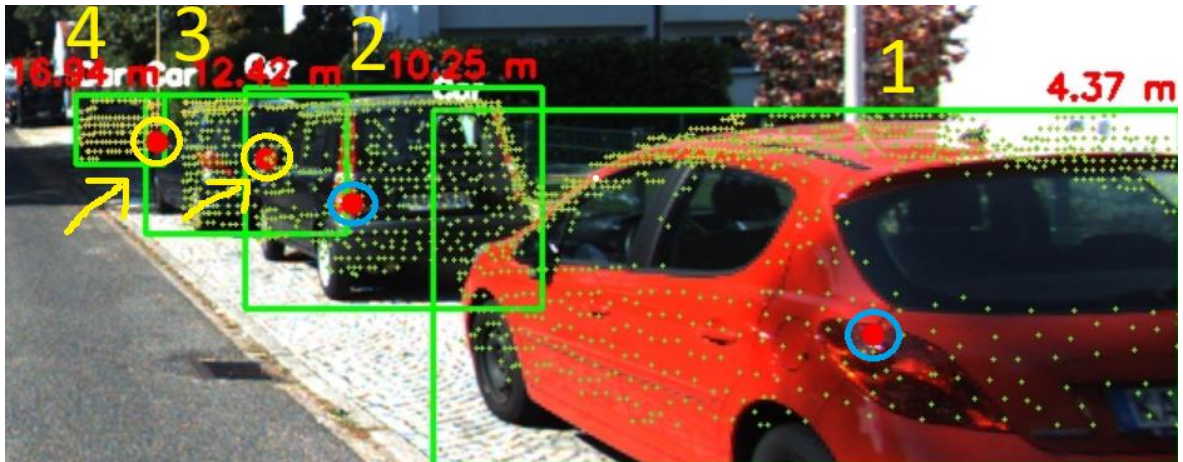


Hình 4. 26 Hình ảnh về thực nghiệm 4 khoảng cách sau khi dùng Median

- Mean



Hình 4. 27 Hình ảnh về thực nghiệm 3 khoảng cách sau khi dùng Mean



Hình 4. 28 Nhận xét

Nhận xét: Với kết quả ở Min có thể thấy xe số 2 không còn bị nhầm lẫn và lấy nhầm và lấy điểm ở phần gương của xe số 1 nữa. Ở xe số 3 nó không còn lấy nhầm vào phần đuôi của xe số 2 nữa, Tuy nhiên có vẻ lấy nhầm phần gương của xe số 2 có thể do vật thể nằm ở xa nên segment chưa giải quyết được hoàn toàn.

Và việc sử dụng segment cũng đã giúp các điểm PC không còn lấy vào các phần background nữa.

4.3.4 Thực nghiệm 5

4.3.4.1 Kết hợp dữ liệu và tính khoảng cách

Mục tiêu:

- Khắc phục triệt để vấn đề nhầm lẫn trong việc tính khoảng cách
- Nâng cấp để dự đoán trên video

Dữ liệu: chúng ta sẽ dùng tập dữ liệu gồm nhiều frame ảnh được kitti cung cấp ở [The KITTI Vision Benchmark Suite](http://www.kitti.vision.uni-luebeck.de/)

Data Category: City

Before browsing, please wait some moments until this page is fully loaded.



2011_09_26_drive_0001 (0.4 GB)

Length: 114 frames (00:11 minutes)

Image resolution: 1392 x 512 pixels

Labels: 12 Cars, 0 Vans, 0 Trucks, 0 Pedestrians, 0 Sitters, 2 Cyclists, 1 Trams, 0 Misc

Downloads: [\[unsynced+unrectified data\]](#) [\[synced+rectified data\]](#) [\[calibration\]](#) [\[tracklets\]](#)

Hình 4. 29 Hình ảnh về video để thực hiện liên tục khoảng cách đối tượng

Nguồn: [The KITTI Vision Benchmark Suite](http://www.kitti.vision.uni-luebeck.de/)

Ta sẽ dùng **2011_09_26_drive_0001 (0.4 GB)** để thử nghiệm, Chỉ cần sử dụng 2 phần là [\[unsynced+unrectified data\]](#) và [\[calibration\]](#)

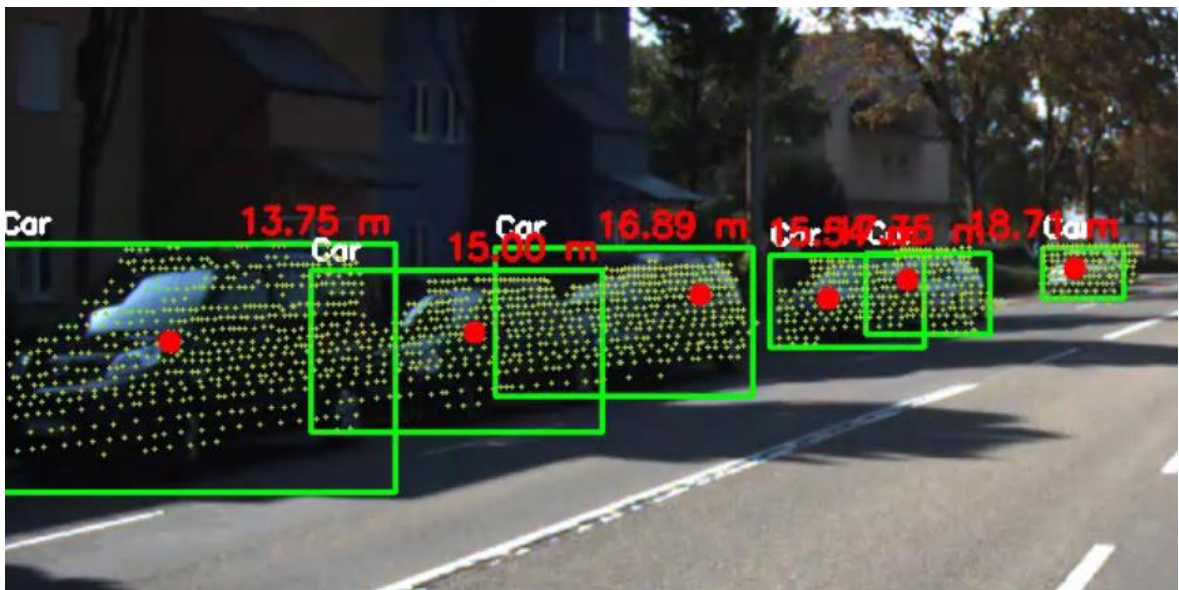
- unsynced+unrectified data: có tổng cộng 6 folder và ta chỉ sử dụng các folder sau:
 - image_02:
 - data: chứa ảnh RGB (.PNG)
 - timestamps.txt: để đồng bộ với LiDAR
 - velodyne_points:
 - data: chứa dữ liệu PC (.txt)
 - timestamps.txt
- calibration:
 - calib_cam_to_cam.txt: chỉ trích xuất R_rect_02 (R0) và P_rect_02 (P2) để chiếu PC lên ảnh.
 - calib_velo_to_cam.txt: chỉ trích xuất R và T để ghép thành ma trận Tr

Quy trình thực hiện:

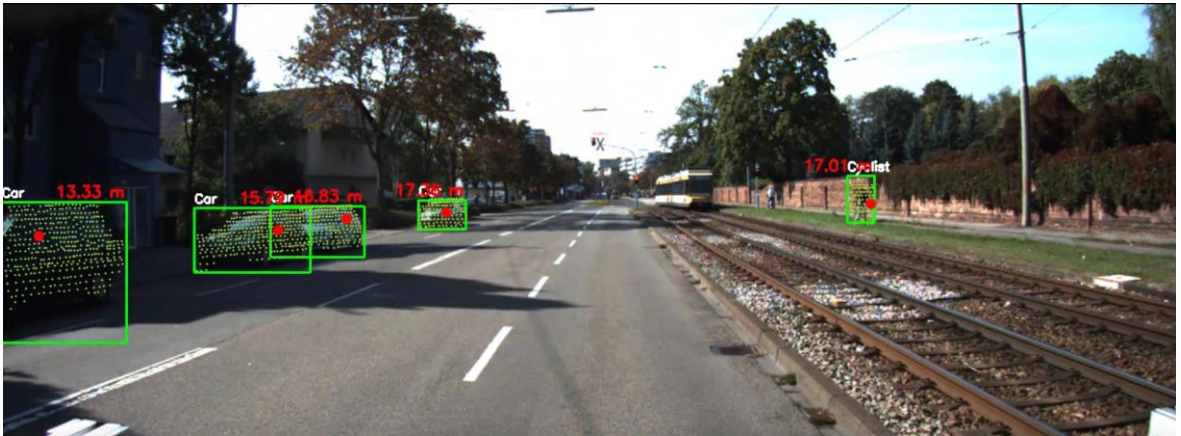
- Bước 1: Đọc file timestamps.txt
 - Mỗi dòng có định dạng YYYY-MM-DD HH: MM: SS. ssssss. Ví dụ: 2011-09-26 13:02:25.446243840
 - Ta sẽ tách phần ở trước và sau dấu chấm để chuyển thành datetime sau đó cộng thêm phần thập phân.
 - Trả về 1 mảng float64 với độ dài bằng số file ảnh, mỗi phần tử chứa thời gian UNIX (giây có phần thập phân).
 - Thu được **ts_img** cho ảnh và **ts_velo** cho LiDAR.
- Bước 2: Đồng bộ ảnh và LiDAR:
 - Tính chênh lệch thời gian giữa ảnh **i** và **tất cả** LiDAR scans.
 - Chọn scan có chênh lệch thấp nhất **j**.
 - Ta sẽ có kết quả là mỗi ảnh sẽ luôn gắn đúng scan LiDAR tương ứng (cùng thời điểm).
- Bước 3: đọc file calibration để lấy:
 - **R_rect_02 (R0)** và **P_rect_02 (P2)** từ calib_cam_to_cam.txt
 - **R** và **T** từ calib_velo_to_cam.txt sau đó ghép thành Tr(3x4)

- Bước 4: Thực hiện “Dựng PC LiDAR trong hệ camera và chiếu lên ảnh để lấy tọa độ pixel” tương tự *Bước 1 ở Thực nghiệm 3* nhưng không thực hiện “lọc PC theo trục Y”
- Bước 5: Phát hiện và xử lý Segment cho 3 class Car, Pedestrian và Cyclist.
 - Với mỗi BBox được phát hiện, tìm mask tương ứng (có IoU cao nhất). Sau đó dùng mask này để lọc các điểm PC. Nếu không tìm được mask tương ứng thì mới lọc theo BBox.
 - Trên các tập điểm sau khi lọc, thực hiện:
 - Tính centroid 3D để tìm điểm tâm của vật thể
 - Sau khi có điểm tâm ta sẽ thực hiện phương pháp Radius Nearest Neighbors để lấy các điểm nằm trong bán kính tùy chỉnh.
 - Tính khoảng cách dựa trên kết quả Radius Nearest Neighbors.
 - (Kết quả Radius Nearest Neighbors sẽ chỉ sử dụng để tính khoảng cách, phần hiển thị các điểm PC sẽ vẫn dùng các điểm sau khi lọc từ mask).
- Bước 6: Ghép frame thành video
 - Sau khi hoàn thành tính toán cho mỗi frame thì ta sẽ thực hiện ghép lại để được 1 video hoàn chỉnh với fps (số khung hình/ giây) tùy chỉnh.
- Bước 6: Trực quan hóa kết quả.

Kết quả:



Hình 4. 30 Hình ảnh thực nghiệm 5



Hình 4. 31 Hình ảnh thực nghiệm 5

Nhận xét: Có thể thấy việc áp dụng Segment và RNN (Radius Nearest Neighbors) đã giải quyết được vấn đề nhầm lẫn trong việc tính khoảng cách,

4.4 Thực nghiệm phụ

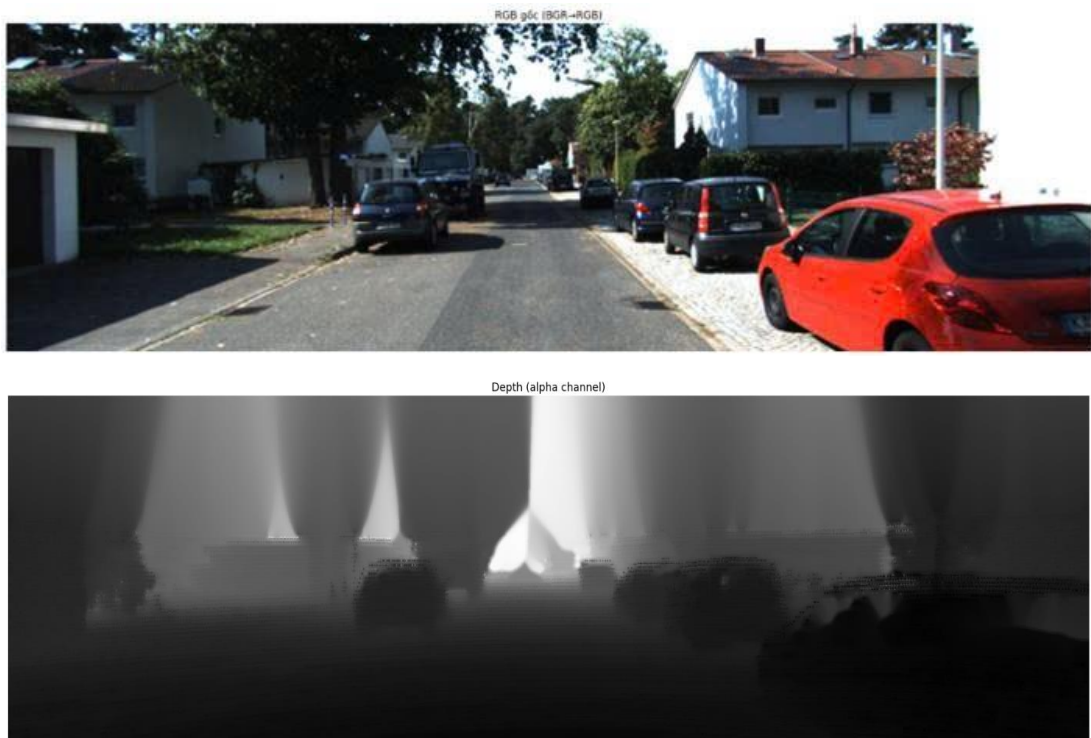
Các tham số chung:

- Epochs: 50
- patience: 3
- batch: 16
- imgsz: 640
- mixup: 0.1
- mosaic: 1.0

Dữ liệu chung:

- Ảnh RGB (PNG) từ tập train
- Label (dùng Label Yolo Format ở thực nghiệm 2)
- Tạo ảnh RGBD:
 - Thực hiện xử lý PC tương tự bước 1 ở thực nghiệm 1 và thu được kết quả là tọa độ (u, v) và Tọa độ (X_{cam} , Y_{cam} , Z_{cam}) cho toàn bộ tập train. Lưu ý với (X_{cam} , Y_{cam} , Z_{cam}) là tọa độ Point Cloud đã chuyển sang hệ tọa độ của camera rồi.
 - Tạo depth map 2D:
 - Tính khoảng cách Euclid (d_i) của mỗi điểm so với camera.

- Khởi tạo ma trận chiều sâu depth kích thước $H \times W$ với giá trị ban đầu $= +\infty$.
- Với mỗi điểm, nếu tọa độ pixel nằm trong ảnh, gán khoảng cách đi vào pixel đó nếu nhỏ hơn giá trị hiện tại.
- Với các pixel không có điểm nào gán (vẫn $= +\infty$ thì sẽ gán thành 0)
- Cắt chỉ lấy các trị depth tối đa là 100m. Sau đó scale về dạng 16-bit.
- Inpainting: Chuyển từ 16-bit về 8-bit. Sau đó sử dụng thuật toán **INPAINT_TELEA** để nội suy thông tin depth từ vùng xung quanh (vì `cv2.inpaint` chỉ hoạt động với ảnh 8-bit hoặc 32-bit). Cuối cùng chuyển lại về dạng 16-bit.
- Stack kênh depth vào ảnh RGB thành ảnh 4 kênh RGBD.
- Kết quả: (chúng ta cần tách riêng 2 kênh RGB và D để hiển thị).



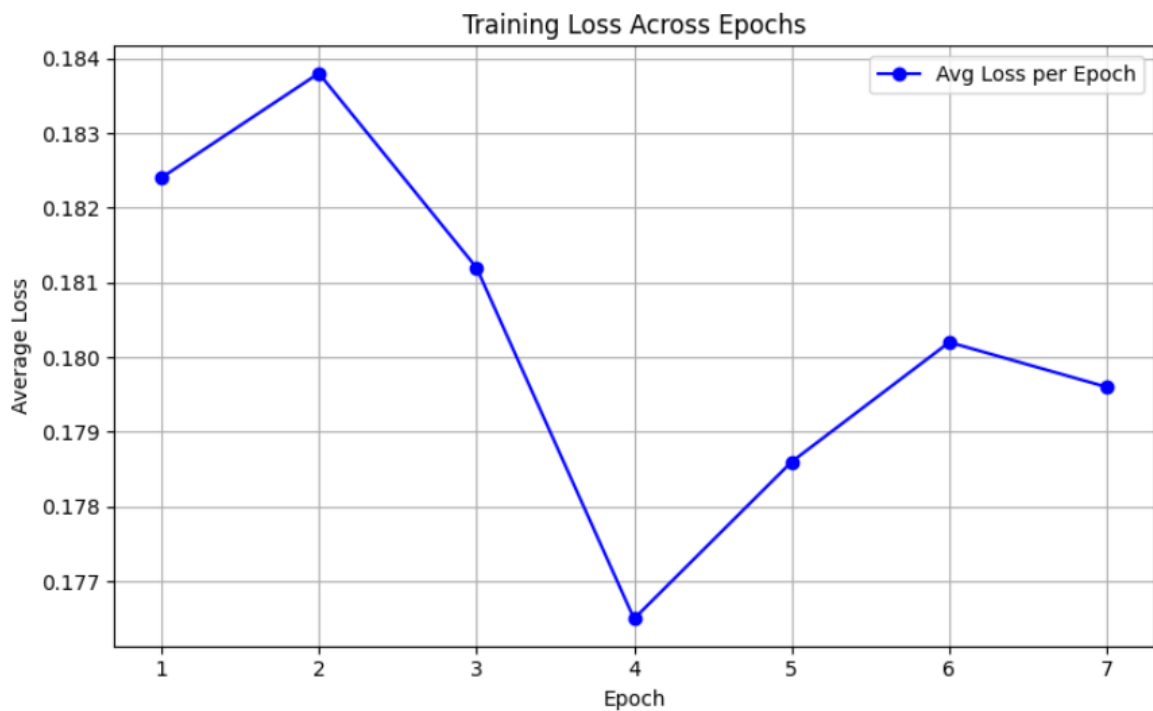
Hình 4.32. Ảnh Depth sau khi nội suy và tách từ kênh RGBD

- Sau khi hoàn tất quá trình tạo ảnh RGBD cho toàn bộ tập train ta sẽ chia thành train và val với tỉ lệ 90/10

4.4.1 Nhánh sử dụng tích chập để sinh ảnh có chứa thông tin depth

Mục tiêu: Là một ảnh RGBD sẽ được đi qua một mạng tích chập (Adapter_RGBD) sẽ sinh ra một ảnh (RGB) mang thông tin của depth từ đó sẽ giúp ích cho việc phát hiện đối tượng tốt hơn từ đó làm tăng tỉ lệ chính xác cho mô hình.

Quá trình huấn luyện Adapter:



Hình 4.33. Hình ảnh hàm loss của Adapter

Dữ liệu mới được sinh ra:



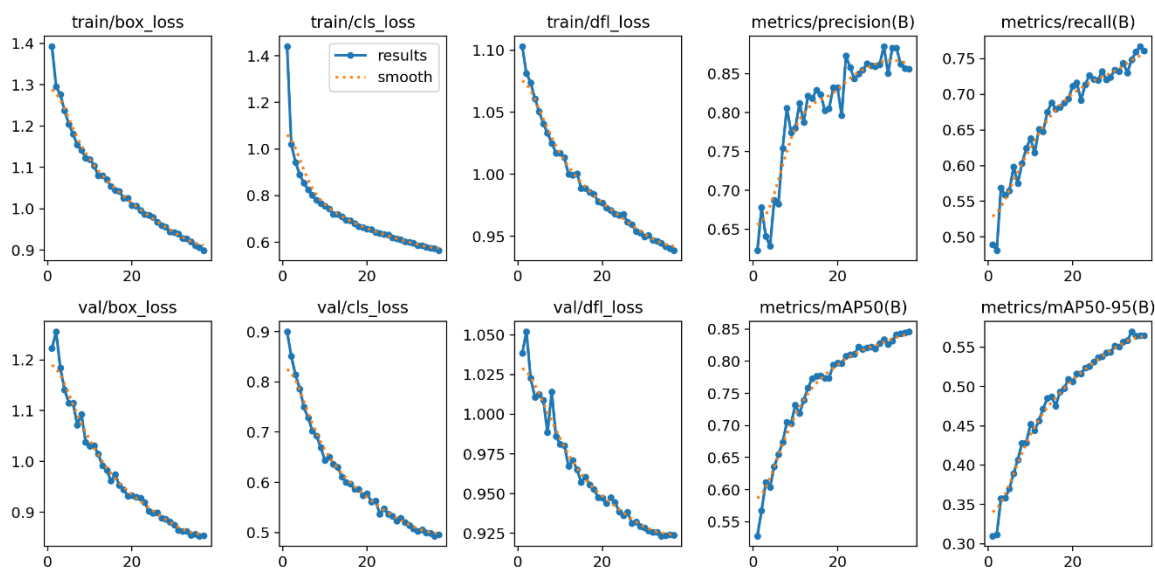
Hình 4.34. Hình ảnh sau khi qua mô hình Adapter và sinh ảnh

Ảnh trước đó:



Hình 4.35. Hình ảnh trước khi qua mô hình Adapter và sinh ảnh

Quá trình train YOLOv8n với tập dữ liệu mới được sinh ra:

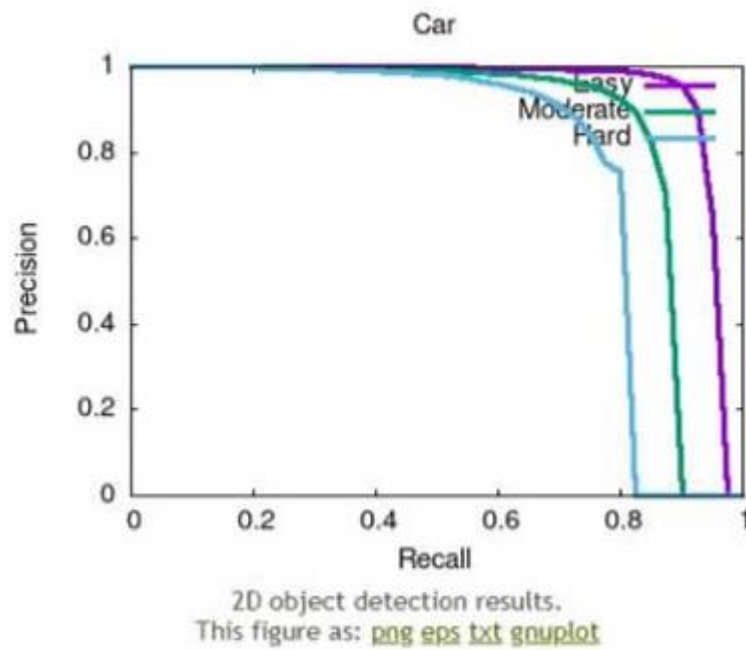


Hình 4.36. Chi tiết về hàm loss trong quá trình huấn luyện trên data RGB mới được sinh ra

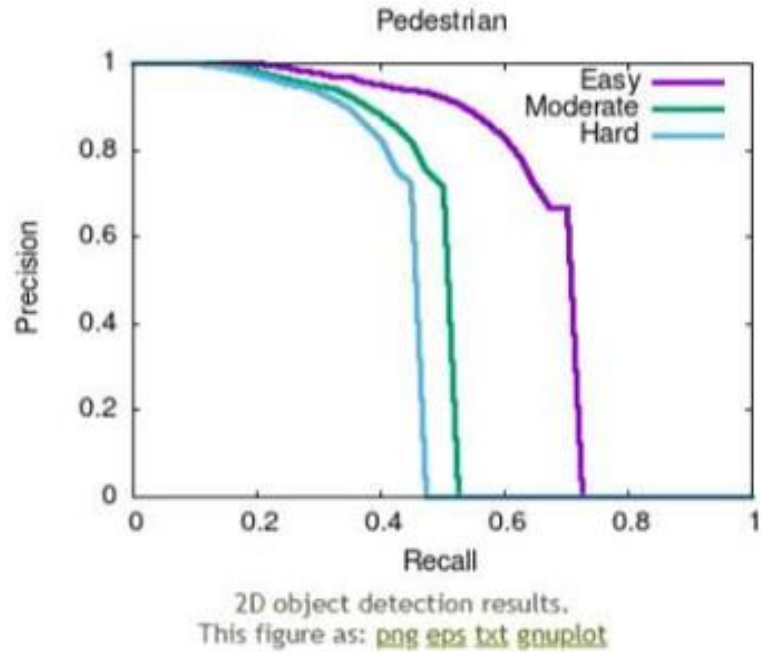
Kết quả khi train bộ dữ liệu mới với YOLOv8n và đánh giá trên benchmark của KITTI:

Bảng 4. 7. Bảng so sánh mAP của các lớp

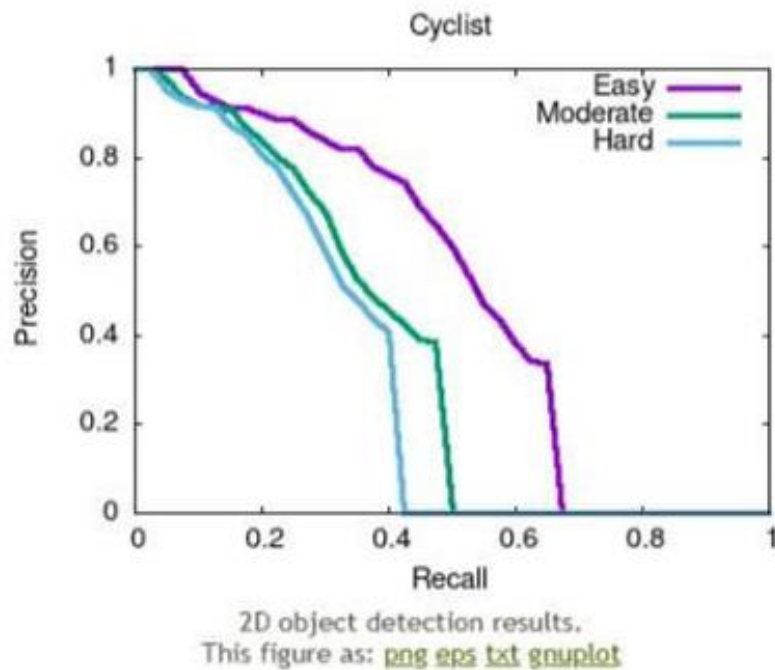
Benchmark	Easy	Moderate	Hard
Car	93.40%	84.98%	76.87%
Pedestrian	64.62%	46.50%	41.78%
Cyclist	48.59%	34.01%	29.56%



Hình 4.37. Kết quả đánh giá trên Benchmark KITTI của lớp Car



Hình 4.38. Kết quả giá trên Benchmark KITTI của lớp Pedestrian



Hình 4.39. Kết quả đánh giá trên Benchmark KITTI của lớp Cyclist

Nhận xét: Việc bổ sung thông tin chiều sâu (Depth) thông qua Adapter nhằm hòa trộn kênh RGBD về lại không gian RGB có mang lại một số cải thiện nhất định, tuy nhiên chưa thực sự hiệu quả rõ rệt trong việc nâng cao độ chính xác phát hiện vật thể, đặc biệt là các đối tượng nhỏ như là Pedestrian, Cyclist.

Dù Adapter chỉ gồm các tầng khá nông, việc ép Depth vào 3 kênh mà không làm suy giảm mAP của các lớp cho thấy Adapter vẫn thực hiện được vai trò duy trì thông tin. Tuy nhiên, do thiết kế đơn giản và hàm loss chưa khuyến khích khai thác đặc trưng từ Depth, mô hình có xu hướng phục hồi lại ảnh RGB ban đầu hơn là tận dụng chiều sâu một cách chiến lược.

Đáng chú ý, việc sử dụng các kỹ thuật data augmentation (đặc biệt là thay đổi độ sáng và tương phản) giúp mô hình học tốt hơn các đặc trưng thị giác, điều này cho thấy Depth không chỉ đóng vai trò cung cấp thông tin 3D, mà còn có tiềm năng tạo ra những biến thể bổ sung mang tính tăng cường học sâu.

Tổng thể, so với baseline YOLOv8n huấn luyện chỉ với RGB, phương pháp bổ sung Depth + Augmentation cho thấy tiềm năng cải thiện kết quả, nhất là với các lớp nhỏ như Cyclist, Pedestrian. Ngoài ra vẫn còn tiềm năng tối ưu hóa kiến trúc Adapter và cơ chế huấn luyện nhằm khai thác triệt để thông tin chiều sâu.

4.4.2 Nhánh thay đổi đầu vào của YOLOv8 từ 3 chanel thành 4 chanel

Mục tiêu:

- Mục tiêu của “Early Fusion” là kết hợp ngay từ đầu cả thông tin màu sắc và độ sâu, để mạng học đồng thời cả hai nguồn dữ liệu bằng cách mở rộng YOLOv8 (detect) để nhận **4 kênh đầu vào** (R, G, B, Depth) thay vì 3 kênh RGB như bình thường.

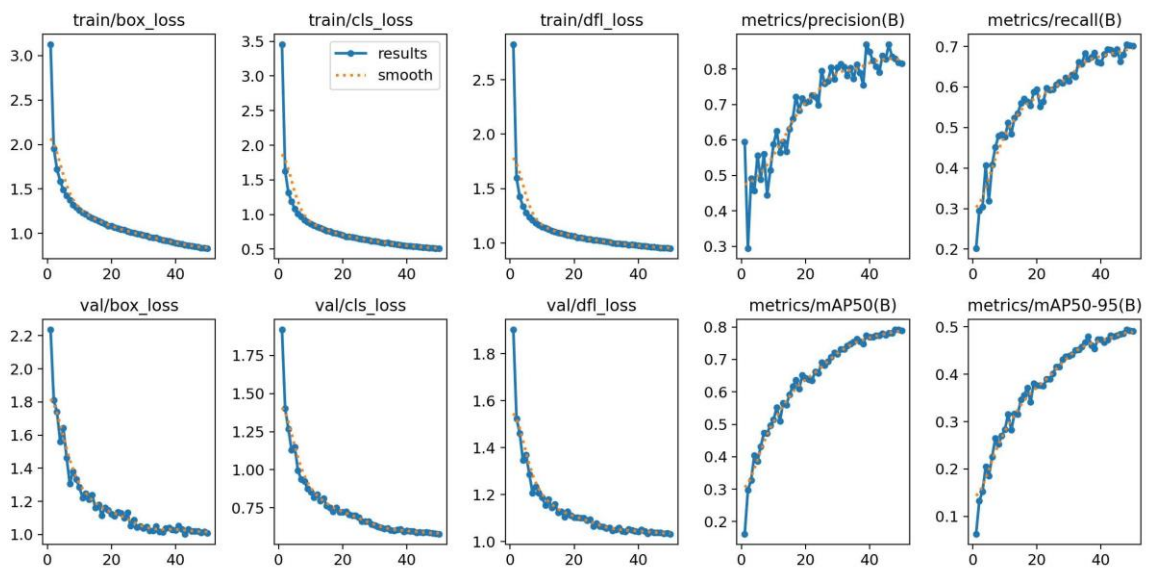
- Fine-tune từ weights COCO (3-kênh) → tận dụng ưu điểm pre-trained, đồng thời remap conv1 để chấp nhận depth.

Huấn luyện:

- Định nghĩa Dataset 4 kênh (rgbd_data.yaml)
- Custom Dataset Loader (4 kênh)
- Cấu hình Model (yolo_4chan-det.yaml)
- Tái sử dụng Pretrained Weights từ COCO:
 - Tạo model random-init
 - Tải checkpoint COCO (3 kênh)
 - Filter chỉ lấy weights có cùng kích thước:
 - Loại bỏ toàn bộ các key liên quan đến conv1

- Duyệt qua từng cặp (k, v) trong state_dict của checkpoint, Kiểm tra xem mô hình hiện tại có tham số cùng tên không.
 - Nếu không có thì bỏ qua.
 - Nếu có, so sánh kích thước của hai tensor và chỉ giữ lại khi kích thước trùng khớp.
- Remap conv1 (3→4 kênh): kênh depth sẽ được khởi tạo là trung bình của RGB

Quá trình train YOLOv8n:

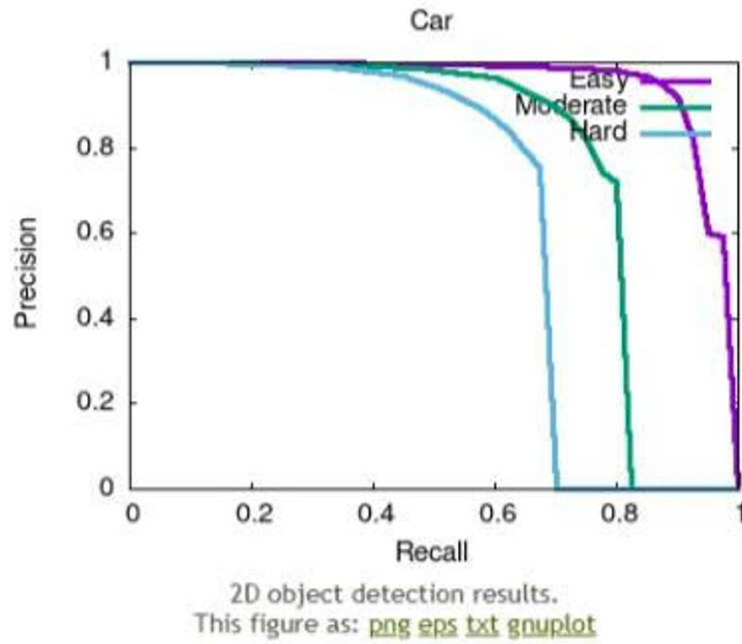


Hình 4.40. Kết quả train và val loss

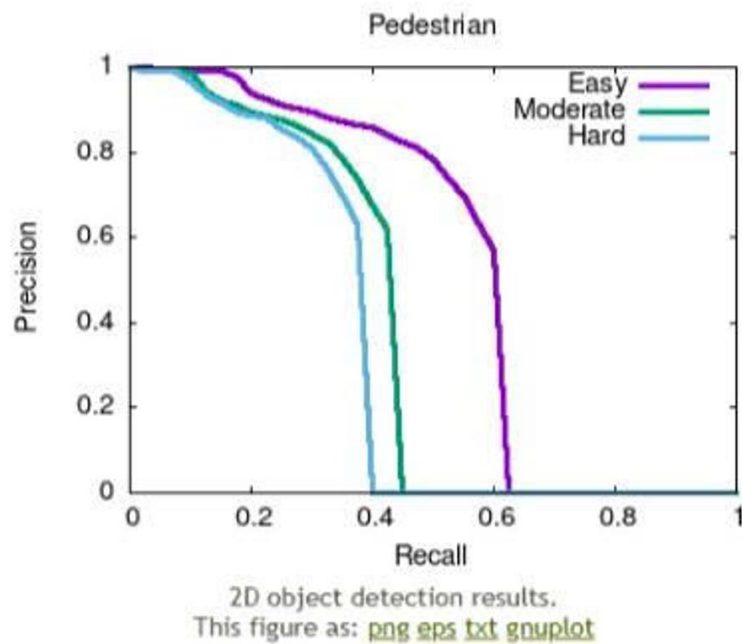
Kết quả khi train với YOLOv8n và đánh giá trên benchmark của KITTI:

Bảng 4.8. Bảng so sánh mAP của các lớp

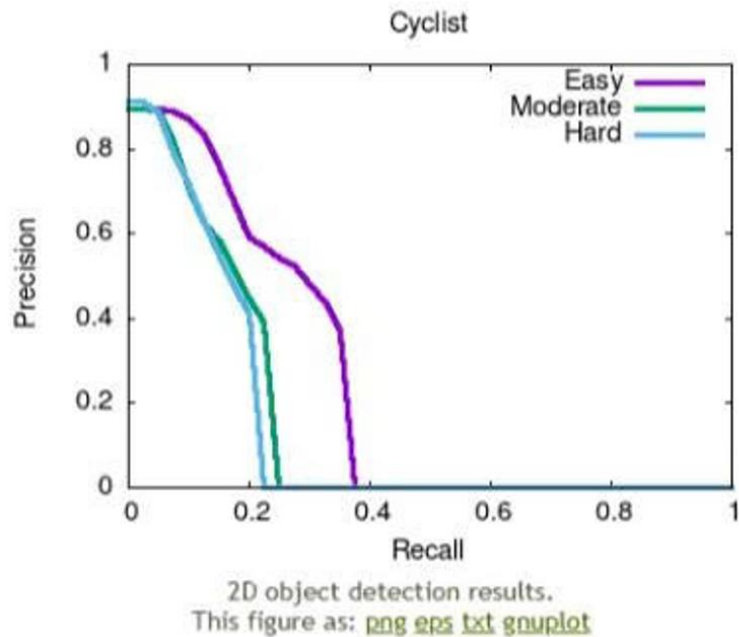
Benchmark	Easy	Moderate	Hard
Car	94.19%	76.56%	64.34%
Pedestrian	52.25%	36.90%	32.60%
Cyclist	23.33%	14.67%	13.41%



Hình 4.41. Kết quả đánh giá trên Benchmark KITTI của lớp Car



Hình 4.42. Kết quả đánh giá trên Benchmark KITTI của lớp Pedestrian



Hình 4.43. Kết quả đánh giá trên Benchmark KITTI của lớp Cyclist

Nhận xét: Việc bổ sung thêm thông tin chiều sâu (Depth) và ép vào ảnh RGB thành RGBD sau đó sử dụng ảnh này, thay đổi số chanel mà YOLOv8 nhận ban đầu 3 thành 4 kênh mà không thay đổi bất cứ kiến trúc mạng gì cũng như không thay đổi hàm mất mát là tiếp cận đơn giản nhưng chưa tối ưu mà chúng tôi nghĩ đến khiến cho việc mô hình nhận nhầm các Point Cloud trở thành nhiễu. Bởi vì mô hình gốc YOLOv8 không được thiết kế để xử lý tín hiệu không gian từ Point Cloud, nên depth channel có thể bị xem là một ảnh bình thường dẫn đến mô hình hiểu sai và bỏ qua đi các đặc trưng hình học quan trọng. Cụ thể ở các mẫu trường hợp Moderate hay Hard nơi yêu cầu phân biệt chính xác vật thể trong ánh sáng kém, che khuất, vật thể nhỏ thì depth không được xử lý đúng cách trong mô hình khiến gây nhiễu loạn trong quá trình học đặc trưng dẫn đến bị kém đi.

4.4.3 Tổng hợp kết quả

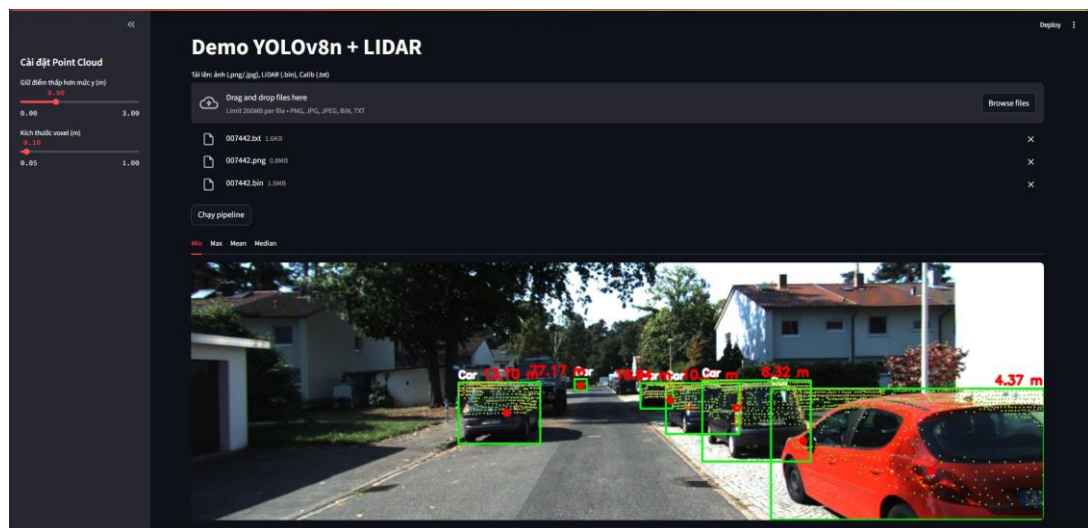
Bảng 4.9. Bảng tổng hợp kết quả mAP

Car			
	Easy	Moderate	Hard
YOLOv8n (RGB)	93.37%	84.93%	75.03%
YOLOv8n (RGBD)	94.19%	76.56%	64.34%
YOLOv8n Adapter RGBD	93.40%	84.98%	76.87%
Pedestrian			
	Easy	Moderate	Hard
YOLOv8n (RGB)	62.83%	46.37%	41.67%
YOLOv8n (RGBD)	52.25%	36.90%	32.60%
YOLOv8n Adapter RGBD	64.62%	46.50%	41.78%
Cyclist			
	Easy	Moderate	Hard
YOLOv8n (RGB)	49.42%	32.43%	28.80%
YOLOv8n (RGBD)	23.33%	14.67%	13.41%
YOLOv8n Adapter RGBD	48.59%	34.01%	29.56%

4.5 Web demo

Kết hợp kết quả phát hiện 2D của RGBD_Adapter với dữ liệu PC để tính khoảng cách cho từng đối tượng trong ảnh, sau đó hiển thị trực quan trên giao diện Streamlit. Với Pipeline xử lý tương tự như ở phần Thực nghiệm 3. Nhưng bổ sung 1 số phần nhỏ:

- Tạo một thanh sidebar để điều chỉnh 2 tham số:
 - ground_thresh: lọc điểm theo trục Y camera.
 - voxel_size: kích thước ô voxel để giảm mẫu.
- Tách thành 4 để so sánh trực quan hóa cho 4 giá trị Min, Max, Median và Mean.



Hình 4. 44. Website demo trên streamlit

KẾT LUẬN

Những điều đã làm được

Trong khuôn khổ đề tài này chúng tôi đã xây dựng và triển khai thành công một pipeline hoàn chỉnh cho bài toán phát hiện và định vị đối tượng trong không gian 3D bằng cách kết hợp dữ liệu ảnh RGB với Point Cloud thu được từ cảm biến LiDAR. Pipeline này không chỉ dừng lại ở việc phát hiện các đối tượng như xe hơi (Car), người đi bộ (Pedestrian), người đi xe đạp (Cyclist) mà còn tích hợp thêm bước ước lượng khoảng cách đến từng đối tượng được phát hiện thông qua việc chiếu dữ liệu Point Cloud lên ảnh và xử lý mối quan hệ hình học giữa bounding box và không gian 3D.

Bên cạnh đó, khóa luận này cũng đề xuất và triển khai một nhánh xử lý mở rộng theo hướng Early Fusion, trong đó ảnh RGB và độ sâu (Depth) được kết hợp lại thành RGBD để huấn luyện lại mô hình YOLOv8. Để thử nghiệm nhánh này thì chúng tôi đã thử nghiệm trên hai hướng tiếp cận khác nhau. Cụ thể là một hướng sẽ thông qua Adapter_RGBD – một mô hình mạng tích chập được chúng tôi thiết kế nhằm mục đích trích lọc thông tin Depth và ánh xạ về RGB. Kết quả cho thấy mô hình YOLO_RGBD cho hiệu suất phát hiện tốt hơn rõ rệt đặc biệt là đối với các đối tượng có kích thước nhỏ hoặc ở khoảng cách xa. Hướng còn lại là sử dụng tính linh hoạt kích cỡ kênh đầu vào của YOLOv8 để dùng Depth làm thông tin bổ trợ giúp mô hình tốt hơn. Tuy nhiên ở hướng này chúng tôi ghi nhận là chưa có cải thiện rõ ràng thậm chí mô hình còn nhận Depth làm nhiễu gây khó cho mô hình khi phát hiện đối tượng dẫn đến không cải thiện so với baseline RGB.

Ngoài ra, chúng tôi còn thực hiện hướng thực nghiệm mở rộng nhằm khai thác tối đa thông tin hình học từ Point Cloud. Thay vì sử dụng toàn bộ Point Cloud hoặc crop sơ theo bounding box 2D, chúng tôi áp dụng kỹ thuật crop theo phân đoạn đối tượng, sử dụng mask từ mô hình phân đoạn ảnh để chỉ giữ lại phần Point Cloud thuộc về đối tượng cần xét. Điều này giúp việc ước lượng khoảng cách chính xác và tinh gọn hơn.

Những điều chưa làm được

Mặc dù đã hoàn thành được một pipeline khả dụng cho bài toán phát hiện và định vị trong không gian 3D tuy nhiên khóa luận này vẫn còn những hạn chế nhất định.

Thứ nhất, mô hình chỉ dừng lại ở mức phát hiện đối tượng 2D trên ảnh RGB và ước lượng khoảng cách (Depth) từ dữ liệu LiDAR đã được căn chỉnh, chưa mở rộng sang phát hiện 3D bounding box. Bên cạnh đó chúng tôi trực tiếp sử dụng trực tiếp ma trận hiệu chỉnh có sẵn từ KITTI, nên chưa giải quyết được bài toán căn chỉnh tự động Camera – LiDAR.

Thứ hai, mặc dù nhóm đã crop Point Cloud theo mask phân đoạn đối tượng thay vì chỉ dùng bbox 2D, giúp loại bỏ nhiễu nền, nhưng pipeline hiện tại vẫn còn đơn giản. Cụ thể, giảm mật độ điểm chỉ dùng centroid sampling, chưa khai thác đặc trưng hình học bằng các mô hình như PointNet++. Việc ước lượng khoảng cách dựa trên tọa độ trung bình (X, Y, Z) kết hợp điểm gần nhất trong vùng mask (sử dụng Radius Nearest Neighbor), nhưng chưa tận dụng tốt cấu trúc 3D và không xử lý được các trường hợp vật thể nhỏ, thấp hoặc điểm thừa.

Thứ ba, phần đánh giá mô hình được thực hiện trực tiếp trên tập test của KITTI với ba lớp chính Car, Pedestrian, Cyclist. Tuy nhiên do đặc thù của KITTI là ghi nhận lại trong điều kiện ánh sáng và thời tiết tiêu chuẩn (ban ngày, trời quang), nên khả năng tổng quát hóa mô hình trong các điều kiện phức tạp hơn như mưa gió, sương mù, đêm tối thì chưa được kiểm chứng. Ngoài ra phần so sánh giữa các mô hình YOLO với ảnh RGB (baseline) và YOLO với RGBD hiện chỉ dừng lại ở mức độ thử nghiệm sơ bộ, chưa tối ưu hóa các siêu tham số hoặc fine tuning đầy đủ do còn hạn chế về mặt tài nguyên huấn luyện.

Cuối cùng khóa luận chưa tận dụng được hết khả năng kết hợp dữ liệu ảnh RGB và Point Cloud theo hướng mid- fusion, late- fusion. Việc tích hợp Depth vào YOLO chủ yếu theo dạng early- fusion đơn giản (ép kênh), chưa sử dụng các module học sâu đa modal để tận dụng triệt để ưu điểm của từng nguồn dữ liệu. Bên cạnh đó chưa phân tích sâu vào mối quan hệ giữa chất lượng Depth map và độ chính xác trong phát hiện đối tượng, chưa đào sâu vào hướng giải quyết trường hợp bị thiếu Depth. Đây là những khía cạnh quan trọng cần nghiên cứu sâu thêm về sau.

Hướng phát triển

Mặc dù đề tài đã giải quyết một số vấn đề liên quan đến độ sâu thực sự của đối tượng bằng cách khai thác dữ liệu Point Cloud. Tuy nhiên vẫn còn rất nhiều điểm hạn chế cần những hướng phát triển khác để đảm bảo độ chính xác này tốt hơn ngoài cũng để tận dụng Depth của Point Cloud tốt hơn ví dụ như giúp tăng độ chính xác khi phát hiện vật thể hơn thì chúng tôi đề xuất các hướng phát triển tiếp theo như sau:

Thứ nhất, phát hiện bounding box 3D thay vì 2D. Mặc dù hiện tại mô hình có thể ước lượng khoảng cách đối tượng bằng dữ liệu LiDAR, nhưng chỉ là trên hộp 2D. Trong tương lai sẽ mở rộng pipeline để dự đoán trực tiếp hộp 3D bằng cách kết hợp ảnh RGB với Point Cloud. Việc tận dụng đồng thời hai nguồn dữ liệu này sẽ giúp mô hình xác định chính xác vị trí, kích thước và hình dạng thực của vật thể trong không gian 3D.

Thứ hai, căn chỉnh tự động. Trong đề tài này chúng tôi chỉ sử dụng ma trận hiệu chỉnh cố định từ bộ dữ liệu KITTI. Nên khi áp dụng sang một camera khác hay một LiDAR khác, việc căn chỉnh camera và LiDAR có thể sẽ bị thay đổi. Do đó hướng nghiên cứu tiếp theo mà nhóm muốn nhắm đến là việc tự động căn chỉnh giữa các cảm biến để thích ứng linh hoạt hơn.

Thứ ba, mặc dù nhóm đã sử dụng phân đoạn ảnh (YOLOv8.seg) để crop Point Cloud chính xác hơn và loại bỏ nhiễu nền, pipeline hiện tại vẫn chưa khai thác tốt đặc trưng hình học không gian 3D. Việc giảm mật độ điểm chỉ dùng centroid sampling, chưa tích hợp các mô hình học sâu như PointNet++ để học đặc trưng từ chính cấu trúc 3D. Hướng phát triển tiếp theo là bổ sung phân đoạn trực tiếp trên Point Cloud nhằm tăng độ chi tiết và độ tin cậy trong các tình huống phức tạp.

Thứ tư, cải thiện khả năng tổng quát hóa của mô hình. Do việc dữ liệu hiện tại có điều kiện ánh sáng và thời tiết lý tưởng, mô hình YOLO_RGBD chưa chứng minh được hiệu quả trong môi trường thực tế đa dạng hơn. Vì vậy sẽ tăng cường dữ liệu như thêm dữ liệu từ các nguồn khác như nuScenes, Waymo để đánh giá trong điều kiện khắc nghiệt hơn.

Thứ năm, phát triển theo hướng mid, late- fusion. Do đây chỉ là thử nghiệm của chúng tôi về hiệu quả của Point Cloud thu được từ LiDAR nhưng do chỉ là ép kênh đầu vào thôi chưa tận dụng hết các thông tin của Depth như là không gian. Nên chúng

tôi đề xuất các hướng nghiên cứu mạnh hơn để trích xuất nhiều thông tin từ Point Cloud nhất như Mid- Fusion nơi mà các đặc trưng của hai luồng dữ liệu sẽ được kết hợp lại. Hay Late- Fusion nơi mà kết quả đầu ra của luồng dữ liệu sẽ được kết hợp lại từ đó cho độ chính xác khi phát hiện đối tượng tốt hơn.

Thứ sáu, phát triển module xử lý thiếu Depth. Trong quá trình thực nghiệm thì chúng tôi phát hiện ra rằng nhiều vùng ảnh có thể không có dữ liệu LiDAR tương ứng do vùng quét hạn chế hay do vật thể bị cản gây ra khiếm khuyết LiDAR. Hướng mở rộng là phát triển module Depth completion.

Tổng kết lại, các hướng phát triển nêu trên không chỉ khắc phục những hạn chế hiện tại mà còn mở ra tiềm năng khai thác triệt để không gian 3D từ dữ liệu Point Cloud, từ đó nâng cao độ chính xác, tính tổng quát và khả năng thích ứng của mô hình trong các môi trường thực tế phức tạp hơn.

TÀI LIỆU THAM KHẢO

- [1] F. P. o. C. Vision, "Overview | Image Formation," Youtuber, 1 3 2021. [Online]. Available:
https://www.youtube.com/watch?v=_QjxbQKY4ds&list=PL2zRqk16wsdr9X5rgF-d0pkzPdkHZ4KiT. [Accessed 27 6 2025].
- [2] S. Mallick, "Geometry of Image Formation," LearnOpenCV, 20 2 2020. [Online]. Available: <https://learnopencv.com/geometry-of-image-formation/>. [Accessed 25 6 2025].
- [3] LearnOpenCV, "camera calibration," LearnOpenCV, 2022. [Online]. Available: <https://learnopencv.com/category/camera-calibration/>. [Accessed 2 7 2025].
- [4] F. P. o. C. Vision, "Overview | Camera Calibration," Youtube, 19 4 2021. [Online]. Available: <https://www.youtube.com/watch?v=S-UHiFsn-GI&list=PL2zRqk16wsdoCCLpou-dGo7QQNks1Ppzo>. [Accessed 1 7 2025].
- [5] doxygen, "Camera Calibration," OpenCV, 28 6 2025. [Online]. Available: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html. [Accessed 16 6 2025].
- [6] T. Batpurev, "Stereo Camera Calibration and Triangulation with OpenCV and Python," temugeb, 2 2 2021. [Online]. Available: <https://temugeb.github.io/opencv/python/2021/02/02/stereo-camera-calibration-and-triangulation.html>. [Accessed 24 6 2025].
- [7] I. Boles, "GEOG 232, Week 8 Lecture: LiDAR, Part 1," 13 3 2022. [Online]. Available: <https://www.youtube.com/watch?v=p9F0NhV8EFs>. [Accessed 2025 6 15].
- [8] Dathop, "LiDAR là gì? LiDAR hoạt động như thế nào?," dathop, [Online]. Available: <https://dathop.com.vn/post/lidar-la-gi-lidar-hoat-dong-nhu-the-nao.html>. [Accessed 2025 6 15].
- [9] Synopsys, "What Is LiDAR?," 31 7 2023. [Online]. Available: <https://www.synopsys.com/glossary/what-is-lidar.html>. [Accessed 2025 5 17].
- [10] L. A. Wasser, "The Basics of LiDAR - Light Detection and Ranging - Remote Sensing," Neonscience, 13 9 2024. [Online]. Available:

- <https://www.neonscience.org/resources/learning-hub/tutorials/lidar-basics>.
[Accessed 20 6 2025].
- [11] Open3D, "Point Cloud," Open3D, 27 1 2024. [Online]. Available: <https://www.open3d.org/docs/latest/tutorial/Basic/pointcloud.html#Voxel-downsampling>. [Accessed 28 6 2025].
- [12] Open3D, "Open3D documentation," Open3D, 2024. [Online]. Available: <https://www.open3d.org/>. [Accessed 19 6 2025].
- [13] Open3D, "Basic," Open3D, 3 8 2023. [Online]. Available: <https://www.open3d.org/docs/latest/tutorial/Basic/index.html>. [Accessed 2025 6 17].
- [14] Open3D, "Point cloud outlier removal," Open3D, 2024. [Online]. Available: https://www.open3d.org/docs/latest/tutorial/Advanced/pointcloud_outlier_removal.html. [Accessed 2025 6 26].
- [15] Open3D, "KDTree," KDTree, 2022. [Online]. Available: <https://www.open3d.org/docs/latest/tutorial/Basic/kdtree.html>. [Accessed 26 6 2025].
- [16] Open3D, "octree," octree, 2023. [Online]. Available: <https://www.open3d.org/docs/release/tutorial/geometry/octree.html>. [Accessed 5 6 2025].
- [17] Open3D, "Advanced," Advanced, 2020. [Online]. Available: <https://www.open3d.org/docs/latest/tutorial/Advanced/index.html>. [Accessed 3 6 2025].
- [18] V. Hoàng, "Tìm hiểu về YOLO trong bài toán real-time object detection," Viblo, 16 11 2019. [Online]. Available: <https://viblo.asia/p/tim-hieu-ve-yolo-trong-bai-toan-real-time-object-detection-yMnKMdvr57P>. [Accessed 3 6 2025].
- [19] Juan Terven, Diana Cordova-Esparza, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," arxiv, 4 2 2024. [Online]. Available: <https://arxiv.org/abs/2304.00501>. [Accessed 27 6 2025].

- [20] D. Warfield, "YOLO — Intuitively and Exhaustively Explained," medium, 31 5 2024. [Online]. Available: <https://medium.com/intuitively-and-exhaustively-explained/yolo-intuitively-and-exhaustively-explained-83143925c7a9>. [Accessed 5 6 2025].
- [21] B. S. S. soumyadip, "Mastering All YOLO Models from YOLOv1 to YOLOv12: Papers Explained (2025)," LearnOpenCV, 26 12 2023. [Online]. Available: <https://learnopencv.com/mastering-all-yolo-models/>. [Accessed 25 6 2025].
- [22] M. Shenoda, "Real-time Object Detection: YOLOv1 Re-Implementation in PyTorch," arxiv, 28 5 2023. [Online]. Available: <https://arxiv.org/abs/2305.17786>. [Accessed 4 7 2025].
- [23] T. Keldenich, "How to make Bounding Boxes in Python – Best function," inside-machinelearning, 18 1 2023. [Online]. Available: <https://inside-machinelearning.com/en/bounding-boxes-python-function/>. [Accessed 5 6 2025].
- [24] F. Jacob Solawetz, "What is YOLOv8? A Complete Guide," roboflow, 23 10 2024. [Online]. Available: <https://blog.roboflow.com/what-is-yolov8/>. [Accessed 3 7 2025].
- [25] T. Keldenich, "YOLOv8 – How to use it? Best Tutorial," Inside machine Learning, 18 1 2023. [Online]. Available: <https://inside-machinelearning.com/en/yolov8-how-to-use/#boxzilla-13443>. [Accessed 5 6 2025].
- [26] ultralytics, "Ultralytics Docs," github, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>. [Accessed 31 6 2025].
- [27] D. P. Hidayatullah, "YOLOv8 Architecture Detailed Explanation - A Complete Breakdown," Youtube, 28 10 2023. [Online]. Available: <https://www.youtube.com/watch?v=HQXhDO7COj8>. [Accessed 5 7 2025].

